

Manual del Programador Competitivo

Antti Laaksonen

Traducción al español

Borrador 5 de agosto de 2026

Índice general

Prefacio	v
I Técnicas básicas	1
1. Introducción	3
1.1. Lenguajes de programación	3
1.2. Entrada y salida	4
1.3. Trabajar con números	6
1.4. Acortar el código	8
1.5. Matemáticas	10
1.6. Concursos y recursos	15
2. Complejidad temporal	19
2.1. Reglas de cálculo	19
2.2. Clases de complejidad	22
2.3. Estimar la eficiencia	23
2.4. Suma máxima de subarreglo	24
3. Ordenamiento	27
3.1. Teoría del ordenamiento	27
3.2. Ordenamiento en C++	32
3.3. Búsqueda binaria	34
4. Estructuras de datos	39
4.1. Arreglos dinámicos	39
4.2. Estructuras de conjunto	41
4.3. Estructuras de mapa	42
4.4. Iteradores y rangos	43
4.5. Otras estructuras	46
4.6. Comparación con el ordenamiento	50
5. Búsqueda completa	53
5.1. Generar subconjuntos	53
5.2. Generar permutaciones	55
5.3. Backtracking	56
5.4. Podar la búsqueda	58
5.5. Encuentro en el medio	61

II Algoritmos de grafos	63
Bibliografía	65

Prefacio

El propósito de este libro es ofrecerte una introducción completa a la programación competitiva. Se asume que ya conoces los fundamentos de la programación, pero no se necesita ninguna experiencia previa en programación competitiva.

El libro está especialmente pensado para estudiantes que quieren aprender algoritmos y posiblemente participar en la Olimpiada Internacional de Informática (IOI) o en el Concurso Internacional de Programación Colegial (ICPC). Por supuesto, el libro también es adecuado para cualquier otra persona interesada en la programación competitiva.

Se necesita mucho tiempo para llegar a ser un buen programador competitivo, pero también es una oportunidad para aprender mucho. Puedes estar seguro de que obtendrás una buena comprensión general de los algoritmos si dedicas tiempo a leer el libro, resolver problemas y participar en concursos.

El libro está en desarrollo continuo. Siempre puedes enviar comentarios sobre el libro a `ahslaaks@cs.helsinki.fi`.

Helsinki, agosto de 2019
Antti Laaksonen

Parte I

Técnicas básicas

Capítulo 1

Introducción

La programación competitiva combina dos temas: (1) el diseño de algoritmos y (2) la implementación de algoritmos.

El **diseño de algoritmos** consiste en la resolución de problemas y el pensamiento matemático. Se requieren habilidades para analizar problemas y resolverlos de forma creativa. Un algoritmo para resolver un problema tiene que ser a la vez correcto y eficiente, y el núcleo del problema suele consistir en idear un algoritmo eficiente.

El conocimiento teórico de los algoritmos es importante para los programadores competitivos. Por lo general, la solución de un problema es una combinación de técnicas bien conocidas e ideas nuevas. Las técnicas que aparecen en la programación competitiva también constituyen la base de la investigación científica de los algoritmos.

La **implementación de algoritmos** requiere buenas habilidades de programación. En la programación competitiva, las soluciones se califican probando un algoritmo implementado mediante un conjunto de casos de prueba. Por lo tanto, no basta con que la idea del algoritmo sea correcta, sino que la implementación también tiene que ser correcta.

Un buen estilo de código en los concursos es directo y conciso. Los programas deben escribirse con rapidez, porque no hay mucho tiempo disponible. A diferencia de la ingeniería de software tradicional, los programas son cortos (normalmente a lo sumo unos pocos cientos de líneas de código), y no necesitan mantenerse después del concurso.

1.1. Lenguajes de programación

En la actualidad, los lenguajes de programación más populares usados en los concursos son C++, Python y Java. Por ejemplo, en el Google Code Jam 2017, entre los mejores 3.000 participantes, el 79% usó C++, el 16% usó Python y el 8% usó Java [29]. Algunos participantes también usaron varios lenguajes.

Mucha gente piensa que C++ es la mejor opción para un programador competitivo, y C++ casi siempre está disponible en los sistemas de concursos. Los beneficios de usar C++ son que es un lenguaje muy eficiente y su biblioteca estándar contiene una gran colección de estructuras de datos y algoritmos.

Por otro lado, es bueno dominar varios lenguajes y comprender sus fortalezas. Por ejemplo, si se necesitan enteros muy grandes en el problema, Python puede ser una buena opción, porque contiene operaciones integradas para realizar cálculos con enteros grandes. Aun así, la mayoría de los problemas en los concursos de programación se plantean de modo que usar un lenguaje de programación específico no sea una ventaja injusta.

Todos los programas de ejemplo de este libro están escritos en C++, y a menudo se usan las estructuras de datos y los algoritmos de la biblioteca estándar. Los programas siguen el estándar C++11, que puede usarse en la mayoría de los concursos hoy en día. Si todavía no sabes programar en C++, ahora es un buen momento para empezar a aprender.

Plantilla de código C++

Una plantilla de código C++ típica para programación competitiva tiene este aspecto:

```
#include <bits/stdc++.h>

using namespace std;

int main() {
    // solution comes here
}
```

La línea `#include` al principio del código es una característica del compilador `g++` que nos permite incluir toda la biblioteca estándar. Así, no es necesario incluir por separado bibliotecas como `iostream`, `vector` y `algorithm`, sino que están disponibles automáticamente.

La línea `using` declara que las clases y funciones de la biblioteca estándar pueden usarse directamente en el código. Sin la línea `using` tendríamos que escribir, por ejemplo, `std::cout`, pero ahora basta con escribir `cout`.

El código puede compilarse usando el siguiente comando:

```
g++ -std=c++11 -O2 -Wall test.cpp -o test
```

Este comando produce un archivo binario `test` a partir del código fuente `test.cpp`. El compilador sigue el estándar C++11 (`-std=c++11`), optimiza el código (`-O2`) y muestra advertencias sobre posibles errores (`-Wall`).

1.2. Entrada y salida

En la mayoría de los concursos, se usan flujos estándar para leer la entrada y escribir la salida. En C++, los flujos estándar son `cin` para la entrada y `cout` para la salida. Además, pueden usarse las funciones de C `scanf` y `printf`.

La entrada del programa normalmente consta de números y cadenas separados por espacios y saltos de línea. Pueden leerse del flujo `cin` de la siguiente

manera:

```
int a, b;
string x;
cin >> a >> b >> x;
```

Este tipo de código siempre funciona, suponiendo que haya al menos un espacio o salto de línea entre cada elemento de la entrada. Por ejemplo, el código anterior puede leer ambas de las siguientes entradas:

```
123 456 monkey
```

```
123    456
monkey
```

El flujo cout se usa para la salida de la siguiente manera:

```
int a = 123, b = 456;
string x = "monkey";
cout << a << " " << b << " " << x << "\n";
```

La entrada y la salida a veces son un cuello de botella en el programa. Las siguientes líneas al principio del código hacen que la entrada y la salida sean más eficientes:

```
ios::sync_with_stdio(0);
cin.tie(0);
```

Nota que el salto de línea "\n" funciona más rápido que endl, porque endl siempre provoca una operación de vaciado (flush).

Las funciones de C scanf y printf son una alternativa a los flujos estándar de C++. Suelen ser un poco más rápidas, pero también son más difíciles de usar. El siguiente código lee dos enteros de la entrada:

```
int a, b;
scanf("%d %d", &a, &b);
```

El siguiente código imprime dos enteros:

```
int a = 123, b = 456;
printf("%d %d\n", a, b);
```

A veces el programa debe leer una línea completa de la entrada, que posiblemente contenga espacios. Esto puede lograrse usando la función getline:

```
string s;
getline(cin, s);
```

Si la cantidad de datos es desconocida, el siguiente bucle es útil:

```
while (cin >> x) {  
    // code  
}
```

Este bucle lee elementos de la entrada uno tras otro, hasta que no hay más datos disponibles en la entrada.

En algunos sistemas de concursos, se usan archivos para la entrada y la salida. Una solución sencilla para esto es escribir el código como de costumbre usando flujos estándar, pero añadir las siguientes líneas al principio del código:

```
freopen("input.txt", "r", stdin);  
freopen("output.txt", "w", stdout);
```

Después de esto, el programa lee la entrada del archivo "input.txt" y escribe la salida en el archivo "output.txt".

1.3. Trabajar con números

Enteros

El tipo entero más usado en programación competitiva es `int`, que es un tipo de 32 bits con un rango de valores de $-2^{31} \dots 2^{31} - 1$ o aproximadamente $-2 \cdot 10^9 \dots 2 \cdot 10^9$. Si el tipo `int` no es suficiente, puede usarse el tipo de 64 bits `long long`. Tiene un rango de valores de $-2^{63} \dots 2^{63} - 1$ o aproximadamente $-9 \cdot 10^{18} \dots 9 \cdot 10^{18}$.

El siguiente código define una variable `long long`:

```
long long x = 123456789123456789LL;
```

El sufijo `LL` significa que el tipo del número es `long long`.

Un error común al usar el tipo `long long` es que el tipo `int` se sigue usando en algún lugar del código. Por ejemplo, el siguiente código contiene un error sutil:

```
int a = 123456789;  
long long b = a*a;  
cout << b << "\n"; // -1757895751
```

Aunque la variable `b` es de tipo `long long`, ambos números en la expresión `a*a` son de tipo `int` y el resultado también es de tipo `int`. Por eso, la variable `b` contendrá un resultado erróneo. El problema puede resolverse cambiando el tipo de `a` a `long long` o cambiando la expresión a `(long long)a*a`.

Normalmente los problemas de los concursos se plantean de modo que el tipo `long long` sea suficiente. Aun así, es bueno saber que el compilador `g++` también proporciona un tipo de 128 bits `__int128_t` con un rango de valores de $-2^{127} \dots 2^{127} - 1$ o aproximadamente $-10^{38} \dots 10^{38}$. Sin embargo, este tipo no está disponible en todos los sistemas de concursos.

Aritmética modular

Denotamos por $x \bmod m$ el residuo cuando x se divide entre m . Por ejemplo, $17 \bmod 5 = 2$, porque $17 = 3 \cdot 5 + 2$.

A veces, la respuesta a un problema es un número muy grande pero basta con darla "módulo m ", es decir, el residuo cuando la respuesta se divide entre m (por ejemplo, "módulo $10^9 + 7$ "). La idea es que aunque la respuesta real sea muy grande, basta con usar los tipos `int` y `long long`.

Una propiedad importante del residuo es que en la suma, la resta y la multiplicación, el residuo puede tomarse antes de la operación:

$$\begin{aligned}(a + b) \bmod m &= (a \bmod m + b \bmod m) \bmod m \\(a - b) \bmod m &= (a \bmod m - b \bmod m) \bmod m \\(a \cdot b) \bmod m &= (a \bmod m \cdot b \bmod m) \bmod m\end{aligned}$$

Así, podemos tomar el residuo después de cada operación y los números nunca se harán demasiado grandes.

Por ejemplo, el siguiente código calcula $n!$, el factorial de n , módulo m :

```
long long x = 1;
for (int i = 2; i <= n; i++) {
    x = (x*i)%m;
}
cout << x%m << "\n";
```

Normalmente queremos que el residuo esté siempre entre $0 \dots m - 1$. Sin embargo, en C++ y otros lenguajes, el residuo de un número negativo es cero o negativo. Una forma sencilla de asegurarse de que no haya residuos negativos es calcular primero el residuo como de costumbre y luego sumar m si el resultado es negativo:

```
x = x%m;
if (x < 0) x += m;
```

Sin embargo, esto solo es necesario cuando hay restas en el código y el residuo puede volverse negativo.

Números de punto flotante

Los tipos de punto flotante habituales en programación competitiva son el `double` de 64 bits y, como extensión en el compilador `g++`, el `long double` de 80 bits. En la mayoría de los casos, `double` es suficiente, pero `long double` es más preciso.

La precisión requerida de la respuesta suele indicarse en el enunciado del problema. Una forma sencilla de mostrar la respuesta es usar la función `printf` e indicar el número de posiciones decimales en la cadena de formato. Por ejemplo, el siguiente código imprime el valor de x con 9 posiciones decimales:

```
printf("%.9f\n", x);
```

Una dificultad al usar números de punto flotante es que algunos números no pueden representarse con exactitud como números de punto flotante, y habrá errores de redondeo. Por ejemplo, el resultado del siguiente código es sorprendente:

```
double x = 0.3*3+0.1;
printf("%.20f\n", x); // 0.99999999999999988898
```

Debido a un error de redondeo, el valor de x es un poco menor que 1, mientras que el valor correcto sería 1.

Es arriesgado comparar números de punto flotante con el operador `==`, porque es posible que los valores debieran ser iguales pero no lo sean debido a errores de precisión. Una mejor forma de comparar números de punto flotante es suponer que dos números son iguales si la diferencia entre ellos es menor que ϵ , donde ϵ es un número pequeño.

En la práctica, los números pueden compararse de la siguiente manera ($\epsilon = 10^{-9}$):

```
if (abs(a-b) < 1e-9) {
    // a and b are equal
}
```

Nota que aunque los números de punto flotante son inexactos, los enteros hasta cierto límite todavía pueden representarse con exactitud. Por ejemplo, usando `double`, es posible representar con exactitud todos los enteros cuyo valor absoluto sea a lo sumo 2^{53} .

1.4. Acortar el código

El código corto es ideal en programación competitiva, porque los programas deben escribirse lo más rápido posible. Por eso, los programadores competitivos a menudo definen nombres más cortos para los tipos de datos y otras partes del código.

Nombres de tipos

Usando el comando `typedef` es posible dar un nombre más corto a un tipo de dato. Por ejemplo, el nombre `long long` es largo, así que podemos definir un nombre más corto `ll`:

```
typedef long long ll;
```

Después de esto, el código

```
long long a = 123456789;
long long b = 987654321;
cout << a*b << "\n";
```

puede acortarse de la siguiente manera:

```
ll a = 123456789;
ll b = 987654321;
cout << a*b << "\n";
```

El comando typedef también puede usarse con tipos más complejos. Por ejemplo, el siguiente código da el nombre vi a un vector de enteros y el nombre pi a un par que contiene dos enteros.

```
typedef vector<int> vi;
typedef pair<int,int> pi;
```

Macros

Otra forma de acortar el código es definir **macros**. Una macro significa que ciertas cadenas del código se cambiarán antes de la compilación. En C++, las macros se definen usando la palabra clave #define.

Por ejemplo, podemos definir las siguientes macros:

```
#define F first
#define S second
#define PB push_back
#define MP make_pair
```

Después de esto, el código

```
v.push_back(make_pair(y1,x1));
v.push_back(make_pair(y2,x2));
int d = v[i].first+v[i].second;
```

puede acortarse de la siguiente manera:

```
v.PB(MP(y1,x1));
v.PB(MP(y2,x2));
int d = v[i].F+v[i].S;
```

Una macro también puede tener parámetros lo que hace posible acortar bucles y otras estructuras. Por ejemplo, podemos definir la siguiente macro:

```
#define REP(i,a,b) for (int i = a; i <= b; i++)
```

Después de esto, el código

```
for (int i = 1; i <= n; i++) {
    search(i);
}
```

puede acortarse de la siguiente manera:

```
REP(i,1,n) {  
    search(i);  
}
```

A veces las macros provocan errores que pueden ser difíciles de detectar. Por ejemplo, considera la siguiente macro que calcula el cuadrado de un número:

```
#define SQ(a) a*a
```

Esta macro *no* siempre funciona como se espera. Por ejemplo, el código

```
cout << SQ(3+3) << "\n";
```

corresponde al código

```
cout << 3+3*3+3 << "\n"; // 15
```

Una mejor versión de la macro es la siguiente:

```
#define SQ(a) (a)*(a)
```

Ahora el código

```
cout << SQ(3+3) << "\n";
```

corresponde al código

```
cout << (3+3)*(3+3) << "\n"; // 36
```

1.5. Matemáticas

Las matemáticas desempeñan un papel importante en la programación competitiva, y no es posible llegar a ser un programador competitivo exitoso sin tener buenas habilidades matemáticas. Esta sección analiza algunos conceptos y fórmulas matemáticas importantes que se necesitarán más adelante en el libro.

Fórmulas de sumas

Cada suma de la forma

$$\sum_{x=1}^n x^k = 1^k + 2^k + 3^k + \dots + n^k,$$

donde k es un entero positivo, tiene una fórmula cerrada que es un polinomio de grado $k + 1$. Por ejemplo¹,

$$\sum_{x=1}^n x = 1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

y

$$\sum_{x=1}^n x^2 = 1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}.$$

Una **progresión aritmética** es una sucesión de números donde la diferencia entre dos números consecutivos cualesquiera es constante. Por ejemplo,

$$3, 7, 11, 15$$

es una progresión aritmética con constante 4. La suma de una progresión aritmética puede calcularse usando la fórmula

$$\underbrace{a + \dots + b}_{n \text{ números}} = \frac{n(a+b)}{2}$$

donde a es el primer número, b es el último número y n es la cantidad de números. Por ejemplo,

$$3 + 7 + 11 + 15 = \frac{4 \cdot (3 + 15)}{2} = 36.$$

La fórmula se basa en el hecho de que la suma consta de n números y el valor de cada número es $(a + b)/2$ en promedio.

Una **progresión geométrica** es una sucesión de números donde la razón entre dos números consecutivos cualesquiera es constante. Por ejemplo,

$$3, 6, 12, 24$$

es una progresión geométrica con constante 2. La suma de una progresión geométrica puede calcularse usando la fórmula

$$a + ak + ak^2 + \dots + b = \frac{bk - a}{k - 1}$$

donde a es el primer número, b es el último número y la razón entre números consecutivos es k . Por ejemplo,

$$3 + 6 + 12 + 24 = \frac{24 \cdot 2 - 3}{2 - 1} = 45.$$

Esta fórmula puede deducirse de la siguiente manera. Sea

$$S = a + ak + ak^2 + \dots + b.$$

Multiplicando ambos lados por k , obtenemos

$$kS = ak + ak^2 + ak^3 + \dots + bk,$$

¹ Existe incluso una fórmula general para tales sumas, llamada **fórmula de Faulhaber**, pero es demasiado compleja para presentarla aquí.

y resolviendo la ecuación

$$kS - S = bk - a$$

se obtiene la fórmula.

Un caso especial de la suma de una progresión geométrica es la fórmula

$$1 + 2 + 4 + 8 + \dots + 2^{n-1} = 2^n - 1.$$

Una **suma armónica** es una suma de la forma

$$\sum_{x=1}^n \frac{1}{x} = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n}.$$

Una cota superior para una suma armónica es $\log_2(n) + 1$. En efecto, podemos modificar cada término $1/k$ de modo que k se convierta en la potencia de dos más cercana que no exceda a k . Por ejemplo, cuando $n = 6$, podemos estimar la suma de la siguiente manera:

$$1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \frac{1}{6} \leq 1 + \frac{1}{2} + \frac{1}{2} + \frac{1}{4} + \frac{1}{4} + \frac{1}{4}.$$

Esta cota superior consta de $\log_2(n) + 1$ partes (1 , $2 \cdot 1/2$, $4 \cdot 1/4$, etc.), y el valor de cada parte es a lo sumo 1 .

Teoría de conjuntos

Un **conjunto** es una colección de elementos. Por ejemplo, el conjunto

$$X = \{2, 4, 7\}$$

contiene los elementos 2 , 4 y 7 . El símbolo $\emptyset$ denota un conjunto vacío, y $|S|$ denota el tamaño de un conjunto S , es decir, el número de elementos del conjunto. Por ejemplo, en el conjunto anterior, $|X| = 3$.

Si un conjunto S contiene un elemento x , escribimos $x \in S$, y en caso contrario escribimos $x \notin S$. Por ejemplo, en el conjunto anterior

$$4 \in X \quad \text{y} \quad 5 \notin X.$$

Pueden construirse nuevos conjuntos usando operaciones de conjuntos:

- La **intersección** $A \cap B$ consta de los elementos que están tanto en A como en B . Por ejemplo, si $A = \{1, 2, 5\}$ y $B = \{2, 4\}$, entonces $A \cap B = \{2\}$.
- La **unión** $A \cup B$ consta de los elementos que están en A o en B o en ambos. Por ejemplo, si $A = \{3, 7\}$ y $B = \{2, 3, 8\}$, entonces $A \cup B = \{2, 3, 7, 8\}$.
- El **complemento** $\bar{A}$ consta de los elementos que no están en A . La interpretación de un complemento depende del **conjunto universal**, que contiene todos los elementos posibles. Por ejemplo, si $A = \{1, 2, 5, 7\}$ y el conjunto universal es $\{1, 2, \dots, 10\}$, entonces $\bar{A} = \{3, 4, 6, 8, 9, 10\}$.
- La **diferencia** $A \setminus B = A \cap \bar{B}$ consta de los elementos que están en A pero no en B . Nota que B puede contener elementos que no están en A . Por ejemplo, si $A = \{2, 3, 7, 8\}$ y $B = \{3, 5, 8\}$, entonces $A \setminus B = \{2, 7\}$.

Si cada elemento de A también pertenece a S , decimos que A es un **subconjunto** de S , denotado por $A \subset S$. Un conjunto S siempre tiene $2^{|S|}$ subconjuntos,

incluido el conjunto vacío. Por ejemplo, los subconjuntos del conjunto $\{2, 4, 7\}$ son

$$\emptyset, \{2\}, \{4\}, \{7\}, \{2, 4\}, \{2, 7\}, \{4, 7\} \text{ y } \{2, 4, 7\}.$$

Algunos conjuntos de uso frecuente son $\mathbb{N}$ (números naturales), $\mathbb{Z}$ (enteros), $\mathbb{Q}$ (números racionales) y $\mathbb{R}$ (números reales). El conjunto $\mathbb{N}$ puede definirse de dos maneras, según la situación: o bien $\mathbb{N} = \{0, 1, 2, \dots\}$ o bien $\mathbb{N} = \{1, 2, 3, \dots\}$.

También podemos construir un conjunto usando una regla de la forma

$$\{f(n) : n \in S\},$$

donde $f(n)$ es alguna función. Este conjunto contiene todos los elementos de la forma $f(n)$, donde n es un elemento de S . Por ejemplo, el conjunto

$$X = \{2n : n \in \mathbb{Z}\}$$

contiene todos los enteros pares.

Lógica

El valor de una expresión lógica es **verdadero** (1) o **falso** (0). Los operadores lógicos más importantes son $\neg$ (**negación**), $\wedge$ (**conjunción**), $\vee$ (**disyunción**), $\Rightarrow$ (**implicación**) y $\Leftrightarrow$ (**equivalencia**). La siguiente tabla muestra los significados de estos operadores:

A	B	$\neg A$	$\neg B$	$A \wedge B$	$A \vee B$	$A \Rightarrow B$	$A \Leftrightarrow B$
0	0	1	1	0	0	1	1
0	1	1	0	0	1	1	0
1	0	0	1	0	1	0	0
1	1	0	0	1	1	1	1

La expresión $\neg A$ tiene el valor opuesto de A . La expresión $A \wedge B$ es verdadera si tanto A como B son verdaderas, y la expresión $A \vee B$ es verdadera si A o B o ambas son verdaderas. La expresión $A \Rightarrow B$ es verdadera si siempre que A es verdadera, también B es verdadera. La expresión $A \Leftrightarrow B$ es verdadera si A y B son ambas verdaderas o ambas falsas.

Un **predicado** es una expresión que es verdadera o falsa según sus parámetros. Los predicados suelen denotarse con letras mayúsculas. Por ejemplo, podemos definir un predicado $P(x)$ que es verdadero exactamente cuando x es un número primo. Usando esta definición, $P(7)$ es verdadero pero $P(8)$ es falso.

Un **cuantificador** conecta una expresión lógica con los elementos de un conjunto. Los cuantificadores más importantes son $\forall$ (**para todo**) y $\exists$ (**existe**). Por ejemplo,

$$\forall x(\exists y(y < x))$$

significa que para cada elemento x del conjunto, existe un elemento y en el conjunto tal que y es menor que x . Esto es verdadero en el conjunto de los enteros, pero falso en el conjunto de los números naturales.

Usando la notación descrita arriba, podemos expresar muchos tipos de proposiciones lógicas. Por ejemplo,

$$\forall x((x > 1 \wedge \neg P(x)) \Rightarrow (\exists a(\exists b(a > 1 \wedge b > 1 \wedge x = ab))))$$

significa que si un número x es mayor que 1 y no es un número primo, entonces existen números a y b que son mayores que 1 y cuyo producto es x . Esta proposición es verdadera en el conjunto de los enteros.

Funciones

La función $\lfloor x \rfloor$ redondea el número x hacia abajo a un entero, y la función $\lceil x \rceil$ redondea el número x hacia arriba a un entero. Por ejemplo,

$$\lfloor 3/2 \rfloor = 1 \quad \text{y} \quad \lceil 3/2 \rceil = 2.$$

Las funciones $\min(x_1, x_2, \dots, x_n)$ y $\max(x_1, x_2, \dots, x_n)$ dan el menor y el mayor de los valores $x_1, x_2, \dots, x_n$. Por ejemplo,

$$\min(1, 2, 3) = 1 \quad \text{y} \quad \max(1, 2, 3) = 3.$$

El **factorial** $n!$ puede definirse

$$\prod_{x=1}^n x = 1 \cdot 2 \cdot 3 \cdot \dots \cdot n$$

o recursivamente

$$\begin{aligned} 0! &= 1 \\ n! &= n \cdot (n-1)! \end{aligned}$$

Los **números de Fibonacci** aparecen en muchas situaciones. Pueden definirse recursivamente de la siguiente manera:

$$\begin{aligned} f(0) &= 0 \\ f(1) &= 1 \\ f(n) &= f(n-1) + f(n-2) \end{aligned}$$

Los primeros números de Fibonacci son

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \dots$$

También existe una fórmula cerrada para calcular los números de Fibonacci, que a veces se llama **fórmula de Binet**:

$$f(n) = \frac{(1 + \sqrt{5})^n - (1 - \sqrt{5})^n}{2^n \sqrt{5}}.$$

Logaritmos

El **logaritmo** de un número x se denota $\log_k(x)$, donde k es la base del logaritmo. Según la definición, $\log_k(x) = a$ exactamente cuando $k^a = x$.

Una propiedad útil de los logaritmos es que $\log_k(x)$ es igual al número de veces que tenemos que dividir x entre k antes de llegar al número 1. Por ejemplo, $\log_2(32) = 5$ porque se necesitan 5 divisiones entre 2:

$$32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$$

Los logaritmos se usan a menudo en el análisis de algoritmos, porque muchos algoritmos eficientes reducen algo a la mitad en cada paso. Por eso, podemos estimar la eficiencia de tales algoritmos usando logaritmos.

El logaritmo de un producto es

$$\log_k(ab) = \log_k(a) + \log_k(b),$$

y en consecuencia,

$$\log_k(x^n) = n \cdot \log_k(x).$$

Además, el logaritmo de un cociente es

$$\log_k\left(\frac{a}{b}\right) = \log_k(a) - \log_k(b).$$

Otra fórmula útil es

$$\log_u(x) = \frac{\log_k(x)}{\log_k(u)},$$

y usando esto, es posible calcular logaritmos en cualquier base si hay una forma de calcular logaritmos en alguna base fija.

El **logaritmo natural** $\ln(x)$ de un número x es un logaritmo cuya base es $e \approx 2.71828$. Otra propiedad de los logaritmos es que el número de dígitos de un entero x en base b es $\lfloor \log_b(x) \rfloor + 1$. Por ejemplo, la representación de 123 en base 2 es 1111011 y $\lfloor \log_2(123) \rfloor + 1 = 7$.

1.6. Concursos y recursos

IOI

La Olimpiada Internacional de Informática (IOI) es un concurso de programación anual para estudiantes de secundaria. Cada país puede enviar un equipo de cuatro estudiantes al concurso. Suele haber unos 300 participantes de 80 países.

La IOI consta de dos concursos de cinco horas de duración. En ambos concursos, se pide a los participantes que resuelvan tres tareas algorítmicas de dificultad variada. Las tareas se dividen en subtareas, cada una de las cuales tiene una puntuación asignada. Aunque los concursantes se dividen en equipos, compiten como individuos.

El temario de la IOI [41] regula los temas que pueden aparecer en las tareas de la IOI. Casi todos los temas del temario de la IOI están cubiertos por este libro.

Los participantes de la IOI se seleccionan mediante concursos nacionales. Antes de la IOI, se organizan muchos concursos regionales, como la Olimpiada Báltica de Informática (BOI), la Olimpiada Centroeuropa de Informática (CEOI) y la Olimpiada de Informática de Asia-Pacífico (APIO).

Algunos países organizan concursos de práctica en línea para futuros participantes de la IOI, como el Concurso Abierto Croata de Informática [11] y la Olimpiada de Computación de EE. UU. [68]. Además, hay una gran colección de problemas de concursos polacos disponible en línea [60].

ICPC

El Concurso Internacional de Programación Colegial (ICPC) es un concurso de programación anual para estudiantes universitarios. Cada equipo del concurso consta de tres estudiantes, y a diferencia de la IOI, los estudiantes trabajan juntos; solo hay una computadora disponible para cada equipo.

El ICPC consta de varias etapas, y finalmente los mejores equipos son invitados a las Finales Mundiales. Aunque hay decenas de miles de participantes en el concurso, solo hay un pequeño número² de plazas disponibles para la final, así que incluso avanzar a las finales es un gran logro en algunas regiones.

En cada concurso ICPC, los equipos tienen cinco horas de tiempo para resolver unos diez problemas algorítmicos. La solución de un problema se acepta solo si resuelve todos los casos de prueba de forma eficiente. Durante el concurso, los competidores pueden ver los resultados de otros equipos, pero durante la última hora el marcador se congela y no es posible ver los resultados de los últimos envíos.

Los temas que pueden aparecer en el ICPC no están tan bien especificados como los de la IOI. En cualquier caso, está claro que se necesita más conocimiento en el ICPC, especialmente más habilidades matemáticas.

Concursos en línea

También hay muchos concursos en línea que están abiertos para todos. En este momento, el sitio de concursos más activo es Codeforces, que organiza concursos aproximadamente cada semana. En Codeforces, los participantes se dividen en dos divisiones: los principiantes compiten en la Div2 y los programadores más experimentados en la Div1. Otros sitios de concursos son AtCoder, CS Academy, HackerRank y Topcoder.

Algunas empresas organizan concursos en línea con finales presenciales. Ejemplos de tales concursos son el Facebook Hacker Cup, el Google Code Jam y Yandex.Algorithm. Por supuesto, las empresas también usan esos concursos para reclutar: tener un buen desempeño en un concurso es una buena forma de demostrar las habilidades de uno.

²El número exacto de plazas para la final varía de año en año; en 2017, hubo 133 plazas para la final.

Libros

Ya hay algunos libros (además de este) que se centran en la programación competitiva y la resolución algorítmica de problemas:

- S. S. Skiena y M. A. Revilla: *Programming Challenges: The Programming Contest Training Manual* [59]
- S. Halim y F. Halim: *Competitive Programming 3: The New Lower Bound of Programming Contests* [33]
- K. Diks et al.: *Looking for a Challenge? The Ultimate Problem Set from the University of Warsaw Programming Competitions* [15]

Los dos primeros libros están pensados para principiantes, mientras que el último libro contiene material avanzado.

Por supuesto, los libros generales de algoritmos también son adecuados para los programadores competitivos. Algunos libros populares son:

- T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein: *Introduction to Algorithms* [13]
- J. Kleinberg y É. Tardos: *Algorithm Design* [45]
- S. S. Skiena: *The Algorithm Design Manual* [58]

Capítulo 2

Complejidad temporal

La eficiencia de los algoritmos es importante en la programación competitiva. Normalmente, es fácil diseñar un algoritmo que resuelva el problema lentamente, pero el verdadero reto es idear un algoritmo rápido. Si el algoritmo es demasiado lento, obtendrá solo puntos parciales o ningún punto en absoluto.

La **complejidad temporal** de un algoritmo estima cuánto tiempo usará el algoritmo para una entrada dada. La idea es representar la eficiencia como una función cuyo parámetro es el tamaño de la entrada. Al calcular la complejidad temporal, podemos averiguar si el algoritmo es lo bastante rápido sin implementarlo.

2.1. Reglas de cálculo

La complejidad temporal de un algoritmo se denota $O(\dots)$ donde los tres puntos representan alguna función. Normalmente, la variable n denota el tamaño de la entrada. Por ejemplo, si la entrada es un arreglo de números, n será el tamaño del arreglo, y si la entrada es una cadena, n será la longitud de la cadena.

Bucles

Una razón común por la que un algoritmo es lento es que contiene muchos bucles que recorren la entrada. Cuantos más bucles anidados contiene el algoritmo, más lento es. Si hay k bucles anidados, la complejidad temporal es $O(n^k)$.

Por ejemplo, la complejidad temporal del siguiente código es $O(n)$:

```
for (int i = 1; i <= n; i++) {  
    // code  
}
```

Y la complejidad temporal del siguiente código es $O(n^2)$:

```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= n; j++) {  
        // code  
    }  
}
```

```
}
```

Orden de magnitud

Una complejidad temporal no nos dice el número exacto de veces que se ejecuta el código dentro de un bucle, sino que solo muestra el orden de magnitud. En los siguientes ejemplos, el código dentro del bucle se ejecuta $3n$, $n + 5$ y $\lceil n/2 \rceil$ veces, pero la complejidad temporal de cada código es $O(n)$.

```
for (int i = 1; i <= 3*n; i++) {  
    // code  
}
```

```
for (int i = 1; i <= n+5; i++) {  
    // code  
}
```

```
for (int i = 1; i <= n; i += 2) {  
    // code  
}
```

Como otro ejemplo, la complejidad temporal del siguiente código es $O(n^2)$:

```
for (int i = 1; i <= n; i++) {  
    for (int j = i+1; j <= n; j++) {  
        // code  
    }  
}
```

Fases

Si el algoritmo consta de fases consecutivas, la complejidad temporal total es la mayor complejidad temporal de una sola fase. La razón de esto es que la fase más lenta suele ser el cuello de botella del código.

Por ejemplo, el siguiente código consta de tres fases con complejidades temporales $O(n)$, $O(n^2)$ y $O(n)$. Así, la complejidad temporal total es $O(n^2)$.

```
for (int i = 1; i <= n; i++) {  
    // code  
}  
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= n; j++) {  
        // code  
    }  
}
```

```
for (int i = 1; i <= n; i++) {  
    // code  
}
```

Varias variables

A veces la complejidad temporal depende de varios factores. En este caso, la fórmula de la complejidad temporal contiene varias variables.

Por ejemplo, la complejidad temporal del siguiente código es $O(nm)$:

```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= m; j++) {  
        // code  
    }  
}
```

Recursión

La complejidad temporal de una función recursiva depende del número de veces que se llama a la función y de la complejidad temporal de una sola llamada. La complejidad temporal total es el producto de esos valores.

Por ejemplo, considera la siguiente función:

```
void f(int n) {  
    if (n == 1) return;  
    f(n-1);  
}
```

La llamada $f(n)$ provoca n llamadas a la función, y la complejidad temporal de cada llamada es $O(1)$. Así, la complejidad temporal total es $O(n)$.

Como otro ejemplo, considera la siguiente función:

```
void g(int n) {  
    if (n == 1) return;  
    g(n-1);  
    g(n-1);  
}
```

En este caso cada llamada a la función genera otras dos llamadas, excepto para $n = 1$. Veamos qué ocurre cuando se llama a g con el parámetro n . La siguiente tabla muestra las llamadas a la función producidas por esta única llamada:

llamada a la función	número de llamadas
$g(n)$	1
$g(n-1)$	2
$g(n-2)$	4
...	...
$g(1)$	2^{n-1}

Con base en esto, la complejidad temporal es

$$1 + 2 + 4 + \dots + 2^{n-1} = 2^n - 1 = O(2^n).$$

2.2. Clases de complejidad

La siguiente lista contiene complejidades temporales comunes de los algoritmos:

- $O(1)$ El tiempo de ejecución de un algoritmo de **tiempo constante** no depende del tamaño de la entrada. Un algoritmo de tiempo constante típico es una fórmula directa que calcula la respuesta.
- $O(\log n)$ Un algoritmo **logarítmico** a menudo reduce a la mitad el tamaño de la entrada en cada paso. El tiempo de ejecución de tal algoritmo es logarítmico, porque $\log_2 n$ es igual al número de veces que n debe dividirse entre 2 para llegar a 1.
- $O(\sqrt{n})$ Un **algoritmo de raíz cuadrada** es más lento que $O(\log n)$ pero más rápido que $O(n)$. Una propiedad especial de las raíces cuadradas es que $\sqrt{n} = n/\sqrt{n}$, así que la raíz cuadrada $\sqrt{n}$ se encuentra, en cierto sentido, en el medio de la entrada.
- $O(n)$ Un algoritmo **lineal** recorre la entrada un número constante de veces. Esta suele ser la mejor complejidad temporal posible, porque normalmente es necesario acceder a cada elemento de la entrada al menos una vez antes de dar la respuesta.
- $O(n \log n)$ Esta complejidad temporal a menudo indica que el algoritmo ordena la entrada, porque la complejidad temporal de los algoritmos de ordenamiento eficientes es $O(n \log n)$. Otra posibilidad es que el algoritmo use una estructura de datos donde cada operación toma $O(\log n)$ de tiempo.
- $O(n^2)$ Un algoritmo **cuadrático** a menudo contiene dos bucles anidados. Es posible recorrer todos los pares de elementos de la entrada en tiempo $O(n^2)$.
- $O(n^3)$ Un algoritmo **cúbico** a menudo contiene tres bucles anidados. Es posible recorrer todas las ternas de elementos de la entrada en tiempo $O(n^3)$.
- $O(2^n)$ Esta complejidad temporal a menudo indica que el algoritmo itera sobre todos los subconjuntos de los elementos de la entrada. Por ejemplo, los subconjuntos de $\{1, 2, 3\}$ son $\emptyset$, $\{1\}$, $\{2\}$, $\{3\}$, $\{1, 2\}$, $\{1, 3\}$, $\{2, 3\}$ y $\{1, 2, 3\}$.

$O(n!)$ Esta complejidad temporal a menudo indica que el algoritmo itera sobre todas las permutaciones de los elementos de la entrada. Por ejemplo, las permutaciones de $\{1,2,3\}$ son $(1,2,3)$, $(1,3,2)$, $(2,1,3)$, $(2,3,1)$, $(3,1,2)$ y $(3,2,1)$.

Un algoritmo es **polinómico** si su complejidad temporal es a lo sumo $O(n^k)$ donde k es una constante. Todas las complejidades temporales anteriores excepto $O(2^n)$ y $O(n!)$ son polinómicas. En la práctica, la constante k suele ser pequeña, y por lo tanto una complejidad temporal polinómica significa a grandes rasgos que el algoritmo es *eficiente*.

La mayoría de los algoritmos de este libro son polinómicos. Aun así, hay muchos problemas importantes para los que no se conoce ningún algoritmo polinómico, es decir, nadie sabe cómo resolverlos de forma eficiente. Los problemas **NP-difíciles** son un conjunto importante de problemas, para los que no se conoce ningún algoritmo polinómico¹.

2.3. Estimar la eficiencia

Al calcular la complejidad temporal de un algoritmo, es posible comprobar, antes de implementar el algoritmo, que es lo bastante eficiente para el problema. El punto de partida para las estimaciones es el hecho de que una computadora moderna puede realizar algunos cientos de millones de operaciones en un segundo.

Por ejemplo, supongamos que el límite de tiempo de un problema es un segundo y el tamaño de la entrada es $n = 10^5$. Si la complejidad temporal es $O(n^2)$, el algoritmo realizará unas $(10^5)^2 = 10^{10}$ operaciones. Esto debería tomar al menos algunas decenas de segundos, así que el algoritmo parece ser demasiado lento para resolver el problema.

Por otro lado, dado el tamaño de la entrada, podemos intentar *adivinar* la complejidad temporal requerida del algoritmo que resuelve el problema. La siguiente tabla contiene algunas estimaciones útiles suponiendo un límite de tiempo de un segundo.

tamaño de la entrada	complejidad temporal requerida
$n \leq 10$	$O(n!)$
$n \leq 20$	$O(2^n)$
$n \leq 500$	$O(n^3)$
$n \leq 5000$	$O(n^2)$
$n \leq 10^6$	$O(n \log n)$ u $O(n)$
n es grande	$O(1)$ u $O(\log n)$

Por ejemplo, si el tamaño de la entrada es $n = 10^5$, probablemente se espere que la complejidad temporal del algoritmo sea $O(n)$ u $O(n \log n)$. Esta información

¹Un libro clásico sobre el tema es *Computers and Intractability: A Guide to the Theory of NP-Completeness*, de M. R. Garey y D. S. Johnson [28].

facilita el diseño del algoritmo, porque descarta enfoques que darían un algoritmo con una peor complejidad temporal.

Aun así, es importante recordar que una complejidad temporal es solo una estimación de la eficiencia, porque oculta los *factores constantes*. Por ejemplo, un algoritmo que se ejecuta en tiempo $O(n)$ puede realizar $n/2$ o $5n$ operaciones. Esto tiene un efecto importante en el tiempo de ejecución real del algoritmo.

2.4. Suma máxima de subarreglo

A menudo hay varios algoritmos posibles para resolver un problema de modo que sus complejidades temporales sean distintas. Esta sección analiza un problema clásico que tiene una solución directa en $O(n^3)$. Sin embargo, al diseñar un algoritmo mejor, es posible resolver el problema en tiempo $O(n^2)$ e incluso en tiempo $O(n)$.

Dado un arreglo de n números, nuestra tarea es calcular la **suma máxima de subarreglo**, es decir, la mayor suma posible de una secuencia de valores consecutivos del arreglo². El problema es interesante cuando puede haber valores negativos en el arreglo. Por ejemplo, en el arreglo

-1	2	4	-3	5	2	-5	2
----	---	---	----	---	---	----	---

el siguiente subarreglo produce la suma máxima 10:

-1	2	4	-3	5	2	-5	2
----	---	---	----	---	---	----	---

Suponemos que se permite un subarreglo vacío, así que la suma máxima de subarreglo es siempre al menos 0.

Algoritmo 1

Una forma directa de resolver el problema es recorrer todos los subarreglos posibles, calcular la suma de los valores de cada subarreglo y mantener la suma máxima. El siguiente código implementa este algoritmo:

```
int best = 0;
for (int a = 0; a < n; a++) {
    for (int b = a; b < n; b++) {
        int sum = 0;
        for (int k = a; k <= b; k++) {
            sum += array[k];
        }
        best = max(best, sum);
    }
}
cout << best << "\n";
```

²El libro *Programming Pearls*, de J. Bentley [8], popularizó el problema.

Las variables a y b fijan el primer y último índice del subarreglo, y la suma de los valores se calcula en la variable sum . La variable $best$ contiene la suma máxima encontrada durante la búsqueda.

La complejidad temporal del algoritmo es $O(n^3)$, porque consta de tres bucles anidados que recorren la entrada.

Algoritmo 2

Es fácil hacer el Algoritmo 1 más eficiente eliminando un bucle de él. Esto es posible calculando la suma al mismo tiempo que se mueve el extremo derecho del subarreglo. El resultado es el siguiente código:

```
int best = 0;
for (int a = 0; a < n; a++) {
    int sum = 0;
    for (int b = a; b < n; b++) {
        sum += array[b];
        best = max(best, sum);
    }
}
cout << best << "\n";
```

Después de este cambio, la complejidad temporal es $O(n^2)$.

Algoritmo 3

Sorprendentemente, es posible resolver el problema en tiempo $O(n)^3$, lo que significa que basta con un solo bucle. La idea es calcular, para cada posición del arreglo, la suma máxima de un subarreglo que termina en esa posición. Después de esto, la respuesta al problema es el máximo de esas sumas.

Considera el subproblema de encontrar el subarreglo de suma máxima que termina en la posición k . Hay dos posibilidades:

1. El subarreglo solo contiene el elemento en la posición k .
2. El subarreglo consta de un subarreglo que termina en la posición $k - 1$, seguido del elemento en la posición k .

En el último caso, como queremos encontrar un subarreglo con suma máxima, el subarreglo que termina en la posición $k - 1$ también debe tener la suma máxima. Así, podemos resolver el problema de forma eficiente calculando la suma máxima de subarreglo para cada posición final, de izquierda a derecha.

El siguiente código implementa el algoritmo:

```
int best = 0, sum = 0;
for (int k = 0; k < n; k++) {
```

³En [8], este algoritmo de tiempo lineal se atribuye a J. B. Kadane, y el algoritmo a veces se llama **algoritmo de Kadane**.

```

    sum = max(array[k], sum+array[k]);
    best = max(best, sum);
}
cout << best << "\n";

```

El algoritmo solo contiene un bucle que recorre la entrada, así que la complejidad temporal es $O(n)$. Esta es también la mejor complejidad temporal posible, porque cualquier algoritmo para el problema tiene que examinar todos los elementos del arreglo al menos una vez.

Comparación de eficiencia

Es interesante estudiar qué tan eficientes son los algoritmos en la práctica. La siguiente tabla muestra los tiempos de ejecución de los algoritmos anteriores para distintos valores de n en una computadora moderna.

En cada prueba, la entrada se generó aleatoriamente. El tiempo necesario para leer la entrada no se midió.

tamaño del arreglo n	Algoritmo 1	Algoritmo 2	Algoritmo 3
10^2	0.0 s	0.0 s	0.0 s
10^3	0.1 s	0.0 s	0.0 s
10^4	> 10.0 s	0.1 s	0.0 s
10^5	> 10.0 s	5.3 s	0.0 s
10^6	> 10.0 s	> 10.0 s	0.0 s
10^7	> 10.0 s	> 10.0 s	0.0 s

La comparación muestra que todos los algoritmos son eficientes cuando el tamaño de la entrada es pequeño, pero las entradas más grandes hacen notar diferencias notables en los tiempos de ejecución de los algoritmos. El Algoritmo 1 se vuelve lento cuando $n = 10^4$, y el Algoritmo 2 se vuelve lento cuando $n = 10^5$. Solo el Algoritmo 3 es capaz de procesar incluso las entradas más grandes de forma instantánea.

Capítulo 3

Ordenamiento

El **ordenamiento** es un problema fundamental del diseño de algoritmos. Muchos algoritmos eficientes usan el ordenamiento como subrutina, porque a menudo es más fácil procesar los datos si los elementos están en orden.

Por ejemplo, el problema "¿un arreglo contiene dos elementos iguales?" es fácil de resolver usando ordenamiento. Si el arreglo contiene dos elementos iguales, quedarán uno junto al otro después de ordenar, así que es fácil encontrarlos. De igual forma, el problema "¿cuál es el elemento más frecuente de un arreglo?" puede resolverse de manera similar.

Hay muchos algoritmos de ordenamiento, y también son buenos ejemplos de cómo aplicar distintas técnicas de diseño de algoritmos. Los algoritmos generales de ordenamiento eficientes funcionan en tiempo $O(n \log n)$, y muchos algoritmos que usan el ordenamiento como subrutina también tienen esta complejidad temporal.

3.1. Teoría del ordenamiento

El problema básico del ordenamiento es el siguiente:

Dado un arreglo que contiene n elementos, tu tarea es ordenar los elementos en orden creciente.

Por ejemplo, el arreglo

1	3	8	2	9	2	5	6
---	---	---	---	---	---	---	---

quedará como sigue después de ordenar:

1	2	2	3	5	6	8	9
---	---	---	---	---	---	---	---

Algoritmos $O(n^2)$

Los algoritmos sencillos para ordenar un arreglo funcionan en tiempo $O(n^2)$. Tales algoritmos son cortos y normalmente constan de dos bucles anidados. Un

famoso algoritmo de ordenamiento de tiempo $O(n^2)$ es el **ordenamiento de burbuja**, donde los elementos "burbujean" en el arreglo según sus valores.

El ordenamiento de burbuja consta de n rondas. En cada ronda, el algoritmo recorre los elementos del arreglo. Siempre que se encuentran dos elementos consecutivos que no están en el orden correcto, el algoritmo los intercambia. El algoritmo puede implementarse de la siguiente manera:

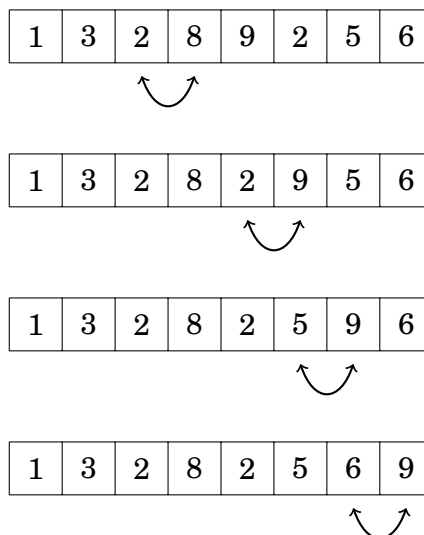
```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n-1; j++) {  
        if (array[j] > array[j+1]) {  
            swap(array[j], array[j+1]);  
        }  
    }  
}
```

Después de la primera ronda del algoritmo, el elemento más grande estará en la posición correcta, y en general, después de k rondas, los k elementos más grandes estarán en las posiciones correctas. Así, después de n rondas, todo el arreglo estará ordenado.

Por ejemplo, en el arreglo

1	3	8	2	9	2	5	6
---	---	---	---	---	---	---	---

la primera ronda del ordenamiento de burbuja intercambia elementos de la siguiente manera:



Inversiones

El ordenamiento de burbuja es un ejemplo de un algoritmo de ordenamiento que siempre intercambia elementos *consecutivos* del arreglo. Resulta que la complejidad temporal de tal algoritmo es *siempre* al menos $O(n^2)$, porque en el peor caso, se requieren $O(n^2)$ intercambios para ordenar el arreglo.

Un concepto útil al analizar algoritmos de ordenamiento es una **inversión**: un par de elementos del arreglo ($\text{array}[a], \text{array}[b]$) tal que $a < b$ y $\text{array}[a] > \text{array}[b]$, es decir, los elementos están en el orden incorrecto. Por ejemplo, el arreglo

1	2	2	6	3	5	9	8
---	---	---	---	---	---	---	---

tiene tres inversiones: (6,3), (6,5) y (9,8). El número de inversiones indica cuánto trabajo se necesita para ordenar el arreglo. Un arreglo está completamente ordenado cuando no hay inversiones. Por otro lado, si los elementos del arreglo están en el orden inverso, el número de inversiones es el mayor posible:

$$1 + 2 + \dots + (n - 1) = \frac{n(n - 1)}{2} = O(n^2)$$

Intercambiar un par de elementos consecutivos que están en el orden incorrecto elimina exactamente una inversión del arreglo. Por lo tanto, si un algoritmo de ordenamiento solo puede intercambiar elementos consecutivos, cada intercambio elimina a lo sumo una inversión, y la complejidad temporal del algoritmo es al menos $O(n^2)$.

Algoritmos $O(n \log n)$

Es posible ordenar un arreglo de forma eficiente en tiempo $O(n \log n)$ usando algoritmos que no se limitan a intercambiar elementos consecutivos. Uno de tales algoritmos es el **ordenamiento por mezcla**¹, que se basa en la recursión.

El ordenamiento por mezcla ordena un subarreglo $\text{array}[a \dots b]$ de la siguiente manera:

1. Si $a = b$, no hagas nada, porque el subarreglo ya está ordenado.
2. Calcula la posición del elemento del medio: $k = \lfloor (a + b)/2 \rfloor$.
3. Ordena recursivamente el subarreglo $\text{array}[a \dots k]$.
4. Ordena recursivamente el subarreglo $\text{array}[k + 1 \dots b]$.
5. *Mezcla* los subarreglos ordenados $\text{array}[a \dots k]$ y $\text{array}[k + 1 \dots b]$ en un subarreglo ordenado $\text{array}[a \dots b]$.

El ordenamiento por mezcla es un algoritmo eficiente, porque reduce a la mitad el tamaño del subarreglo en cada paso. La recursión consta de $O(\log n)$ niveles, y procesar cada nivel toma $O(n)$ de tiempo. Mezclar los subarreglos $\text{array}[a \dots k]$ y $\text{array}[k + 1 \dots b]$ es posible en tiempo lineal, porque ya están ordenados.

Por ejemplo, considera ordenar el siguiente arreglo:

1	3	6	2	8	2	5	9
---	---	---	---	---	---	---	---

¹Según [47], el ordenamiento por mezcla fue inventado por J. von Neumann en 1945.

El arreglo se dividirá en dos subarreglos de la siguiente manera:

1	3	6	2	8	2	5	9
---	---	---	---	---	---	---	---

Luego, los subarreglos se ordenarán recursivamente de la siguiente manera:

1	2	3	6	2	5	8	9
---	---	---	---	---	---	---	---

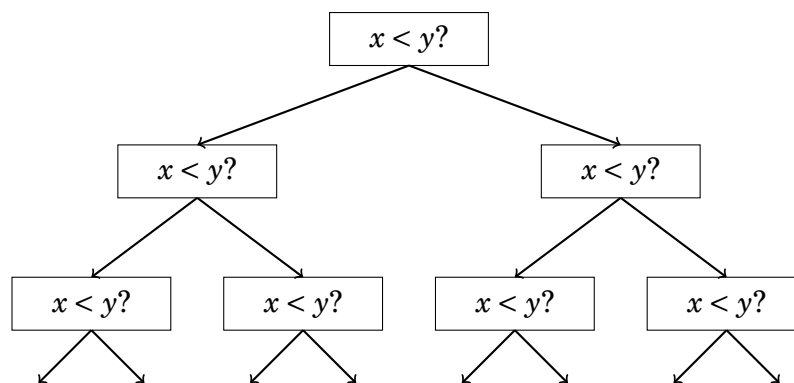
Finalmente, el algoritmo mezcla los subarreglos ordenados y crea el arreglo ordenado final:

1	2	2	3	5	6	8	9
---	---	---	---	---	---	---	---

Cota inferior del ordenamiento

¿Es posible ordenar un arreglo más rápido que en tiempo $O(n \log n)$? Resulta que esto *no* es posible cuando nos restringimos a algoritmos de ordenamiento que se basan en comparar elementos del arreglo.

La cota inferior de la complejidad temporal puede demostrarse considerando el ordenamiento como un proceso donde cada comparación de dos elementos da más información sobre el contenido del arreglo. El proceso crea el siguiente árbol:



Aquí " $x < y$?" significa que se comparan algunos elementos x y y . Si $x < y$, el proceso continúa hacia la izquierda, y en caso contrario hacia la derecha. Los resultados del proceso son las posibles formas de ordenar el arreglo, un total de $n!$ formas. Por esta razón, la altura del árbol debe ser al menos

$$\log_2(n!) = \log_2(1) + \log_2(2) + \cdots + \log_2(n).$$

Obtenemos una cota inferior para esta suma eligiendo los últimos $n/2$ elementos y cambiando el valor de cada elemento a $\log_2(n/2)$. Esto produce una estimación

$$\log_2(n!) \geq (n/2) \cdot \log_2(n/2),$$

así que la altura del árbol y el número mínimo posible de pasos en un algoritmo de ordenamiento en el peor caso es al menos $n \log n$.

Ordenamiento por conteo

La cota inferior $n \log n$ no se aplica a algoritmos que no comparan elementos del arreglo sino que usan alguna otra información. Un ejemplo de tal algoritmo es el **ordenamiento por conteo**, que ordena un arreglo en tiempo $O(n)$ suponiendo que todo elemento del arreglo es un entero entre $0 \dots c$ y $c = O(n)$.

El algoritmo crea un arreglo de *conteo*, cuyos índices son los elementos del arreglo original. El algoritmo recorre el arreglo original y calcula cuántas veces aparece cada elemento en el arreglo.

Por ejemplo, el arreglo

1	3	6	9	9	3	5	9
---	---	---	---	---	---	---	---

corresponde al siguiente arreglo de conteo:

1	2	3	4	5	6	7	8	9
1	0	2	0	1	1	0	0	3

Por ejemplo, el valor en la posición 3 del arreglo de conteo es 2, porque el elemento 3 aparece 2 veces en el arreglo original.

La construcción del arreglo de conteo toma $O(n)$ de tiempo. Después de esto, el arreglo ordenado puede crearse en tiempo $O(n)$ porque el número de apariciones de cada elemento puede recuperarse del arreglo de conteo. Así, la complejidad temporal total del ordenamiento por conteo es $O(n)$.

El ordenamiento por conteo es un algoritmo muy eficiente pero solo puede usarse cuando la constante c es lo bastante pequeña, de modo que los elementos del arreglo puedan usarse como índices en el arreglo de conteo.

3.2. Ordenamiento en C++

Casi nunca es buena idea usar un algoritmo de ordenamiento hecho en casa en un concurso, porque hay buenas implementaciones disponibles en los lenguajes de programación. Por ejemplo, la biblioteca estándar de C++ contiene la función `sort` que puede usarse fácilmente para ordenar arreglos y otras estructuras de datos.

Usar una función de biblioteca tiene muchos beneficios. Primero, ahorra tiempo porque no hay necesidad de implementar la función. Segundo, la implementación de la biblioteca es sin duda correcta y eficiente: no es probable que una función de ordenamiento hecha en casa sea mejor.

En esta sección veremos cómo usar la función `sort` de C++. El siguiente código ordena un vector en orden creciente:

```
vector<int> v = {4,2,5,3,5,8,3};  
sort(v.begin(),v.end());
```

Después del ordenamiento, el contenido del vector será `[2,3,3,4,5,5,8]`. El orden de ordenamiento por defecto es creciente, pero un orden inverso es posible de la siguiente manera:

```
sort(v.rbegin(),v.rend());
```

Un arreglo ordinario puede ordenarse de la siguiente manera:

```
int n = 7; // array size  
int a[] = {4,2,5,3,5,8,3};  
sort(a,a+n);
```

El siguiente código ordena la cadena s:

```
string s = "monkey";  
sort(s.begin(), s.end());
```

Ordenar una cadena significa que los caracteres de la cadena se ordenan. Por ejemplo, la cadena "monkey" se convierte en "ekmnoy".

Operadores de comparación

La función `sort` requiere que se defina un **operador de comparación** para el tipo de dato de los elementos que se van a ordenar. Al ordenar, este operador se usará siempre que sea necesario averiguar el orden de dos elementos.

La mayoría de los tipos de datos de C++ tienen un operador de comparación integrado, y los elementos de esos tipos pueden ordenarse automáticamente. Por ejemplo, los números se ordenan según sus valores y las cadenas se ordenan en orden alfabético.

Los pares (`pair`) se ordenan principalmente según sus primeros elementos (`first`). Sin embargo, si los primeros elementos de dos pares son iguales, se ordenan según sus segundos elementos (`second`):

```
vector<pair<int,int>> v;  
v.push_back({1,5});  
v.push_back({2,3});  
v.push_back({1,2});  
sort(v.begin(), v.end());
```

Después de esto, el orden de los pares es (1,2), (1,5) y (2,3).

De manera similar, las tuplas (`tuple`) se ordenan principalmente por el primer elemento, secundariamente por el segundo elemento, etc.²:

```
vector<tuple<int,int,int>> v;  
v.push_back({2,1,4});  
v.push_back({1,5,3});  
v.push_back({2,1,3});  
sort(v.begin(), v.end());
```

Después de esto, el orden de las tuplas es (1,5,3), (2,1,3) y (2,1,4).

Structs definidos por el usuario

Los structs definidos por el usuario no tienen un operador de comparación automáticamente. El operador debe definirse dentro del struct como una función `operator<`, cuyo parámetro es otro elemento del mismo tipo. El operador debe devolver `true` si el elemento es menor que el parámetro, y `false` en caso contrario.

²Nota que en algunos compiladores más antiguos, hay que usar la función `make_tuple` para crear una tupla en lugar de llaves (por ejemplo, `make_tuple(2,1,4)` en vez de `{2,1,4}`).

Por ejemplo, el siguiente struct P contiene las coordenadas x e y de un punto. El operador de comparación se define de modo que los puntos se ordenen principalmente por la coordenada x y secundariamente por la coordenada y.

```
struct P {
    int x, y;
    bool operator<(const P &p) {
        if (x != p.x) return x < p.x;
        else return y < p.y;
    }
};
```

Funciones de comparación

También es posible dar una **función de comparación** externa a la función sort como una función de retorno (callback). Por ejemplo, la siguiente función de comparación comp ordena cadenas principalmente por longitud y secundariamente por orden alfabético:

```
bool comp(string a, string b) {
    if (a.size() != b.size()) return a.size() < b.size();
    return a < b;
}
```

Ahora un vector de cadenas puede ordenarse de la siguiente manera:

```
sort(v.begin(), v.end(), comp);
```

3.3. Búsqueda binaria

Un método general para buscar un elemento en un arreglo es usar un bucle for que recorra los elementos del arreglo. Por ejemplo, el siguiente código busca un elemento x en un arreglo:

```
for (int i = 0; i < n; i++) {
    if (array[i] == x) {
        // x found at index i
    }
}
```

La complejidad temporal de este enfoque es $O(n)$, porque en el peor caso, es necesario revisar todos los elementos del arreglo. Si el orden de los elementos es arbitrario, este es también el mejor enfoque posible, porque no hay información adicional disponible sobre en qué parte del arreglo deberíamos buscar el elemento x .

Sin embargo, si el arreglo está *ordenado*, la situación es distinta. En este caso es posible realizar la búsqueda mucho más rápido, porque el orden de los elementos del arreglo guía la búsqueda. El siguiente algoritmo de **búsqueda binaria** busca eficientemente un elemento en un arreglo ordenado en tiempo $O(\log n)$.

Método 1

La forma habitual de implementar la búsqueda binaria se asemeja a buscar una palabra en un diccionario. La búsqueda mantiene una región activa en el arreglo, que inicialmente contiene todos los elementos del arreglo. Luego, se realiza una serie de pasos, cada uno de los cuales reduce a la mitad el tamaño de la región.

En cada paso, la búsqueda revisa el elemento del medio de la región activa. Si el elemento del medio es el elemento buscado, la búsqueda termina. En caso contrario, la búsqueda continúa recursivamente hacia la mitad izquierda o derecha de la región, según el valor del elemento del medio.

La idea anterior puede implementarse de la siguiente manera:

```
int a = 0, b = n-1;
while (a <= b) {
    int k = (a+b)/2;
    if (array[k] == x) {
        // x found at index k
    }
    if (array[k] > x) b = k-1;
    else a = k+1;
}
```

En esta implementación, la región activa es $a \dots b$, e inicialmente la región es $0 \dots n-1$. El algoritmo reduce a la mitad el tamaño de la región en cada paso, así que la complejidad temporal es $O(\log n)$.

Método 2

Un método alternativo para implementar la búsqueda binaria se basa en una forma eficiente de recorrer los elementos del arreglo. La idea es hacer saltos y reducir la velocidad cuando nos acercamos al elemento buscado.

La búsqueda recorre el arreglo de izquierda a derecha, y la longitud inicial del salto es $n/2$. En cada paso, la longitud del salto se reduce a la mitad: primero $n/4$, luego $n/8$, $n/16$, etc., hasta que finalmente la longitud es 1. Después de los saltos, o bien se ha encontrado el elemento buscado o sabemos que no aparece en el arreglo.

El siguiente código implementa la idea anterior:

```
int k = 0;
for (int b = n/2; b >= 1; b /= 2) {
    while (k+b < n && array[k+b] <= x) k += b;
```

```

}
if (array[k] == x) {
    // x found at index k
}

```

Durante la búsqueda, la variable b contiene la longitud actual del salto. La complejidad temporal del algoritmo es $O(\log n)$, porque el código del bucle `while` se ejecuta a lo sumo dos veces por cada longitud de salto.

Funciones de C++

La biblioteca estándar de C++ contiene las siguientes funciones que se basan en la búsqueda binaria y funcionan en tiempo logarítmico:

- `lower_bound` devuelve un puntero al primer elemento del arreglo cuyo valor es al menos x .
- `upper_bound` devuelve un puntero al primer elemento del arreglo cuyo valor es mayor que x .
- `equal_range` devuelve ambos punteros anteriores.

Las funciones suponen que el arreglo está ordenado. Si no existe tal elemento, el puntero apunta al elemento posterior al último elemento del arreglo. Por ejemplo, el siguiente código averigua si un arreglo contiene un elemento con valor x :

```

auto k = lower_bound(array, array+n, x) - array;
if (k < n && array[k] == x) {
    // x found at index k
}

```

Luego, el siguiente código cuenta el número de elementos cuyo valor es x :

```

auto a = lower_bound(array, array+n, x);
auto b = upper_bound(array, array+n, x);
cout << b-a << "\n";

```

Usando `equal_range`, el código se vuelve más corto:

```

auto r = equal_range(array, array+n, x);
cout << r.second-r.first << "\n";

```

Encontrar la solución más pequeña

Un uso importante de la búsqueda binaria es encontrar la posición donde cambia el valor de una *función*. Supongamos que deseamos encontrar el valor más pequeño k que es una solución válida para un problema. Se nos da una

función $ok(x)$ que devuelve true si x es una solución válida y false en caso contrario. Además, sabemos que $ok(x)$ es false cuando $x < k$ y true cuando $x \geq k$. La situación se ve de la siguiente manera:

x	0	1	...	$k-1$	k	$k+1$	...
$ok(x)$	false	false	...	false	true	true	...

Ahora, el valor de k puede encontrarse usando búsqueda binaria:

```
int x = -1;
for (int b = z; b >= 1; b /= 2) {
    while (!ok(x+b)) x += b;
}
int k = x+1;
```

La búsqueda encuentra el mayor valor de x para el cual $ok(x)$ es false. Así, el siguiente valor $k = x + 1$ es el menor valor posible para el cual $ok(k)$ es true. La longitud inicial del salto z tiene que ser lo bastante grande, por ejemplo algún valor para el cual sepamos de antemano que $ok(z)$ es true.

El algoritmo llama a la función ok $O(\log z)$ veces, así que la complejidad temporal total depende de la función ok . Por ejemplo, si la función funciona en tiempo $O(n)$, la complejidad temporal total es $O(n \log z)$.

Encontrar el valor máximo

La búsqueda binaria también puede usarse para encontrar el valor máximo de una función que es primero creciente y luego decreciente. Nuestra tarea es encontrar una posición k tal que

- $f(x) < f(x+1)$ cuando $x < k$, y
- $f(x) > f(x+1)$ cuando $x \geq k$.

La idea es usar búsqueda binaria para encontrar el mayor valor de x para el cual $f(x) < f(x+1)$. Esto implica que $k = x + 1$ porque $f(x+1) > f(x+2)$. El siguiente código implementa la búsqueda:

```
int x = -1;
for (int b = z; b >= 1; b /= 2) {
    while (f(x+b) < f(x+b+1)) x += b;
}
int k = x+1;
```

Nota que a diferencia de la búsqueda binaria ordinaria, aquí no se permite que valores consecutivos de la función sean iguales. En ese caso no sería posible saber cómo continuar la búsqueda.

Capítulo 4

Estructuras de datos

Una **estructura de datos** es una forma de almacenar datos en la memoria de una computadora. Es importante elegir una estructura de datos apropiada para un problema, porque cada estructura de datos tiene sus propias ventajas y desventajas. La pregunta crucial es: ¿qué operaciones son eficientes en la estructura de datos elegida?

Este capítulo presenta las estructuras de datos más importantes de la biblioteca estándar de C++. Es buena idea usar la biblioteca estándar siempre que sea posible, porque ahorrará mucho tiempo. Más adelante en el libro aprenderemos sobre estructuras de datos más sofisticadas que no están disponibles en la biblioteca estándar.

4.1. Arreglos dinámicos

Un **arreglo dinámico** es un arreglo cuyo tamaño puede cambiarse durante la ejecución del programa. El arreglo dinámico más popular en C++ es la estructura `vector`, que puede usarse casi como un arreglo ordinario.

El siguiente código crea un vector vacío y le agrega tres elementos:

```
vector<int> v;  
v.push_back(3); // [3]  
v.push_back(2); // [3,2]  
v.push_back(5); // [3,2,5]
```

Después de esto, se puede acceder a los elementos como en un arreglo ordinario:

```
cout << v[0] << "\n"; // 3  
cout << v[1] << "\n"; // 2  
cout << v[2] << "\n"; // 5
```

La función `size` devuelve el número de elementos del vector. El siguiente código recorre el vector e imprime todos sus elementos:

```
for (int i = 0; i < v.size(); i++) {
```

```
    cout << v[i] << "\n";  
}
```

Una forma más corta de recorrer un vector es la siguiente:

```
for (auto x : v) {  
    cout << x << "\n";  
}
```

La función `back` devuelve el último elemento del vector, y la función `pop_back` elimina el último elemento:

```
vector<int> v;  
v.push_back(5);  
v.push_back(2);  
cout << v.back() << "\n"; // 2  
v.pop_back();  
cout << v.back() << "\n"; // 5
```

El siguiente código crea un vector con cinco elementos:

```
vector<int> v = {2,4,2,5,1};
```

Otra forma de crear un vector es dar el número de elementos y el valor inicial de cada elemento:

```
// size 10, initial value 0  
vector<int> v(10);
```

```
// size 10, initial value 5  
vector<int> v(10, 5);
```

La implementación interna de un vector usa un arreglo ordinario. Si el tamaño del vector aumenta y el arreglo se vuelve demasiado pequeño, se reserva un nuevo arreglo y todos los elementos se mueven al nuevo arreglo. Sin embargo, esto no ocurre a menudo y la complejidad temporal promedio de `push_back` es $O(1)$.

La estructura `string` es también un arreglo dinámico que puede usarse casi como un vector. Además, hay una sintaxis especial para las cadenas que no está disponible en otras estructuras de datos. Las cadenas pueden combinarse usando el símbolo `+`. La función `substr(k,x)` devuelve la subcadena que comienza en la posición *k* y tiene longitud *x*, y la función `find(t)` encuentra la posición de la primera aparición de una subcadena *t*.

El siguiente código presenta algunas operaciones con cadenas:

```
string a = "hatti";  
string b = a+a;  
cout << b << "\n"; // hattihatti  
b[5] = 'v';
```

```
cout << b << "\n"; // hattivatti
string c = b.substr(3,4);
cout << c << "\n"; // tiva
```

4.2. Estructuras de conjunto

Un **conjunto** es una estructura de datos que mantiene una colección de elementos. Las operaciones básicas de los conjuntos son la inserción, la búsqueda y la eliminación de elementos.

La biblioteca estándar de C++ contiene dos implementaciones de conjunto: La estructura `set` se basa en un árbol binario balanceado y sus operaciones funcionan en tiempo $O(\log n)$. La estructura `unordered_set` usa hashing, y sus operaciones funcionan en tiempo $O(1)$ en promedio.

La elección de qué implementación de conjunto usar suele ser cuestión de gustos. El beneficio de la estructura `set` es que mantiene el orden de los elementos y ofrece funciones que no están disponibles en `unordered_set`. Por otro lado, `unordered_set` puede ser más eficiente.

El siguiente código crea un conjunto que contiene enteros, y muestra algunas de las operaciones. La función `insert` agrega un elemento al conjunto, la función `count` devuelve el número de apariciones de un elemento en el conjunto, y la función `erase` elimina un elemento del conjunto.

```
set<int> s;
s.insert(3);
s.insert(2);
s.insert(5);
cout << s.count(3) << "\n"; // 1
cout << s.count(4) << "\n"; // 0
s.erase(3);
s.insert(4);
cout << s.count(3) << "\n"; // 0
cout << s.count(4) << "\n"; // 1
```

Un conjunto puede usarse en su mayoría como un vector, pero no es posible acceder a los elementos usando la notación `[]`. El siguiente código crea un conjunto, imprime el número de elementos que contiene, y luego recorre todos los elementos:

```
set<int> s = {2,5,6,8};
cout << s.size() << "\n"; // 4
for (auto x : s) {
    cout << x << "\n";
}
```

Una propiedad importante de los conjuntos es que todos sus elementos son *distintos*. Así, la función `count` siempre devuelve 0 (el elemento no está en el

conjunto) o 1 (el elemento está en el conjunto), y la función `insert` nunca agrega un elemento al conjunto si ya está ahí. El siguiente código ilustra esto:

```
set<int> s;
s.insert(5);
s.insert(5);
s.insert(5);
cout << s.count(5) << "\n"; // 1
```

C++ también contiene las estructuras `multiset` y `unordered_multiset` que por lo demás funcionan como `set` y `unordered_set` pero pueden contener varias instancias de un elemento. Por ejemplo, en el siguiente código las tres instancias del número 5 se agregan a un `multiset`:

```
multiset<int> s;
s.insert(5);
s.insert(5);
s.insert(5);
cout << s.count(5) << "\n"; // 3
```

La función `erase` elimina todas las instancias de un elemento de un `multiset`:

```
s.erase(5);
cout << s.count(5) << "\n"; // 0
```

A menudo, solo debería eliminarse una instancia, lo que puede hacerse de la siguiente manera:

```
s.erase(s.find(5));
cout << s.count(5) << "\n"; // 2
```

4.3. Estructuras de mapa

Un **mapa** es un arreglo generalizado que consta de pares clave-valor. Mientras que las claves en un arreglo ordinario son siempre los enteros consecutivos $0, 1, \dots, n-1$, donde n es el tamaño del arreglo, las claves en un mapa pueden ser de cualquier tipo de dato y no tienen que ser valores consecutivos.

La biblioteca estándar de C++ contiene dos implementaciones de mapa que corresponden a las implementaciones de conjunto: la estructura `map` se basa en un árbol binario balanceado y acceder a los elementos toma $O(\log n)$ de tiempo, mientras que la estructura `unordered_map` usa hashing y acceder a los elementos toma $O(1)$ de tiempo en promedio.

El siguiente código crea un mapa donde las claves son cadenas y los valores son enteros:

```
map<string, int> m;
m["monkey"] = 4;
```



```
m["banana"] = 3;
m["harpsichord"] = 9;
cout << m["banana"] << "\n"; // 3
```

Si se solicita el valor de una clave pero el mapa no la contiene, la clave se agrega automáticamente al mapa con un valor por defecto. Por ejemplo, en el siguiente código, la clave "aybaltu" con valor 0 se agrega al mapa.

```
map<string,int> m;
cout << m["aybaltu"] << "\n"; // 0
```

La función `count` comprueba si una clave existe en un mapa:

```
if (m.count("aybaltu")) {
    // key exists
}
```

El siguiente código imprime todas las claves y valores de un mapa:

```
for (auto x : m) {
    cout << x.first << " " << x.second << "\n";
}
```

4.4. Iteradores y rangos

Muchas funciones de la biblioteca estándar de C++ operan con iteradores. Un **iterador** es una variable que apunta a un elemento de una estructura de datos.

Los iteradores de uso frecuente `begin` y `end` definen un rango que contiene todos los elementos de una estructura de datos. El iterador `begin` apunta al primer elemento de la estructura de datos, y el iterador `end` apunta a la posición *posterior* al último elemento. La situación se ve de la siguiente manera:

```

{ 3, 4, 6, 8, 12, 13, 14, 17 }
  ↑                               ↑
  s.begin()                       s.end()
```

Nota la asimetría en los iteradores: `s.begin()` apunta a un elemento de la estructura de datos, mientras que `s.end()` apunta fuera de la estructura de datos. Así, el rango definido por los iteradores es *semiabierto*.

Trabajar con rangos

Los iteradores se usan en las funciones de la biblioteca estándar de C++ a las que se les da un rango de elementos de una estructura de datos. Normalmente, queremos procesar todos los elementos de una estructura de datos, así que se le dan a la función los iteradores `begin` y `end`.

Por ejemplo, el siguiente código ordena un vector usando la función `sort`, luego invierte el orden de los elementos usando la función `reverse`, y finalmente mezcla el orden de los elementos usando la función `random_shuffle`.

```
sort(v.begin(), v.end());  
reverse(v.begin(), v.end());  
random_shuffle(v.begin(), v.end());
```

Estas funciones también pueden usarse con un arreglo ordinario. En este caso, a las funciones se les dan punteros al arreglo en lugar de iteradores:

```
sort(a, a+n);
reverse(a, a+n);
random_shuffle(a, a+n);
```

Iteradores de conjunto

Los iteradores se usan a menudo para acceder a los elementos de un conjunto. El siguiente código crea un iterador `it` que apunta al elemento más pequeño de un conjunto:

```
set<int>::iterator it = s.begin();
```

Una forma más corta de escribir el código es la siguiente:

```
auto it = s.begin();
```

Al elemento al que apunta un iterador se puede acceder usando el símbolo `*`. Por ejemplo, el siguiente código imprime el primer elemento del conjunto:

```
auto it = s.begin();
cout << *it << "\n";
```

Los iteradores pueden moverse usando los operadores `++` (hacia adelante) y `--` (hacia atrás), lo que significa que el iterador se mueve al siguiente o al anterior elemento del conjunto.

El siguiente código imprime todos los elementos en orden creciente:

```
for (auto it = s.begin(); it != s.end(); it++) {
    cout << *it << "\n";
}
```

El siguiente código imprime el elemento más grande del conjunto:

```
auto it = s.end(); it--;
cout << *it << "\n";
```

La función `find(x)` devuelve un iterador que apunta a un elemento cuyo valor es `x`. Sin embargo, si el conjunto no contiene `x`, el iterador será `end`.

```
auto it = s.find(x);
if (it == s.end()) {
    // x is not found
}
```

La función `lower_bound(x)` devuelve un iterador al elemento más pequeño del conjunto cuyo valor es *al menos* `x`, y la función `upper_bound(x)` devuelve un iterador al elemento más pequeño del conjunto cuyo valor es *mayor que* `x`. En ambas funciones, si tal elemento no existe, el valor devuelto es `end`. Estas

funciones no son compatibles con la estructura `unordered_set` que no mantiene el orden de los elementos.

Por ejemplo, el siguiente código encuentra el elemento más cercano a x :

```
auto it = s.lower_bound(x);
if (it == s.begin()) {
    cout << *it << "\n";
} else if (it == s.end()) {
    it--;
    cout << *it << "\n";
} else {
    int a = *it; it--;
    int b = *it;
    if (x-b < a-x) cout << b << "\n";
    else cout << a << "\n";
}
```

El código supone que el conjunto no está vacío, y recorre todos los casos posibles usando un iterador `it`. Primero, el iterador apunta al elemento más pequeño cuyo valor es al menos x . Si `it` es igual a `begin`, el elemento correspondiente es el más cercano a x . Si `it` es igual a `end`, el elemento más grande del conjunto es el más cercano a x . Si ninguno de los casos anteriores se cumple, el elemento más cercano a x es o bien el elemento que corresponde a `it` o el elemento anterior.

4.5. Otras estructuras

Bitset

Un **bitset** es un arreglo cuyo cada valor es 0 o 1. Por ejemplo, el siguiente código crea un `bitset` que contiene 10 elementos:

```
bitset<10> s;
s[1] = 1;
s[3] = 1;
s[4] = 1;
s[7] = 1;
cout << s[4] << "\n"; // 1
cout << s[5] << "\n"; // 0
```

El beneficio de usar `bitsets` es que requieren menos memoria que los arreglos ordinarios, porque cada elemento de un `bitset` solo usa un bit de memoria. Por ejemplo, si se almacenan n bits en un arreglo de `int`, se usarán $32n$ bits de memoria, pero un `bitset` correspondiente solo requiere n bits de memoria. Además, los valores de un `bitset` pueden manipularse de forma eficiente usando operadores de bits, lo que hace posible optimizar algoritmos usando `bitsets`.

El siguiente código muestra otra forma de crear el `bitset` anterior:

```
bitset<10> s(string("0010011010")); // from right to left
```

```
cout << s[4] << "\n"; // 1
cout << s[5] << "\n"; // 0
```

La función `count` devuelve el número de unos del `bitset`:

```
bitset<10> s(string("0010011010"));
cout << s.count() << "\n"; // 4
```

El siguiente código muestra ejemplos del uso de operaciones de bits:

```
bitset<10> a(string("0010110110"));
bitset<10> b(string("1011011000"));
cout << (a&b) << "\n"; // 0010010000
cout << (a|b) << "\n"; // 1011111110
cout << (a^b) << "\n"; // 1001101110
```

Deque

Un **deque** es un arreglo dinámico cuyo tamaño puede cambiarse de forma eficiente en ambos extremos del arreglo. Como un vector, un deque ofrece las funciones `push_back` y `pop_back`, pero también incluye las funciones `push_front` y `pop_front` que no están disponibles en un vector.

Un deque puede usarse de la siguiente manera:

```
deque<int> d;
d.push_back(5); // [5]
d.push_back(2); // [5,2]
d.push_front(3); // [3,5,2]
d.pop_back(); // [3,5]
d.pop_front(); // [5]
```

La implementación interna de un deque es más compleja que la de un vector, y por esta razón, un deque es más lento que un vector. Aun así, tanto agregar como eliminar elementos toma $O(1)$ de tiempo en promedio en ambos extremos.

Pila

Una **pila** es una estructura de datos que ofrece dos operaciones de tiempo $O(1)$: agregar un elemento en la cima, y eliminar un elemento de la cima. Solo es posible acceder al elemento de la cima de una pila.

El siguiente código muestra cómo puede usarse una pila:

```
stack<int> s;
s.push(3);
s.push(2);
s.push(5);
cout << s.top(); // 5
s.pop();
```

```
cout << s.top(); // 2
```

Cola

Una **cola** también ofrece dos operaciones de tiempo $O(1)$: agregar un elemento al final de la cola, y eliminar el primer elemento de la cola. Solo es posible acceder al primer y al último elemento de una cola.

El siguiente código muestra cómo puede usarse una cola:

```
queue<int> q;  
q.push(3);  
q.push(2);  
q.push(5);  
cout << q.front(); // 3  
q.pop();  
cout << q.front(); // 2
```

Cola de prioridad

Una **cola de prioridad** mantiene un conjunto de elementos. Las operaciones admitidas son la inserción y, según el tipo de la cola, la recuperación y eliminación del elemento mínimo o máximo. La inserción y la eliminación toman $O(\log n)$ de tiempo, y la recuperación toma $O(1)$ de tiempo.

Aunque un conjunto ordenado admite eficientemente todas las operaciones de una cola de prioridad, el beneficio de usar una cola de prioridad es que tiene factores constantes más pequeños. Una cola de prioridad se implementa normalmente usando una estructura de montículo (heap) que es mucho más simple que un árbol binario balanceado usado en un conjunto ordenado.

Por defecto, los elementos de una cola de prioridad de C++ se ordenan en orden decreciente, y es posible encontrar y eliminar el elemento más grande de la cola. El siguiente código ilustra esto:

```
priority_queue<int> q;  
q.push(3);  
q.push(5);  
q.push(7);  
q.push(2);  
cout << q.top() << "\n"; // 7  
q.pop();  
cout << q.top() << "\n"; // 5  
q.pop();  
q.push(6);  
cout << q.top() << "\n"; // 6  
q.pop();
```

Si queremos crear una cola de prioridad que admita encontrar y eliminar el elemento más pequeño, podemos hacerlo de la siguiente manera:

```
priority_queue<int,vector<int>,greater<int>> q;
```

Estructuras de datos basadas en políticas

El compilador g++ también admite algunas estructuras de datos que no forman parte de la biblioteca estándar de C++. Tales estructuras se llaman estructuras de datos *basadas en políticas* (policy-based). Para usar estas estructuras, deben agregarse las siguientes líneas al código:

```
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;
```

Después de esto, podemos definir una estructura de datos `indexed_set` que es como set pero puede indexarse como un arreglo. La definición para valores `int` es la siguiente:

```
typedef tree<int,null_type,less<int>,rb_tree_tag,
            tree_order_statistics_node_update> indexed_set;
```

Ahora podemos crear un conjunto de la siguiente manera:

```
indexed_set s;
s.insert(2);
s.insert(3);
s.insert(7);
s.insert(9);
```

La particularidad de este conjunto es que tenemos acceso a los índices que los elementos tendrían en un arreglo ordenado. La función `find_by_order` devuelve un iterador al elemento en una posición dada:

```
auto x = s.find_by_order(2);
cout << *x << "\n"; // 7
```

Y la función `order_of_key` devuelve la posición de un elemento dado:

```
cout << s.order_of_key(7) << "\n"; // 2
```

Si el elemento no aparece en el conjunto, obtenemos la posición que el elemento tendría en el conjunto:

```
cout << s.order_of_key(6) << "\n"; // 2
cout << s.order_of_key(8) << "\n"; // 3
```

Ambas funciones funcionan en tiempo logarítmico.

4.6. Comparación con el ordenamiento

A menudo es posible resolver un problema usando estructuras de datos o usando ordenamiento. A veces hay diferencias notables en la eficiencia real de estos enfoques, que pueden estar ocultas en sus complejidades temporales.

Consideremos un problema donde se nos dan dos listas A y B que ambas contienen n elementos. Nuestra tarea es calcular el número de elementos que pertenecen a ambas listas. Por ejemplo, para las listas

$$A = [5, 2, 8, 9] \quad \text{y} \quad B = [3, 2, 9, 5],$$

la respuesta es 3 porque los números 2, 5 y 9 pertenecen a ambas listas.

Una solución directa al problema es recorrer todos los pares de elementos en tiempo $O(n^2)$, pero a continuación nos centraremos en algoritmos más eficientes.

Algoritmo 1

Construimos un conjunto de los elementos que aparecen en A , y después de esto, recorremos los elementos de B y comprobamos para cada elemento si también pertenece a A . Esto es eficiente porque los elementos de A están en un conjunto. Usando la estructura `set`, la complejidad temporal del algoritmo es $O(n \log n)$.

Algoritmo 2

No es necesario mantener un conjunto ordenado, así que en lugar de la estructura `set` también podemos usar la estructura `unordered_set`. Esta es una forma fácil de hacer el algoritmo más eficiente, porque solo tenemos que cambiar la estructura de datos subyacente. La complejidad temporal del nuevo algoritmo es $O(n)$.

Algoritmo 3

En lugar de estructuras de datos, podemos usar ordenamiento. Primero, ordenamos ambas listas A y B . Después de esto, recorremos ambas listas al mismo tiempo y encontramos los elementos comunes. La complejidad temporal del ordenamiento es $O(n \log n)$, y el resto del algoritmo funciona en tiempo $O(n)$, así que la complejidad temporal total es $O(n \log n)$.

Comparación de eficiencia

La siguiente tabla muestra qué tan eficientes son los algoritmos anteriores cuando n varía y los elementos de las listas son enteros aleatorios entre $1 \dots 10^9$:

n	Algoritmo 1	Algoritmo 2	Algoritmo 3
10^6	1.5 s	0.3 s	0.2 s
$2 \cdot 10^6$	3.7 s	0.8 s	0.3 s
$3 \cdot 10^6$	5.7 s	1.3 s	0.5 s
$4 \cdot 10^6$	7.7 s	1.7 s	0.7 s
$5 \cdot 10^6$	10.0 s	2.3 s	0.9 s

Los Algoritmos 1 y 2 son iguales excepto que usan estructuras de conjunto distintas. En este problema, esta elección tiene un efecto importante en el tiempo de ejecución, porque el Algoritmo 2 es 4–5 veces más rápido que el Algoritmo 1.

Sin embargo, el algoritmo más eficiente es el Algoritmo 3 que usa ordenamiento. Solo usa la mitad del tiempo comparado con el Algoritmo 2. Curiosamente, la complejidad temporal tanto del Algoritmo 1 como del Algoritmo 3 es $O(n \log n)$, pero a pesar de esto, el Algoritmo 3 es diez veces más rápido. Esto puede explicarse por el hecho de que el ordenamiento es un procedimiento simple y se realiza solo una vez al principio del Algoritmo 3, y el resto del algoritmo funciona en tiempo lineal. Por otro lado, el Algoritmo 1 mantiene un complejo árbol binario balanceado durante todo el algoritmo.

Capítulo 5

Búsqueda completa

La **búsqueda completa** es un método general que puede usarse para resolver casi cualquier problema de algoritmos. La idea es generar todas las soluciones posibles al problema por fuerza bruta, y luego seleccionar la mejor solución o contar el número de soluciones, según el problema.

La búsqueda completa es una buena técnica si hay tiempo suficiente para recorrer todas las soluciones, porque la búsqueda suele ser fácil de implementar y siempre da la respuesta correcta. Si la búsqueda completa es demasiado lenta, pueden necesitarse otras técnicas, como los algoritmos voraces o la programación dinámica.

5.1. Generar subconjuntos

Primero consideramos el problema de generar todos los subconjuntos de un conjunto de n elementos. Por ejemplo, los subconjuntos de $\{0, 1, 2\}$ son $\emptyset$, $\{0\}$, $\{1\}$, $\{2\}$, $\{0, 1\}$, $\{0, 2\}$, $\{1, 2\}$ y $\{0, 1, 2\}$. Hay dos métodos comunes para generar subconjuntos: podemos realizar una búsqueda recursiva o aprovechar la representación en bits de los enteros.

Método 1

Una forma elegante de recorrer todos los subconjuntos de un conjunto es usar recursión. La siguiente función `search` genera los subconjuntos del conjunto $\{0, 1, \dots, n-1\}$. La función mantiene un vector `subset` que contendrá los elementos de cada subconjunto. La búsqueda comienza cuando la función se llama con el parámetro 0.

```
void search(int k) {
    if (k == n) {
        // process subset
    } else {
        search(k+1);
        subset.push_back(k);
        search(k+1);
        subset.pop_back();
    }
}
```

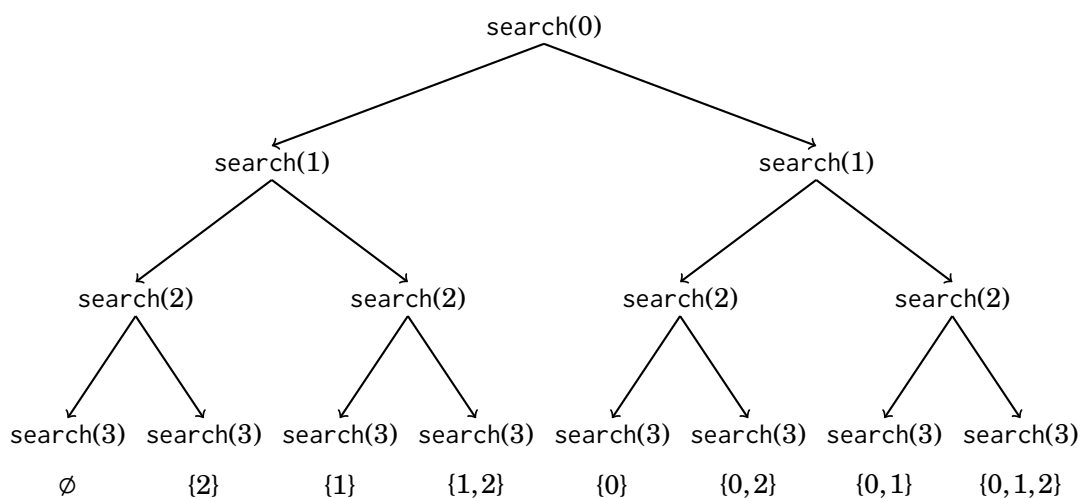
```

    }
}

```

Cuando la función `search` se llama con el parámetro k , decide si incluir el elemento k en el subconjunto o no, y en ambos casos, luego se llama a sí misma con el parámetro $k + 1$. Sin embargo, si $k = n$, la función se da cuenta de que todos los elementos han sido procesados y se ha generado un subconjunto.

El siguiente árbol ilustra las llamadas a la función cuando $n = 3$. Siempre podemos elegir la rama izquierda (k no se incluye en el subconjunto) o la rama derecha (k se incluye en el subconjunto).



Método 2

Otra forma de generar subconjuntos se basa en la representación en bits de los enteros. Cada subconjunto de un conjunto de n elementos puede representarse como una secuencia de n bits, que corresponde a un entero entre $0 \dots 2^n - 1$. Los unos en la secuencia de bits indican qué elementos se incluyen en el subconjunto.

La convención usual es que el último bit corresponde al elemento 0, el penúltimo bit corresponde al elemento 1, y así sucesivamente. Por ejemplo, la representación en bits de 25 es 11001, que corresponde al subconjunto $\{0, 3, 4\}$.

El siguiente código recorre los subconjuntos de un conjunto de n elementos

```

for (int b = 0; b < (1<<n); b++) {
    // process subset
}

```

El siguiente código muestra cómo podemos encontrar los elementos de un subconjunto que corresponde a una secuencia de bits. Al procesar cada subconjunto, el código construye un vector que contiene los elementos del subconjunto.

```

for (int b = 0; b < (1<<n); b++) {
    vector<int> subset;
    for (int i = 0; i < n; i++) {
        if (b & (1<<i)) subset.push_back(i);
    }
}

```

```
}  
}
```

5.2. Generar permutaciones

A continuación consideramos el problema de generar todas las permutaciones de un conjunto de n elementos. Por ejemplo, las permutaciones de $\{0, 1, 2\}$ son $(0, 1, 2)$, $(0, 2, 1)$, $(1, 0, 2)$, $(1, 2, 0)$, $(2, 0, 1)$ y $(2, 1, 0)$. De nuevo, hay dos enfoques: podemos usar recursión o recorrer las permutaciones de forma iterativa.

Método 1

Como los subconjuntos, las permutaciones pueden generarse usando recursión. La siguiente función `search` recorre las permutaciones del conjunto $\{0, 1, \dots, n-1\}$. La función construye un vector `permutation` que contiene la permutación, y la búsqueda comienza cuando la función se llama sin parámetros.

```
void search() {  
    if (permutation.size() == n) {  
        // process permutation  
    } else {  
        for (int i = 0; i < n; i++) {  
            if (chosen[i]) continue;  
            chosen[i] = true;  
            permutation.push_back(i);  
            search();  
            chosen[i] = false;  
            permutation.pop_back();  
        }  
    }  
}
```

Cada llamada a la función agrega un nuevo elemento a `permutation`. El arreglo `chosen` indica qué elementos ya están incluidos en la permutación. Si el tamaño de `permutation` es igual al tamaño del conjunto, se ha generado una permutación.

Método 2

Otro método para generar permutaciones es comenzar con la permutación $\{0, 1, \dots, n-1\}$ y usar repetidamente una función que construye la siguiente permutación en orden creciente. La biblioteca estándar de C++ contiene la función `next_permutation` que puede usarse para esto:

```
vector<int> permutation;  
for (int i = 0; i < n; i++) {
```

```

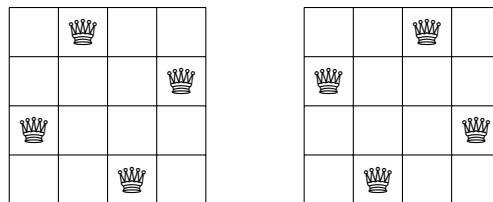
    permutation.push_back(i);
}
do {
    // process permutation
} while (next_permutation(permutation.begin(), permutation.end()));

```

5.3. Backtracking

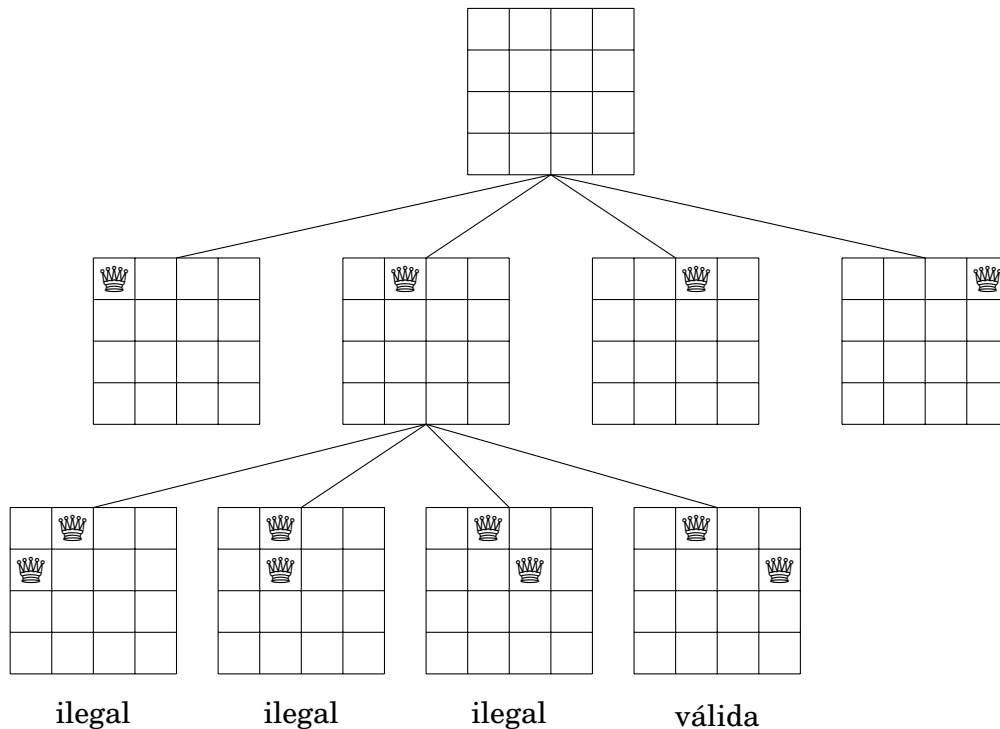
Un algoritmo de **backtracking** (vuelta atrás) comienza con una solución vacía y extiende la solución paso a paso. La búsqueda recorre recursivamente todas las formas distintas en que puede construirse una solución.

Como ejemplo, considera el problema de calcular el número de formas en que n reinas pueden colocarse en un tablero de ajedrez de $n \times n$ de modo que ninguna reina ataque a otra. Por ejemplo, cuando $n = 4$, hay dos soluciones posibles:



El problema puede resolverse usando backtracking colocando reinas en el tablero fila por fila. Más precisamente, se colocará exactamente una reina en cada fila de modo que ninguna reina ataque a ninguna de las reinas colocadas antes. Se ha encontrado una solución cuando todas las n reinas han sido colocadas en el tablero.

Por ejemplo, cuando $n = 4$, algunas soluciones parciales generadas por el algoritmo de backtracking son las siguientes:



En el nivel inferior, las tres primeras configuraciones son ilegales, porque las reinas se atacan entre sí. Sin embargo, la cuarta configuración es válida y puede extenderse a una solución completa colocando dos reinas más en el tablero. Solo hay una forma de colocar las dos reinas restantes.

El algoritmo puede implementarse de la siguiente manera:

```
void search(int y) {
    if (y == n) {
        count++;
        return;
    }
    for (int x = 0; x < n; x++) {
        if (column[x] || diag1[x+y] || diag2[x-y+n-1]) continue;
        column[x] = diag1[x+y] = diag2[x-y+n-1] = 1;
        search(y+1);
        column[x] = diag1[x+y] = diag2[x-y+n-1] = 0;
    }
}
```

La búsqueda comienza llamando a `search(0)`. El tamaño del tablero es $n \times n$, y el código calcula el número de soluciones en `count`.

El código supone que las filas y columnas del tablero se numeran de 0 a $n-1$. Cuando la función `search` se llama con el parámetro y , coloca una reina en la fila y y luego se llama a sí misma con el parámetro $y+1$. Luego, si $y=n$, se ha encontrado una solución y la variable `count` se incrementa en uno.

El arreglo `column` lleva registro de las columnas que contienen una reina, y los arreglos `diag1` y `diag2` llevan registro de las diagonales. No se permite agregar

otra reina a una columna o diagonal que ya contiene una reina. Por ejemplo, las columnas y diagonales del tablero de 4×4 se numeran de la siguiente manera:

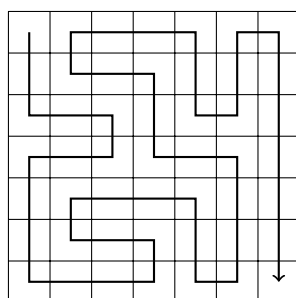
0	1	2	3	0	1	2	3	3	4	5	6
0	1	2	3	1	2	3	4	2	3	4	5
0	1	2	3	2	3	4	5	1	2	3	4
0	1	2	3	3	4	5	6	0	1	2	3
column				diag1				diag2			

Sea $q(n)$ el número de formas de colocar n reinas en un tablero de ajedrez de $n \times n$. El algoritmo de backtracking anterior nos dice que, por ejemplo, $q(8) = 92$. Cuando n aumenta, la búsqueda se vuelve rápidamente lenta, porque el número de soluciones aumenta exponencialmente. Por ejemplo, calcular $q(16) = 14772512$ usando el algoritmo anterior ya toma aproximadamente un minuto en una computadora moderna¹.

5.4. Podar la búsqueda

A menudo podemos optimizar el backtracking podando el árbol de búsqueda. La idea es agregar "inteligencia" al algoritmo para que note tan pronto como sea posible si una solución parcial no puede extenderse a una solución completa. Tales optimizaciones pueden tener un efecto tremendo en la eficiencia de la búsqueda.

Consideremos el problema de calcular el número de caminos en una cuadrícula de $n \times n$ desde la esquina superior izquierda hasta la esquina inferior derecha tales que el camino visite cada casilla exactamente una vez. Por ejemplo, en una cuadrícula de 7×7 , hay 111712 de tales caminos. Uno de los caminos es el siguiente:



Nos centramos en el caso de 7×7 , porque su nivel de dificultad es apropiado para nuestras necesidades. Comenzamos con un algoritmo de backtracking directo, y luego lo optimizamos paso a paso usando observaciones de cómo puede podarse la búsqueda. Después de cada optimización, medimos el tiempo de ejecución del algoritmo y el número de llamadas recursivas, para ver claramente el efecto de cada optimización en la eficiencia de la búsqueda.

¹No se conoce ninguna forma de calcular eficientemente valores más grandes de $q(n)$. El récord actual es $q(27) = 234907967154122528$, calculado en 2016 [55].

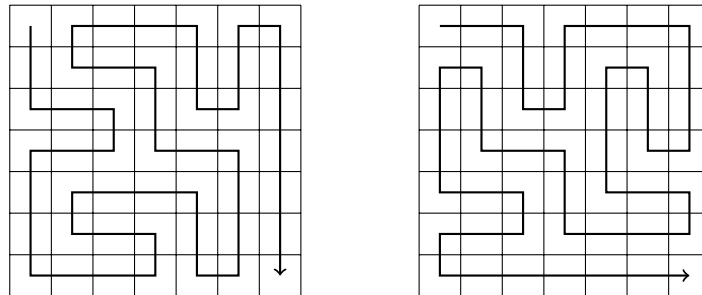
Algoritmo básico

La primera versión del algoritmo no contiene ninguna optimización. Simplemente usamos backtracking para generar todos los caminos posibles desde la esquina superior izquierda hasta la esquina inferior derecha y contamos el número de tales caminos.

- tiempo de ejecución: 483 segundos
- número de llamadas recursivas: 76 mil millones

Optimización 1

En cualquier solución, primero nos movemos un paso hacia abajo o hacia la derecha. Siempre hay dos caminos que son simétricos respecto a la diagonal de la cuadrícula después del primer paso. Por ejemplo, los siguientes caminos son simétricos:

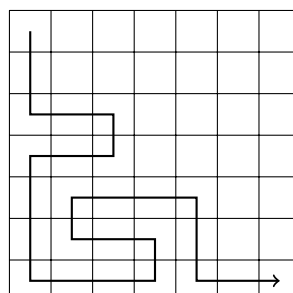


Por lo tanto, podemos decidir que siempre nos movemos primero un paso hacia abajo (o hacia la derecha), y finalmente multiplicar el número de soluciones por dos.

- tiempo de ejecución: 244 segundos
- número de llamadas recursivas: 38 mil millones

Optimización 2

Si el camino llega a la casilla inferior derecha antes de haber visitado todas las demás casillas de la cuadrícula, está claro que no será posible completar la solución. Un ejemplo de esto es el siguiente camino:

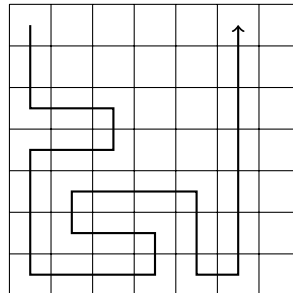


Usando esta observación, podemos terminar la búsqueda de inmediato si llegamos a la casilla inferior derecha demasiado pronto.

- tiempo de ejecución: 119 segundos
- número de llamadas recursivas: 20 mil millones

Optimización 3

Si el camino toca una pared y puede girar a la izquierda o a la derecha, la cuadrícula se divide en dos partes que contienen casillas no visitadas. Por ejemplo, en la siguiente situación, el camino puede girar a la izquierda o a la derecha:

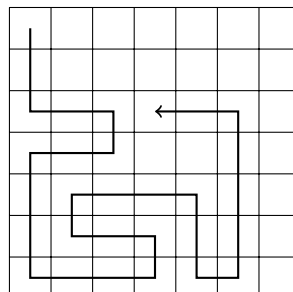


En este caso, ya no podemos visitar todas las casillas, así que podemos terminar la búsqueda. Esta optimización es muy útil:

- tiempo de ejecución: 1.8 segundos
- número de llamadas recursivas: 221 millones

Optimización 4

La idea de la Optimización 3 puede generalizarse: si el camino no puede continuar hacia adelante pero puede girar a la izquierda o a la derecha, la cuadrícula se divide en dos partes que ambas contienen casillas no visitadas. Por ejemplo, considera el siguiente camino:



Está claro que ya no podemos visitar todas las casillas, así que podemos terminar la búsqueda. Después de esta optimización, la búsqueda es muy eficiente:

- tiempo de ejecución: 0.6 segundos

- número de llamadas recursivas: 69 millones

Ahora es un buen momento para dejar de optimizar el algoritmo y ver lo que hemos logrado. El tiempo de ejecución del algoritmo original era de 483 segundos, y ahora, después de las optimizaciones, el tiempo de ejecución es de solo 0.6 segundos. Así, el algoritmo se volvió casi 1000 veces más rápido después de las optimizaciones.

Este es un fenómeno habitual en el backtracking, porque el árbol de búsqueda suele ser grande y hasta observaciones simples pueden podar la búsqueda de forma efectiva. Especialmente útiles son las optimizaciones que ocurren durante los primeros pasos del algoritmo, es decir, en la parte superior del árbol de búsqueda.

5.5. Encuentro en el medio

El **encuentro en el medio** (meet in the middle) es una técnica donde el espacio de búsqueda se divide en dos partes de tamaño aproximadamente igual. Se realiza una búsqueda separada para cada una de las partes, y finalmente se combinan los resultados de las búsquedas.

La técnica puede usarse si hay una forma eficiente de combinar los resultados de las búsquedas. En tal situación, las dos búsquedas pueden requerir menos tiempo que una sola búsqueda grande. Normalmente, podemos convertir un factor de 2^n en un factor de $2^{n/2}$ usando la técnica del encuentro en el medio.

Como ejemplo, considera un problema donde se nos da una lista de n números y un número x , y queremos averiguar si es posible elegir algunos números de la lista de modo que su suma sea x . Por ejemplo, dada la lista $[2, 4, 5, 9]$ y $x = 15$, podemos elegir los números $[2, 4, 9]$ para obtener $2 + 4 + 9 = 15$. Sin embargo, si $x = 10$ para la misma lista, no es posible formar la suma.

Un algoritmo simple para el problema es recorrer todos los subconjuntos de los elementos y comprobar si la suma de alguno de los subconjuntos es x . El tiempo de ejecución de tal algoritmo es $O(2^n)$, porque hay 2^n subconjuntos. Sin embargo, usando la técnica del encuentro en el medio, podemos lograr un algoritmo más eficiente de tiempo $O(2^{n/2})^2$. Nota que $O(2^n)$ y $O(2^{n/2})$ son complejidades distintas porque $2^{n/2}$ es igual a $\sqrt{2^n}$.

La idea es dividir la lista en dos listas A y B tales que ambas listas contengan aproximadamente la mitad de los números. La primera búsqueda genera todos los subconjuntos de A y almacena sus sumas en una lista S_A . Correspondientemente, la segunda búsqueda crea una lista S_B a partir de B . Después de esto, basta con comprobar si es posible elegir un elemento de S_A y otro elemento de S_B tales que su suma sea x . Esto es posible exactamente cuando hay una forma de formar la suma x usando los números de la lista original.

Por ejemplo, supongamos que la lista es $[2, 4, 5, 9]$ y $x = 15$. Primero, dividimos la lista en $A = [2, 4]$ y $B = [5, 9]$. Después de esto, creamos las listas $S_A = [0, 2, 4, 6]$

²Esta idea fue introducida en 1974 por E. Horowitz y S. Sahni [39].

y $S_B = [0, 5, 9, 14]$. En este caso, la suma $x = 15$ es posible de formar, porque S_A contiene la suma 6, S_B contiene la suma 9, y $6 + 9 = 15$. Esto corresponde a la solución $[2, 4, 9]$.

Podemos implementar el algoritmo de modo que su complejidad temporal sea $O(2^{n/2})$. Primero, generamos listas *ordenadas* S_A y S_B , lo que puede hacerse en tiempo $O(2^{n/2})$ usando una técnica similar a la mezcla. Después de esto, como las listas están ordenadas, podemos comprobar en tiempo $O(2^{n/2})$ si la suma x puede crearse a partir de S_A y S_B .

Capítulo 6

Algoritmos voraces

Un **algoritmo voraz** construye una solución al problema tomando siempre la elección que parece la mejor en ese momento. Un algoritmo voraz nunca deshace sus elecciones, sino que construye directamente la solución final. Por esta razón, los algoritmos voraces suelen ser muy eficientes.

La dificultad al diseñar algoritmos voraces es encontrar una estrategia voraz que siempre produzca una solución óptima al problema. Las elecciones localmente óptimas en un algoritmo voraz deberían ser también globalmente óptimas. A menudo es difícil argumentar que un algoritmo voraz funciona.

6.1. Problema de las monedas

Como primer ejemplo, consideramos un problema donde se nos da un conjunto de monedas y nuestra tarea es formar una cantidad de dinero n usando las monedas. Los valores de las monedas son $\text{coins} = \{c_1, c_2, \dots, c_k\}$, y cada moneda puede usarse tantas veces como queramos. ¿Cuál es el número mínimo de monedas necesarias?

Por ejemplo, si las monedas son las monedas de euro (en céntimos)

$$\{1, 2, 5, 10, 20, 50, 100, 200\}$$

y $n = 520$, necesitamos al menos cuatro monedas. La solución óptima es seleccionar las monedas $200 + 200 + 100 + 20$ cuya suma es 520.

Algoritmo voraz

Un algoritmo voraz simple para el problema siempre selecciona la moneda más grande posible, hasta que se haya construido la cantidad de dinero requerida. Este algoritmo funciona en el caso del ejemplo, porque primero seleccionamos dos monedas de 200 céntimos, luego una moneda de 100 céntimos y finalmente una moneda de 20 céntimos. Pero ¿este algoritmo siempre funciona?

Resulta que si las monedas son las monedas de euro, el algoritmo voraz *siempre* funciona, es decir, siempre produce una solución con el menor número posible de monedas. La corrección del algoritmo puede demostrarse de la siguiente manera:

Primero, cada moneda 1, 5, 10, 50 y 100 aparece a lo sumo una vez en una solución óptima, porque si la solución contuviera dos de tales monedas, podríamos reemplazarlas por una moneda y obtener una solución mejor. Por ejemplo, si la solución contuviera las monedas 5 + 5, podríamos reemplazarlas por la moneda 10.

De la misma forma, las monedas 2 y 20 aparecen a lo sumo dos veces en una solución óptima, porque podríamos reemplazar las monedas 2 + 2 + 2 por las monedas 5 + 1 y las monedas 20 + 20 + 20 por las monedas 50 + 10. Además, una solución óptima no puede contener las monedas 2 + 2 + 1 o 20 + 20 + 10, porque podríamos reemplazarlas por las monedas 5 y 50.

Usando estas observaciones, podemos mostrar para cada moneda x que no es posible construir de forma óptima una suma x o cualquier suma mayor usando solo monedas que sean menores que x . Por ejemplo, si $x = 100$, la mayor suma óptima usando las monedas más pequeñas es $50 + 20 + 20 + 5 + 2 + 2 = 99$. Así, el algoritmo voraz que siempre selecciona la moneda más grande produce la solución óptima.

Este ejemplo muestra que puede ser difícil argumentar que un algoritmo voraz funciona, aunque el algoritmo en sí sea simple.

Caso general

En el caso general, el conjunto de monedas puede contener cualquier moneda y el algoritmo voraz *no* necesariamente produce una solución óptima.

Podemos demostrar que un algoritmo voraz no funciona mostrando un contraejemplo donde el algoritmo da una respuesta incorrecta. En este problema podemos encontrar fácilmente un contraejemplo: si las monedas son $\{1, 3, 4\}$ y la suma objetivo es 6, el algoritmo voraz produce la solución $4 + 1 + 1$ mientras que la solución óptima es $3 + 3$.

No se sabe si el problema general de las monedas puede resolverse usando algún algoritmo voraz¹. Sin embargo, como veremos en el Capítulo 7, en algunos casos, el problema general puede resolverse de forma eficiente usando un algoritmo de programación dinámica que siempre da la respuesta correcta.

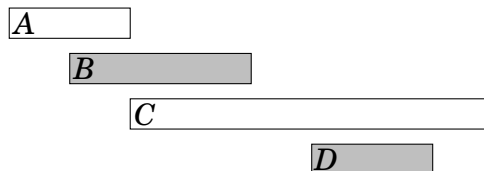
6.2. Planificación

Muchos problemas de planificación pueden resolverse usando algoritmos voraces. Un problema clásico es el siguiente: Dados n eventos con sus tiempos de inicio y fin, encontrar una planificación que incluya tantos eventos como sea posible. No es posible seleccionar un evento parcialmente. Por ejemplo, considera los siguientes eventos:

¹Sin embargo, es posible *comprobar* en tiempo polinómico si el algoritmo voraz presentado en este capítulo funciona para un conjunto dado de monedas [53].

evento	tiempo de inicio	tiempo de fin
<i>A</i>	1	3
<i>B</i>	2	5
<i>C</i>	3	9
<i>D</i>	6	8

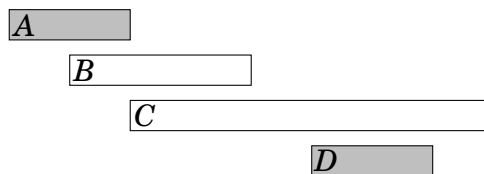
En este caso el número máximo de eventos es dos. Por ejemplo, podemos seleccionar los eventos *B* y *D* de la siguiente manera:



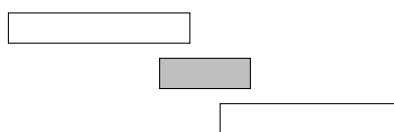
Es posible inventar varios algoritmos voraces para el problema, pero ¿cuál de ellos funciona en todos los casos?

Algoritmo 1

La primera idea es seleccionar los eventos tan *cortos* como sea posible. En el caso del ejemplo este algoritmo selecciona los siguientes eventos:



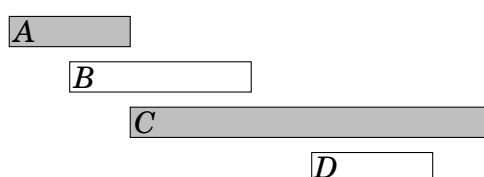
Sin embargo, seleccionar eventos cortos no siempre es una estrategia correcta. Por ejemplo, el algoritmo falla en el siguiente caso:



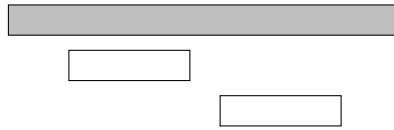
Si seleccionamos el evento corto, solo podemos seleccionar un evento. Sin embargo, sería posible seleccionar ambos eventos largos.

Algoritmo 2

Otra idea es seleccionar siempre el siguiente evento posible que *comience* lo más *temprano* posible. Este algoritmo selecciona los siguientes eventos:



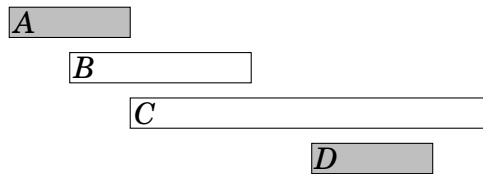
Sin embargo, podemos encontrar un contraejemplo también para este algoritmo. Por ejemplo, en el siguiente caso, el algoritmo solo selecciona un evento:



Si seleccionamos el primer evento, no es posible seleccionar ningún otro evento. Sin embargo, sería posible seleccionar los otros dos eventos.

Algoritmo 3

La tercera idea es seleccionar siempre el siguiente evento posible que *termine* lo más *temprano* posible. Este algoritmo selecciona los siguientes eventos:



Resulta que este algoritmo *siempre* produce una solución óptima. La razón de esto es que siempre es una elección óptima seleccionar primero un evento que termine lo más temprano posible. Después de esto, es una elección óptima seleccionar el siguiente evento usando la misma estrategia, etc., hasta que no podamos seleccionar más eventos.

Una forma de argumentar que el algoritmo funciona es considerar qué ocurre si primero seleccionamos un evento que termina más tarde que el evento que termina lo más temprano posible. Ahora, tendremos a lo sumo un número igual de opciones de cómo podemos seleccionar el siguiente evento. Por lo tanto, seleccionar un evento que termina más tarde nunca puede dar una solución mejor, y el algoritmo voraz es correcto.

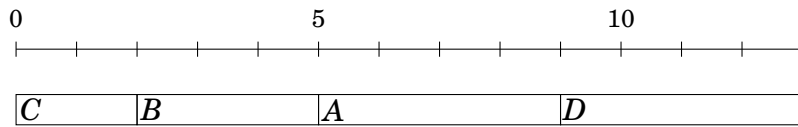
6.3. Tareas y fechas límite

Consideremos ahora un problema donde se nos dan n tareas con duraciones y fechas límite y nuestra tarea es elegir un orden para realizar las tareas. Por cada tarea, ganamos $d - x$ puntos donde d es la fecha límite de la tarea y x es el momento en que terminamos la tarea. ¿Cuál es la mayor puntuación total posible que podemos obtener?

Por ejemplo, supongamos que las tareas son las siguientes:

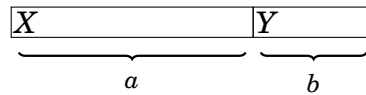
tarea	duración	fecha límite
<i>A</i>	4	2
<i>B</i>	3	5
<i>C</i>	2	7
<i>D</i>	4	5

En este caso, una planificación óptima de las tareas es la siguiente:

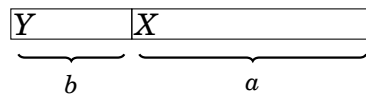


En esta solución, C da 5 puntos, B da 0 puntos, A da -7 puntos y D da -8 puntos, así que la puntuación total es -10 .

Sorprendentemente, la solución óptima al problema no depende para nada de las fechas límite, sino que una estrategia voraz correcta es simplemente realizar las tareas *ordenadas por sus duraciones* en orden creciente. La razón de esto es que si alguna vez realizamos dos tareas una tras otra de modo que la primera tarea tome más tiempo que la segunda tarea, podemos obtener una solución mejor si intercambiamos las tareas. Por ejemplo, considera la siguiente planificación:



Aquí $a > b$, así que deberíamos intercambiar las tareas:



Ahora X da b puntos menos e Y da a puntos más, así que la puntuación total aumenta en $a - b > 0$. En una solución óptima, para cualesquiera dos tareas consecutivas, debe cumplirse que la tarea más corta va antes que la tarea más larga. Así, las tareas deben realizarse ordenadas por sus duraciones.

6.4. Minimizar sumas

A continuación consideramos un problema donde se nos dan n números $a_1, a_2, \dots, a_n$ y nuestra tarea es encontrar un valor x que minimice la suma

$$|a_1 - x|^c + |a_2 - x|^c + \dots + |a_n - x|^c.$$

Nos centramos en los casos $c = 1$ y $c = 2$.

Caso $c = 1$

En este caso, debemos minimizar la suma

$$|a_1 - x| + |a_2 - x| + \dots + |a_n - x|.$$

Por ejemplo, si los números son $[1, 2, 9, 2, 6]$, la mejor solución es seleccionar $x = 2$ que produce la suma

$$|1 - 2| + |2 - 2| + |9 - 2| + |2 - 2| + |6 - 2| = 12.$$

En el caso general, la mejor elección para x es la *mediana* de los números, es decir, el número del medio después de ordenar. Por ejemplo, la lista $[1, 2, 9, 2, 6]$ se convierte en $[1, 2, 2, 6, 9]$ después de ordenar, así que la mediana es 2.

La mediana es una elección óptima, porque si x es menor que la mediana, la suma se hace más pequeña al aumentar x , y si x es mayor que la mediana, la suma se hace más pequeña al disminuir x . Por lo tanto, la solución óptima es que x sea la mediana. Si n es par y hay dos medianas, ambas medianas y todos los valores entre ellas son elecciones óptimas.

Caso $c = 2$

En este caso, debemos minimizar la suma

$$(a_1 - x)^2 + (a_2 - x)^2 + \cdots + (a_n - x)^2.$$

Por ejemplo, si los números son $[1, 2, 9, 2, 6]$, la mejor solución es seleccionar $x = 4$ que produce la suma

$$(1 - 4)^2 + (2 - 4)^2 + (9 - 4)^2 + (2 - 4)^2 + (6 - 4)^2 = 46.$$

En el caso general, la mejor elección para x es el *promedio* de los números. En el ejemplo el promedio es $(1 + 2 + 9 + 2 + 6)/5 = 4$. Este resultado puede deducirse presentando la suma de la siguiente manera:

$$nx^2 - 2x(a_1 + a_2 + \cdots + a_n) + (a_1^2 + a_2^2 + \cdots + a_n^2)$$

La última parte no depende de x , así que podemos ignorarla. Las partes restantes forman una función $nx^2 - 2xs$ donde $s = a_1 + a_2 + \cdots + a_n$. Esta es una parábola que abre hacia arriba con raíces $x = 0$ y $x = 2s/n$, y el valor mínimo es el promedio de las raíces $x = s/n$, es decir, el promedio de los números $a_1, a_2, \dots, a_n$.

6.5. Compresión de datos

Un **código binario** asigna a cada carácter de una cadena una **palabra de código** que consta de bits. Podemos *comprimir* la cadena usando el código binario reemplazando cada carácter por la palabra de código correspondiente. Por ejemplo, el siguiente código binario asigna palabras de código a los caracteres A–D:

carácter	palabra de código
A	00
B	01
C	10
D	11

Este es un código de **longitud constante** lo que significa que la longitud de cada palabra de código es la misma. Por ejemplo, podemos comprimir la cadena AABACDACA de la siguiente manera:

000001001011001000

Usando este código, la longitud de la cadena comprimida es de 18 bits. Sin embargo, podemos comprimir la cadena mejor si usamos un código de **longitud variable** donde las palabras de código pueden tener longitudes distintas. Entonces podemos dar palabras de código cortas para los caracteres que aparecen a menudo y palabras de código largas para los caracteres que aparecen rara vez. Resulta que un código **óptimo** para la cadena anterior es el siguiente:

carácter	palabra de código
A	0
B	110
C	10
D	111

Un código óptimo produce una cadena comprimida que es lo más corta posible. En este caso, la cadena comprimida usando el código óptimo es

001100101110100,

así que solo se necesitan 15 bits en lugar de 18 bits. Así, gracias a un código mejor fue posible ahorrar 3 bits en la cadena comprimida.

Requerimos que ninguna palabra de código sea un prefijo de otra palabra de código. Por ejemplo, no se permite que un código contenga ambas palabras de código 10 y 1011. La razón de esto es que queremos poder generar la cadena original a partir de la cadena comprimida. Si una palabra de código pudiera ser un prefijo de otra palabra de código, esto no siempre sería posible. Por ejemplo, el siguiente código *no* es válido:

carácter	palabra de código
A	10
B	11
C	1011
D	111

Usando este código, no sería posible saber si la cadena comprimida 1011 corresponde a la cadena AB o a la cadena C.

Codificación de Huffman

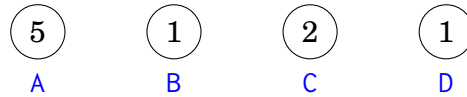
La **codificación de Huffman**² es un algoritmo voraz que construye un código óptimo para comprimir una cadena dada. El algoritmo construye un árbol binario basado en las frecuencias de los caracteres de la cadena, y la palabra de código de cada carácter puede leerse siguiendo un camino desde la raíz hasta el nodo correspondiente. Un movimiento hacia la izquierda corresponde al bit 0, y un movimiento hacia la derecha corresponde al bit 1.

Inicialmente, cada carácter de la cadena está representado por un nodo cuyo peso es el número de veces que el carácter aparece en la cadena. Luego en cada

²D. A. Huffman descubrió este método al resolver una tarea de un curso universitario y publicó el algoritmo en 1952 [40].

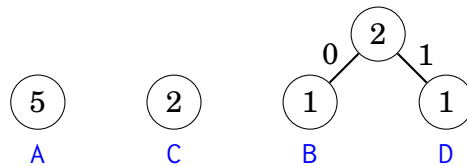
paso se combinan dos nodos con pesos mínimos creando un nuevo nodo cuyo peso es la suma de los pesos de los nodos originales. El proceso continúa hasta que todos los nodos han sido combinados.

A continuación veremos cómo la codificación de Huffman crea el código óptimo para la cadena AABACDACA. Inicialmente, hay cuatro nodos que corresponden a los caracteres de la cadena:

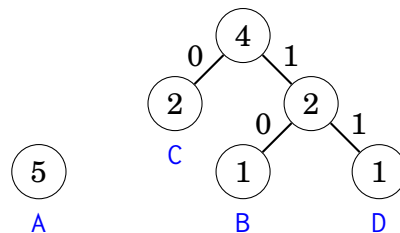


El nodo que representa el carácter A tiene peso 5 porque el carácter A aparece 5 veces en la cadena. Los otros pesos se han calculado de la misma manera.

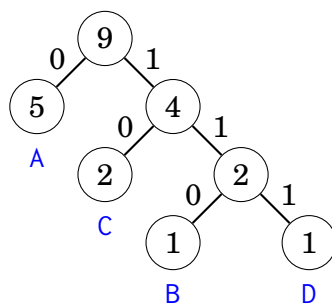
El primer paso es combinar los nodos que corresponden a los caracteres B y D, ambos con peso 1. El resultado es:



Después de esto, se combinan los nodos con peso 2:



Finalmente, se combinan los dos nodos restantes:



Ahora todos los nodos están en el árbol, así que el código está listo. Las siguientes palabras de código pueden leerse del árbol:

carácter	palabra de código
A	0
B	110
C	10
D	111

Capítulo 7

Programación dinámica

La **programación dinámica** es una técnica que combina la corrección de la búsqueda completa y la eficiencia de los algoritmos voraces. La programación dinámica puede aplicarse si el problema puede dividirse en subproblemas superpuestos que pueden resolverse de forma independiente.

Hay dos usos para la programación dinámica:

- **Encontrar una solución óptima:** Queremos encontrar una solución que sea lo más grande posible o lo más pequeña posible.
- **Contar el número de soluciones:** Queremos calcular el número total de soluciones posibles.

Primero veremos cómo puede usarse la programación dinámica para encontrar una solución óptima, y luego usaremos la misma idea para contar las soluciones.

Entender la programación dinámica es un hito en la carrera de todo programador competitivo. Aunque la idea básica es simple, el reto es cómo aplicar la programación dinámica a distintos problemas. Este capítulo presenta un conjunto de problemas clásicos que son un buen punto de partida.

7.1. Problema de las monedas

Primero nos centramos en un problema que ya hemos visto en el Capítulo 6: Dado un conjunto de valores de monedas $\text{coins} = \{c_1, c_2, \dots, c_k\}$ y una suma de dinero objetivo n , nuestra tarea es formar la suma n usando la menor cantidad de monedas posible.

En el Capítulo 6, resolvimos el problema usando un algoritmo voraz que siempre elige la moneda más grande posible. El algoritmo voraz funciona, por ejemplo, cuando las monedas son las monedas de euro, pero en el caso general el algoritmo voraz no necesariamente produce una solución óptima.

Ahora es momento de resolver el problema de forma eficiente usando programación dinámica, de modo que el algoritmo funcione para cualquier conjunto de monedas. El algoritmo de programación dinámica se basa en una función recursiva que recorre todas las posibilidades de cómo formar la suma, como un algoritmo de fuerza bruta. Sin embargo, el algoritmo de programación dinámica

es eficiente porque usa *memoización* y calcula la respuesta de cada subproblema solo una vez.

Formulación recursiva

La idea en la programación dinámica es formular el problema de forma recursiva de modo que la solución del problema pueda calcularse a partir de soluciones de subproblemas más pequeños. En el problema de las monedas, un problema recursivo natural es el siguiente: ¿cuál es el menor número de monedas requerido para formar una suma x ?

Sea $\text{solve}(x)$ el número mínimo de monedas requerido para una suma x . Los valores de la función dependen de los valores de las monedas. Por ejemplo, si $\text{coins} = \{1, 3, 4\}$, los primeros valores de la función son los siguientes:

$\text{solve}(0)$	$=$	0
$\text{solve}(1)$	$=$	1
$\text{solve}(2)$	$=$	2
$\text{solve}(3)$	$=$	1
$\text{solve}(4)$	$=$	1
$\text{solve}(5)$	$=$	2
$\text{solve}(6)$	$=$	2
$\text{solve}(7)$	$=$	2
$\text{solve}(8)$	$=$	2
$\text{solve}(9)$	$=$	3
$\text{solve}(10)$	$=$	3

Por ejemplo, $\text{solve}(10) = 3$, porque se necesitan al menos 3 monedas para formar la suma 10. La solución óptima es $3 + 3 + 4 = 10$.

La propiedad esencial de solve es que sus valores pueden calcularse recursivamente a partir de sus valores más pequeños. La idea es centrarse en la *primera* moneda que elegimos para la suma. Por ejemplo, en el escenario anterior, la primera moneda puede ser 1, 3 o 4. Si primero elegimos la moneda 1, la tarea restante es formar la suma 9 usando el número mínimo de monedas, que es un subproblema del problema original. Por supuesto, lo mismo aplica a las monedas 3 y 4. Así, podemos usar la siguiente fórmula recursiva para calcular el número mínimo de monedas:

$$\begin{aligned}\text{solve}(x) = \min(&\text{solve}(x - 1) + 1, \\ &\text{solve}(x - 3) + 1, \\ &\text{solve}(x - 4) + 1).\end{aligned}$$

El caso base de la recursión es $\text{solve}(0) = 0$, porque no se necesitan monedas para formar una suma vacía. Por ejemplo,

$$\text{solve}(10) = \text{solve}(7) + 1 = \text{solve}(4) + 2 = \text{solve}(0) + 3 = 3.$$

Ahora estamos listos para dar una función recursiva general que calcula el

número mínimo de monedas necesarias para formar una suma x :

$$\text{solve}(x) = \begin{cases} \infty & x < 0 \\ 0 & x = 0 \\ \min_{c \in \text{coins}} \text{solve}(x - c) + 1 & x > 0 \end{cases}$$

Primero, si $x < 0$, el valor es ∞ , porque es imposible formar una suma de dinero negativa. Luego, si $x = 0$, el valor es 0, porque no se necesitan monedas para formar una suma vacía. Finalmente, si $x > 0$, la variable c recorre todas las posibilidades de cómo elegir la primera moneda de la suma.

Una vez que se ha encontrado una función recursiva que resuelve el problema, podemos implementar directamente una solución en C++ (la constante INF denota infinito):

```
int solve(int x) {
    if (x < 0) return INF;
    if (x == 0) return 0;
    int best = INF;
    for (auto c : coins) {
        best = min(best, solve(x-c)+1);
    }
    return best;
}
```

Aun así, esta función no es eficiente, porque puede haber un número exponencial de formas de construir la suma. Sin embargo, a continuación veremos cómo hacer la función eficiente usando una técnica llamada memoización.

Usar memoización

La idea de la programación dinámica es usar **memoización** para calcular de forma eficiente los valores de una función recursiva. Esto significa que los valores de la función se almacenan en un arreglo después de calcularlos. Para cada parámetro, el valor de la función se calcula recursivamente solo una vez, y después de esto, el valor puede recuperarse directamente del arreglo.

En este problema, usamos los arreglos

```
bool ready[N];
int value[N];
```

donde `ready[x]` indica si el valor de `solve(x)` ha sido calculado, y si lo ha sido, `value[x]` contiene este valor. La constante N se ha elegido de modo que todos los valores requeridos quepan en los arreglos.

Ahora la función puede implementarse de forma eficiente de la siguiente manera:

```
int solve(int x) {
    if (x < 0) return INF;
```

```

    if (x == 0) return 0;
    if (ready[x]) return value[x];
    int best = INF;
    for (auto c : coins) {
        best = min(best, solve(x-c)+1);
    }
    value[x] = best;
    ready[x] = true;
    return best;
}

```

La función maneja los casos base $x < 0$ y $x = 0$ como antes. Luego la función comprueba desde `ready[x]` si `solve(x)` ya ha sido almacenado en `value[x]`, y si lo ha sido, la función lo devuelve directamente. En caso contrario la función calcula el valor de `solve(x)` recursivamente y lo almacena en `value[x]`.

Esta función funciona de forma eficiente, porque la respuesta para cada parámetro x se calcula recursivamente solo una vez. Después de que un valor de `solve(x)` se ha almacenado en `value[x]`, puede recuperarse de forma eficiente cada vez que la función sea llamada de nuevo con el parámetro x . La complejidad temporal del algoritmo es $O(nk)$, donde n es la suma objetivo y k es el número de monedas.

Nota que también podemos construir *iterativamente* el arreglo `value` usando un bucle que simplemente calcula todos los valores de `solve` para los parámetros $0 \dots n$:

```

value[0] = 0;
for (int x = 1; x <= n; x++) {
    value[x] = INF;
    for (auto c : coins) {
        if (x-c >= 0) {
            value[x] = min(value[x], value[x-c]+1);
        }
    }
}

```

De hecho, la mayoría de los programadores competitivos prefieren esta implementación, porque es más corta y tiene factores constantes más bajos. De ahora en adelante, también usamos implementaciones iterativas en nuestros ejemplos. Aun así, a menudo es más fácil pensar en soluciones de programación dinámica en términos de funciones recursivas.

Construir una solución

A veces se nos pide tanto encontrar el valor de una solución óptima como dar un ejemplo de cómo puede construirse tal solución. En el problema de las monedas, por ejemplo, podemos declarar otro arreglo que indique para cada suma de dinero la primera moneda de una solución óptima:


```
int first[N];
```

Luego, podemos modificar el algoritmo de la siguiente manera:

```
value[0] = 0;
for (int x = 1; x <= n; x++) {
    value[x] = INF;
    for (auto c : coins) {
        if (x-c >= 0 && value[x-c]+1 < value[x]) {
            value[x] = value[x-c]+1;
            first[x] = c;
        }
    }
}
```

Después de esto, el siguiente código puede usarse para imprimir las monedas que aparecen en una solución óptima para la suma n :

```
while (n > 0) {
    cout << first[n] << "\n";
    n -= first[n];
}
```

Contar el número de soluciones

Consideremos ahora otra versión del problema de las monedas donde nuestra tarea es calcular el número total de formas de producir una suma x usando las monedas. Por ejemplo, si $\text{coins} = \{1, 3, 4\}$ y $x = 5$, hay un total de 6 formas:

- | | |
|-------------|---------|
| ■ 1+1+1+1+1 | ■ 3+1+1 |
| ■ 1+1+3 | ■ 1+4 |
| ■ 1+3+1 | ■ 4+1 |

De nuevo, podemos resolver el problema recursivamente. Sea $\text{solve}(x)$ el número de formas en que podemos formar la suma x . Por ejemplo, si $\text{coins} = \{1, 3, 4\}$, entonces $\text{solve}(5) = 6$ y la fórmula recursiva es

$$\begin{aligned}\text{solve}(x) = & \text{solve}(x-1) + \\ & \text{solve}(x-3) + \\ & \text{solve}(x-4).\end{aligned}$$

Luego, la función recursiva general es la siguiente:

$$\text{solve}(x) = \begin{cases} 0 & x < 0 \\ 1 & x = 0 \\ \sum_{c \in \text{coins}} \text{solve}(x-c) & x > 0 \end{cases}$$

Si $x < 0$, el valor es 0, porque no hay soluciones. Si $x = 0$, el valor es 1, porque solo hay una forma de formar una suma vacía. En caso contrario calculamos la suma de todos los valores de la forma $\text{solve}(x - c)$ donde c está en `coins`.

El siguiente código construye un arreglo `count` tal que `count[x]` es igual al valor de $\text{solve}(x)$ para $0 \leq x \leq n$:

```
count[0] = 1;
for (int x = 1; x <= n; x++) {
    for (auto c : coins) {
        if (x - c >= 0) {
            count[x] += count[x - c];
        }
    }
}
```

A menudo el número de soluciones es tan grande que no se requiere calcular el número exacto sino que basta con dar la respuesta módulo m donde, por ejemplo, $m = 10^9 + 7$. Esto puede hacerse cambiando el código de modo que todos los cálculos se hagan módulo m . En el código anterior, basta con agregar la línea

```
count[x] %= m;
```

después de la línea

```
count[x] += count[x - c];
```

Ahora hemos discutido todas las ideas básicas de la programación dinámica. Como la programación dinámica puede usarse en muchas situaciones distintas, ahora recorreremos un conjunto de problemas que muestran más ejemplos sobre las posibilidades de la programación dinámica.

7.2. Subsecuencia creciente más larga

Nuestro primer problema es encontrar la **subsecuencia creciente más larga** en un arreglo de n elementos. Esta es una secuencia de longitud máxima de elementos del arreglo que va de izquierda a derecha, y cada elemento de la secuencia es mayor que el elemento anterior. Por ejemplo, en el arreglo

0	1	2	3	4	5	6	7
6	2	5	1	7	4	8	3

la subsecuencia creciente más larga contiene 4 elementos:

0	1	2	3	4	5	6	7
6	2	5	1	7	4	8	3



Sea $\text{length}(k)$ la longitud de la subsecuencia creciente más larga que termina en la posición k . Así, si calculamos todos los valores de $\text{length}(k)$ donde $0 \leq k \leq n - 1$, averiguaremos la longitud de la subsecuencia creciente más larga. Por ejemplo, los valores de la función para el arreglo anterior son los siguientes:

```
length(0) = 1
length(1) = 1
length(2) = 2
length(3) = 1
length(4) = 3
length(5) = 2
length(6) = 4
length(7) = 2
```

Por ejemplo, $\text{length}(6) = 4$, porque la subsecuencia creciente más larga que termina en la posición 6 consta de 4 elementos.

Para calcular un valor de $\text{length}(k)$, debemos encontrar una posición $i < k$ para la cual $\text{array}[i] < \text{array}[k]$ y $\text{length}(i)$ sea lo más grande posible. Entonces sabemos que $\text{length}(k) = \text{length}(i) + 1$, porque esta es una forma óptima de agregar $\text{array}[k]$ a una subsecuencia. Sin embargo, si no hay tal posición i , entonces $\text{length}(k) = 1$, lo que significa que la subsecuencia solo contiene $\text{array}[k]$.

Como todos los valores de la función pueden calcularse a partir de sus valores más pequeños, podemos usar programación dinámica. En el siguiente código, los valores de la función se almacenarán en un arreglo `length`.

```
for (int k = 0; k < n; k++) {
    length[k] = 1;
    for (int i = 0; i < k; i++) {
        if (array[i] < array[k]) {
            length[k] = max(length[k], length[i] + 1);
        }
    }
}
```

Este código funciona en tiempo $O(n^2)$, porque consta de dos bucles anidados. Sin embargo, también es posible implementar el cálculo de programación dinámica de forma más eficiente en tiempo $O(n \log n)$. ¿Puedes encontrar una forma de hacer esto?

7.3. Caminos en una cuadrícula

Nuestro siguiente problema es encontrar un camino desde la esquina superior izquierda hasta la esquina inferior derecha de una cuadrícula de $n \times n$, tal que solo nos movamos hacia abajo y hacia la derecha. Cada casilla contiene un entero positivo, y el camino debe construirse de modo que la suma de los valores a lo largo del camino sea lo más grande posible.

La siguiente figura muestra un camino óptimo en una cuadrícula:

3	7	9	2	7
9	8	3	5	5
1	7	9	8	5
3	8	6	4	10
6	3	9	7	8

La suma de los valores en el camino es 67, y esta es la mayor suma posible en un camino desde la esquina superior izquierda hasta la esquina inferior derecha.

Supongamos que las filas y columnas de la cuadrícula se numeran de 1 a n , y $\text{value}[y][x]$ es igual al valor de la casilla (y, x) . Sea $\text{sum}(y, x)$ la suma máxima en un camino desde la esquina superior izquierda hasta la casilla (y, x) . Ahora $\text{sum}(n, n)$ nos dice la suma máxima desde la esquina superior izquierda hasta la esquina inferior derecha. Por ejemplo, en la cuadrícula anterior, $\text{sum}(5, 5) = 67$.

Podemos calcular recursivamente las sumas de la siguiente manera:

$$\text{sum}(y, x) = \max(\text{sum}(y, x - 1), \text{sum}(y - 1, x)) + \text{value}[y][x]$$

La fórmula recursiva se basa en la observación de que un camino que termina en la casilla (y, x) puede venir o bien de la casilla $(y, x - 1)$ o de la casilla $(y - 1, x)$:

			↓	
		→		

Así, seleccionamos la dirección que maximiza la suma. Suponemos que $\text{sum}(y, x) = 0$ si $y = 0$ o $x = 0$ (porque no existen tales caminos), así que la fórmula recursiva también funciona cuando $y = 1$ o $x = 1$.

Como la función sum tiene dos parámetros, el arreglo de programación dinámica también tiene dos dimensiones. Por ejemplo, podemos usar un arreglo

```
int sum[N][N];
```

y calcular las sumas de la siguiente manera:

```
for (int y = 1; y <= n; y++) {
    for (int x = 1; x <= n; x++) {
        sum[y][x] = max(sum[y][x-1], sum[y-1][x]) + value[y][x];
    }
}
```

La complejidad temporal del algoritmo es $O(n^2)$.

7.4. Problemas de la mochila

El término **mochila** (knapsack) se refiere a problemas donde se da un conjunto de objetos, y hay que encontrar subconjuntos con algunas propiedades. Los problemas de la mochila a menudo pueden resolverse usando programación dinámica.

En esta sección, nos centramos en el siguiente problema: Dada una lista de pesos $[w_1, w_2, \dots, w_n]$, determinar todas las sumas que pueden construirse usando los pesos. Por ejemplo, si los pesos son $[1, 3, 3, 5]$, las siguientes sumas son posibles:

0	1	2	3	4	5	6	7	8	9	10	11	12
X	X		X	X	X	X	X	X	X		X	X

En este caso, todas las sumas entre $0 \dots 12$ son posibles, excepto 2 y 10. Por ejemplo, la suma 7 es posible porque podemos seleccionar los pesos $[1, 3, 3]$.

Para resolver el problema, nos centramos en subproblemas donde solo usamos los primeros k pesos para construir sumas. Sea $\text{possible}(x, k) = \text{true}$ si podemos construir una suma x usando los primeros k pesos, y en caso contrario $\text{possible}(x, k) = \text{false}$. Los valores de la función pueden calcularse recursivamente de la siguiente manera:

$$\text{possible}(x, k) = \text{possible}(x - w_k, k - 1) \vee \text{possible}(x, k - 1)$$

La fórmula se basa en el hecho de que podemos usar o no usar el peso w_k en la suma. Si usamos w_k , la tarea restante es formar la suma $x - w_k$ usando los primeros $k - 1$ pesos, y si no usamos w_k , la tarea restante es formar la suma x usando los primeros $k - 1$ pesos. Como casos base,

$$\text{possible}(x, 0) = \begin{cases} \text{true} & x = 0 \\ \text{false} & x \neq 0 \end{cases}$$

porque si no se usan pesos, solo podemos formar la suma 0.

La siguiente tabla muestra todos los valores de la función para los pesos $[1, 3, 3, 5]$ (el símbolo "X" indica los valores verdaderos):

$k \backslash x$	0	1	2	3	4	5	6	7	8	9	10	11	12
0	X												
1	X	X											
2	X	X		X	X								
3	X	X		X	X		X	X					
4	X	X		X	X	X	X	X	X	X		X	X

Después de calcular esos valores, $\text{possible}(x, n)$ nos dice si podemos construir una suma x usando *todos* los pesos.

Sea W la suma total de los pesos. La siguiente solución de programación dinámica de tiempo $O(nW)$ corresponde a la función recursiva:

```
possible[0][0] = true;
for (int k = 1; k <= n; k++) {
    for (int x = 0; x <= W; x++) {
        if (x-w[k] >= 0) possible[x][k] |= possible[x-w[k]][k-1];
        possible[x][k] |= possible[x][k-1];
    }
}
```

Sin embargo, aquí hay una mejor implementación que solo usa un arreglo unidimensional `possible[x]` que indica si podemos construir un subconjunto con suma x . El truco es actualizar el arreglo de derecha a izquierda para cada nuevo peso:

```
possible[0] = true;
for (int k = 1; k <= n; k++) {
    for (int x = W; x >= 0; x--) {
        if (possible[x]) possible[x+w[k]] = true;
    }
}
```

Nota que la idea general presentada aquí puede usarse en muchos problemas de la mochila. Por ejemplo, si se nos dan objetos con pesos y valores, podemos determinar para cada suma de pesos la suma máxima de valores de un subconjunto.

7.5. Distancia de edición

La **distancia de edición** o **distancia de Levenshtein**¹ es el número mínimo de operaciones de edición necesarias para transformar una cadena en otra cadena. Las operaciones de edición permitidas son las siguientes:

- insertar un carácter (p. ej. $ABC \rightarrow ABCA$)
- eliminar un carácter (p. ej. $ABC \rightarrow AC$)
- modificar un carácter (p. ej. $ABC \rightarrow ADC$)

Por ejemplo, la distancia de edición entre LOVE y MOVIE es 2, porque podemos primero realizar la operación $LOVE \rightarrow MOVE$ (modificar) y luego la operación $MOVE \rightarrow MOVIE$ (insertar). Este es el menor número posible de operaciones, porque está claro que solo una operación no es suficiente.

Supongamos que se nos da una cadena x de longitud n y una cadena y de longitud m , y queremos calcular la distancia de edición entre x e y . Para resolver el problema, definimos una función $distance(a, b)$ que da la distancia de edición

¹La distancia lleva el nombre de V. I. Levenshtein, quien la estudió en relación con los códigos binarios [49].

entre los prefijos $x[0 \dots a]$ e $y[0 \dots b]$. Así, usando esta función, la distancia de edición entre x e y es igual a $\text{distance}(n-1, m-1)$.

Podemos calcular los valores de distance de la siguiente manera:

$$\begin{aligned} \text{distance}(a, b) = \min(&\text{distance}(a, b-1) + 1, \\ &\text{distance}(a-1, b) + 1, \\ &\text{distance}(a-1, b-1) + \text{cost}(a, b)). \end{aligned}$$

Aquí $\text{cost}(a, b) = 0$ si $x[a] = y[b]$, y en caso contrario $\text{cost}(a, b) = 1$. La fórmula considera las siguientes formas de editar la cadena x :

- $\text{distance}(a, b-1)$: insertar un carácter al final de x
- $\text{distance}(a-1, b)$: eliminar el último carácter de x
- $\text{distance}(a-1, b-1)$: coincidir o modificar el último carácter de x

En los dos primeros casos, se necesita una operación de edición (insertar o eliminar). En el último caso, si $x[a] = y[b]$, podemos hacer coincidir los últimos caracteres sin editar, y en caso contrario se necesita una operación de edición (modificar).

La siguiente tabla muestra los valores de distance en el caso del ejemplo:

		M	O	V	I	E
L O V E	0	1	2	3	4	5
	1	1	2	3	4	5
	2	2	1	2	3	4
	3	3	2	1	2	3
	4	4	3	2	2	2

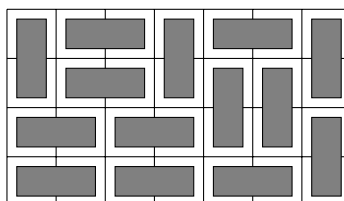
La esquina inferior derecha de la tabla nos dice que la distancia de edición entre LOVE y MOVIE es 2. La tabla también muestra cómo construir la secuencia más corta de operaciones de edición. En este caso el camino es el siguiente:

		M	O	V	I	E
	0	1	2	3	4	5
L	1	1	2	3	4	5
O	2	2	1	2	3	4
V	3	3	2	1	2	3
E	4	4	3	2	2	2

Los últimos caracteres de LOVE y MOVIE son iguales, así que la distancia de edición entre ellos es igual a la distancia de edición entre LOV y MOVI. Podemos usar una operación de edición para eliminar el carácter I de MOVI. Así, la distancia de edición es uno mayor que la distancia de edición entre LOV y MOV, etc.

7.6. Contar teselados

A veces los estados de una solución de programación dinámica son más complejos que combinaciones fijas de números. Como ejemplo, considera el problema de calcular el número de formas distintas de llenar una cuadrícula de $n \times m$ usando fichas de tamaño 1×2 y 2×1 . Por ejemplo, una solución válida para la cuadrícula de 4×7 es



y el número total de soluciones es 781.

El problema puede resolverse usando programación dinámica recorriendo la cuadrícula fila por fila. Cada fila de una solución puede representarse como una cadena que contiene m caracteres del conjunto $\{\sqcap, \sqcup, \sqsubset, \sqsupset\}$. Por ejemplo, la solución anterior consta de cuatro filas que corresponden a las siguientes cadenas:

- $\sqcap \sqsubset \sqcap \sqsubset \sqcap \sqcap$
- $\sqcup \sqsubset \sqcup \sqcap \sqcap \sqcup$
- $\sqsubset \sqsubset \sqsubset \sqcup \sqcup \sqcap$
- $\sqsubset \sqsubset \sqsubset \sqsubset \sqsubset \sqcup$

Sea $\text{count}(k, x)$ el número de formas de construir una solución para las filas $1 \dots k$ de la cuadrícula tal que la cadena x corresponde a la fila k . Es posible usar programación dinámica aquí, porque el estado de una fila está restringido solo por el estado de la fila anterior.

Una solución es válida si la fila 1 no contiene el carácter $\sqcup$, la fila n no contiene el carácter $\sqcap$, y todas las filas consecutivas son *compatibles*. Por ejemplo, las filas $\sqcup \sqsubset \sqcup \sqcap \sqcap \sqcup$ y $\sqsubset \sqsubset \sqsubset \sqcup \sqcup \sqcap$ son compatibles, mientras que las filas $\sqcap \sqsubset \sqcap \sqsubset \sqcap \sqcap$ y $\sqsubset \sqsubset \sqsubset \sqsubset \sqsubset \sqcup$ no son compatibles.

Como una fila consta de m caracteres y hay cuatro opciones para cada carácter, el número de filas distintas es a lo sumo 4^m . Así, la complejidad temporal de la solución es $O(n4^{2m})$ porque podemos recorrer los $O(4^m)$ estados posibles para cada fila, y para cada estado, hay $O(4^m)$ estados posibles para la fila anterior. En la práctica, es buena idea rotar la cuadrícula de modo que el lado más corto tenga longitud m , porque el factor 4^{2m} domina la complejidad temporal.

Es posible hacer la solución más eficiente usando una representación más compacta para las filas. Resulta que basta con saber qué columnas de la fila anterior contienen la casilla superior de una ficha vertical. Así, podemos representar una fila usando solo los caracteres $\sqcap$ y $\square$, donde $\square$ es una combinación de los caracteres $\sqcup$, $\sqsubset$ y $\sqsupset$. Usando esta representación, solo hay 2^m filas distintas y la complejidad temporal es $O(n2^{2m})$.

Como nota final, también hay una sorprendente fórmula directa para calcular el número de teselados²:

$$\prod_{a=1}^{\lceil n/2 \rceil} \prod_{b=1}^{\lceil m/2 \rceil} 4 \cdot \left(\cos^2 \frac{\pi a}{n+1} + \cos^2 \frac{\pi b}{m+1} \right)$$

Esta fórmula es muy eficiente, porque calcula el número de teselados en tiempo $O(nm)$, pero como la respuesta es un producto de números reales, un problema al usar la fórmula es cómo almacenar los resultados intermedios con precisión.

²Sorprendentemente, esta fórmula fue descubierta en 1961 por dos equipos de investigación [43, 67] que trabajaron de forma independiente.

Capítulo 8

Análisis amortizado

La complejidad temporal de un algoritmo suele ser fácil de analizar simplemente examinando la estructura del algoritmo: qué bucles contiene el algoritmo y cuántas veces se ejecutan los bucles. Sin embargo, a veces un análisis directo no da una imagen real de la eficiencia del algoritmo.

El **análisis amortizado** puede usarse para analizar algoritmos que contienen operaciones cuya complejidad temporal varía. La idea es estimar el tiempo total usado en todas esas operaciones durante la ejecución del algoritmo, en lugar de centrarse en operaciones individuales.

8.1. Método de los dos punteros

En el **método de los dos punteros**, se usan dos punteros para recorrer los valores del arreglo. Ambos punteros pueden moverse en una sola dirección, lo que asegura que el algoritmo funcione de forma eficiente. A continuación discutimos dos problemas que pueden resolverse usando el método de los dos punteros.

Suma de subarreglo

Como primer ejemplo, considera un problema donde se nos da un arreglo de n enteros positivos y una suma objetivo x , y queremos encontrar un subarreglo cuya suma sea x o reportar que no existe tal subarreglo.

Por ejemplo, el arreglo

1	3	2	5	1	1	2	3
---	---	---	---	---	---	---	---

contiene un subarreglo cuya suma es 8:

1	3	2	5	1	1	2	3
---	---	---	---	---	---	---	---

Este problema puede resolverse en tiempo $O(n)$ usando el método de los dos punteros. La idea es mantener punteros que apunten al primer y último valor de un subarreglo. En cada turno, el puntero izquierdo se mueve un paso a la derecha, y el puntero derecho se mueve a la derecha mientras la suma del subarreglo

resultante sea a lo sumo x . Si la suma se vuelve exactamente x , se ha encontrado una solución.

Como ejemplo, considera el siguiente arreglo y una suma objetivo $x = 8$:

1	3	2	5	1	1	2	3
---	---	---	---	---	---	---	---

El subarreglo inicial contiene los valores 1, 3 y 2 cuya suma es 6:

1	3	2	5	1	1	2	3
---	---	---	---	---	---	---	---

Luego, el puntero izquierdo se mueve un paso a la derecha. El puntero derecho no se mueve, porque de lo contrario la suma del subarreglo excedería x .

1	3	2	5	1	1	2	3
---	---	---	---	---	---	---	---

De nuevo, el puntero izquierdo se mueve un paso a la derecha, y esta vez el puntero derecho se mueve tres pasos a la derecha. La suma del subarreglo es $2 + 5 + 1 = 8$, así que se ha encontrado un subarreglo cuya suma es x .

1	3	2	5	1	1	2	3
---	---	---	---	---	---	---	---

El tiempo de ejecución del algoritmo depende del número de pasos que se mueve el puntero derecho. Aunque no hay una cota superior útil sobre cuántos pasos puede moverse el puntero en un *solo* turno, sabemos que el puntero se mueve *un total de* $O(n)$ pasos durante el algoritmo, porque solo se mueve a la derecha.

Como tanto el puntero izquierdo como el derecho se mueven $O(n)$ pasos durante el algoritmo, el algoritmo funciona en tiempo $O(n)$.

Problema 2SUM

Otro problema que puede resolverse usando el método de los dos punteros es el siguiente problema, también conocido como el **problema 2SUM**: dado un arreglo de n números y una suma objetivo x , encontrar dos valores del arreglo tales que su suma sea x , o reportar que no existen tales valores.

Para resolver el problema, primero ordenamos los valores del arreglo en orden creciente. Después de eso, recorremos el arreglo usando dos punteros. El puntero izquierdo comienza en el primer valor y se mueve un paso a la derecha en cada turno. El puntero derecho comienza en el último valor y siempre se mueve a la izquierda hasta que la suma del valor izquierdo y derecho sea a lo sumo x . Si la suma es exactamente x , se ha encontrado una solución.

Por ejemplo, considera el siguiente arreglo y una suma objetivo $x = 12$:

1	4	5	6	7	9	9	10
---	---	---	---	---	---	---	----

Las posiciones iniciales de los punteros son las siguientes. La suma de los valores es $1 + 10 = 11$ que es menor que x .

1	4	5	6	7	9	9	10
↑							↑

Luego el puntero izquierdo se mueve un paso a la derecha. El puntero derecho se mueve tres pasos a la izquierda, y la suma se vuelve $4 + 7 = 11$.

1	4	5	6	7	9	9	10
	↑			↑			

Después de esto, el puntero izquierdo se mueve un paso a la derecha de nuevo. El puntero derecho no se mueve, y se ha encontrado una solución $5 + 7 = 12$.

1	4	5	6	7	9	9	10
		↑		↑			

El tiempo de ejecución del algoritmo es $O(n \log n)$, porque primero ordena el arreglo en tiempo $O(n \log n)$, y luego ambos punteros se mueven $O(n)$ pasos.

Nota que es posible resolver el problema de otra forma en tiempo $O(n \log n)$ usando búsqueda binaria. En tal solución, recorremos el arreglo y para cada valor del arreglo, intentamos encontrar otro valor que produzca la suma x . Esto puede hacerse realizando n búsquedas binarias, cada una de las cuales toma $O(\log n)$ de tiempo.

Un problema más difícil es el **problema 3SUM** que pide encontrar *tres* valores del arreglo cuya suma sea x . Usando la idea del algoritmo anterior, este problema puede resolverse en tiempo $O(n^2)$ ¹. ¿Puedes ver cómo?

8.2. Elementos menores más cercanos

El análisis amortizado se usa a menudo para estimar el número de operaciones realizadas sobre una estructura de datos. Las operaciones pueden estar distribuidas de forma desigual de modo que la mayoría de las operaciones ocurran durante una cierta fase del algoritmo, pero el número total de las operaciones sea limitado.

Como ejemplo, considera el problema de encontrar para cada elemento del arreglo el **elemento menor más cercano**, es decir, el primer elemento menor que precede al elemento en el arreglo. Es posible que no exista tal elemento, en cuyo caso el algoritmo debería reportar esto. A continuación veremos cómo el problema puede resolverse de forma eficiente usando una estructura de pila.

¹Durante mucho tiempo, se pensó que resolver el problema 3SUM de forma más eficiente que en tiempo $O(n^2)$ no sería posible. Sin embargo, en 2014, resultó [30] que no es así.

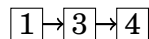
Recorremos el arreglo de izquierda a derecha y mantenemos una pila de elementos del arreglo. En cada posición del arreglo, eliminamos elementos de la pila hasta que el elemento de la cima sea menor que el elemento actual, o la pila esté vacía. Luego, reportamos que el elemento de la cima es el elemento menor más cercano del elemento actual, o si la pila está vacía, no hay tal elemento. Finalmente, agregamos el elemento actual a la pila.

Como ejemplo, considera el siguiente arreglo:

1	3	4	2	5	3	4	2
---	---	---	---	---	---	---	---

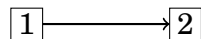
Primero, los elementos 1, 3 y 4 se agregan a la pila, porque cada elemento es mayor que el elemento anterior. Así, el elemento menor más cercano de 4 es 3, y el elemento menor más cercano de 3 es 1.

1	3	4	2	5	3	4	2
---	---	---	---	---	---	---	---



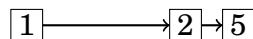
El siguiente elemento 2 es menor que los dos elementos de la cima en la pila. Así, los elementos 3 y 4 se eliminan de la pila, y luego el elemento 2 se agrega a la pila. Su elemento menor más cercano es 1:

1	3	4	2	5	3	4	2
---	---	---	---	---	---	---	---



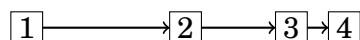
Luego, el elemento 5 es mayor que el elemento 2, así que se agregará a la pila, y su elemento menor más cercano es 2:

1	3	4	2	5	3	4	2
---	---	---	---	---	---	---	---



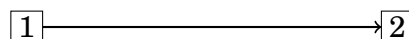
Después de esto, el elemento 5 se elimina de la pila y los elementos 3 y 4 se agregan a la pila:

1	3	4	2	5	3	4	2
---	---	---	---	---	---	---	---



Finalmente, todos los elementos excepto 1 se eliminan de la pila y el último elemento 2 se agrega a la pila:

1	3	4	2	5	3	4	2
---	---	---	---	---	---	---	---



La eficiencia del algoritmo depende del número total de operaciones sobre la pila. Si el elemento actual es mayor que el elemento de la cima en la pila, se agrega directamente a la pila, lo cual es eficiente. Sin embargo, a veces la pila puede contener varios elementos mayores y toma tiempo eliminarlos. Aun así, cada elemento se agrega *exactamente una vez* a la pila y se elimina *a lo sumo una vez* de la pila. Así, cada elemento provoca $O(1)$ operaciones sobre la pila, y el algoritmo funciona en tiempo $O(n)$.

8.3. Mínimo de ventana deslizante

Una **ventana deslizante** es un subarreglo de tamaño constante que se mueve de izquierda a derecha a través del arreglo. En cada posición de la ventana, queremos calcular alguna información sobre los elementos dentro de la ventana. En esta sección, nos centramos en el problema de mantener el **mínimo de ventana deslizante**, lo que significa que debemos reportar el valor más pequeño dentro de cada ventana.

El mínimo de ventana deslizante puede calcularse usando una idea similar a la que usamos para calcular los elementos menores más cercanos. Mantenemos una cola donde cada elemento es mayor que el elemento anterior, y el primer elemento siempre corresponde al elemento mínimo dentro de la ventana. Después de cada movimiento de la ventana, eliminamos elementos del final de la cola hasta que el último elemento de la cola sea menor que el nuevo elemento de la ventana, o la cola quede vacía. También eliminamos el primer elemento de la cola si ya no está dentro de la ventana. Finalmente, agregamos el nuevo elemento de la ventana al final de la cola.

Como ejemplo, considera el siguiente arreglo:

2	1	4	5	3	4	1	2
---	---	---	---	---	---	---	---

Supongamos que el tamaño de la ventana deslizante es 4. En la primera posición de la ventana, el valor más pequeño es 1:

2	1	4	5	3	4	1	2
---	---	---	---	---	---	---	---

1	→	4	→	5
---	---	---	---	---

Luego la ventana se mueve un paso a la derecha. El nuevo elemento 3 es menor que los elementos 4 y 5 en la cola, así que los elementos 4 y 5 se eliminan de la cola y el elemento 3 se agrega a la cola. El valor más pequeño sigue siendo 1.

2	1	4	5	3	4	1	2
---	---	---	---	---	---	---	---

1	→	3
---	---	---

Después de esto, la ventana se mueve de nuevo, y el elemento más pequeño 1 ya no pertenece a la ventana. Así, se elimina de la cola y el valor más pequeño ahora es 3. También el nuevo elemento 4 se agrega a la cola.

2	1	4	5	3	4	1	2
---	---	---	---	---	---	---	---

3	→	4
---	---	---

El siguiente nuevo elemento 1 es menor que todos los elementos de la cola. Así, todos los elementos se eliminan de la cola y esta solo contendrá el elemento 1:

2	1	4	5	3	4	1	2
---	---	---	---	---	---	---	---

1

Finalmente la ventana llega a su última posición. El elemento 2 se agrega a la cola, pero el valor más pequeño dentro de la ventana sigue siendo 1.

2	1	4	5	3	4	1	2
---	---	---	---	---	---	---	---

1 → 2

Como cada elemento del arreglo se agrega a la cola exactamente una vez y se elimina de la cola a lo sumo una vez, el algoritmo funciona en tiempo $O(n)$.

Capítulo 9

Consultas de rango

En este capítulo, discutimos estructuras de datos que nos permiten procesar de forma eficiente consultas de rango. En una **consulta de rango**, nuestra tarea es calcular un valor basado en un subarreglo de un arreglo. Las consultas de rango típicas son:

- $\text{sum}_q(a, b)$: calcular la suma de los valores en el rango $[a, b]$
- $\text{min}_q(a, b)$: encontrar el valor mínimo en el rango $[a, b]$
- $\text{max}_q(a, b)$: encontrar el valor máximo en el rango $[a, b]$

Por ejemplo, considera el rango $[3, 6]$ en el siguiente arreglo:

0	1	2	3	4	5	6	7
1	3	8	4	6	1	3	4

En este caso, $\text{sum}_q(3, 6) = 14$, $\text{min}_q(3, 6) = 1$ y $\text{max}_q(3, 6) = 6$.

Una forma simple de procesar consultas de rango es usar un bucle que recorra todos los valores del arreglo en el rango. Por ejemplo, la siguiente función puede usarse para procesar consultas de suma en un arreglo:

```
int sum(int a, int b) {
    int s = 0;
    for (int i = a; i <= b; i++) {
        s += array[i];
    }
    return s;
}
```

Esta función funciona en tiempo $O(n)$, donde n es el tamaño del arreglo. Así, podemos procesar q consultas en tiempo $O(nq)$ usando la función. Sin embargo, si tanto n como q son grandes, este enfoque es lento. Afortunadamente, resulta que hay formas de procesar consultas de rango mucho más eficientemente.

9.1. Consultas en un arreglo estático

Primero nos centramos en una situación donde el arreglo es *estático*, es decir, los valores del arreglo nunca se actualizan entre las consultas. En este caso, basta con construir una estructura de datos estática que nos diga la respuesta para cualquier consulta posible.

Consultas de suma

Podemos procesar fácilmente consultas de suma en un arreglo estático construyendo un **arreglo de sumas de prefijos**. Cada valor del arreglo de sumas de prefijos es igual a la suma de los valores del arreglo original hasta esa posición, es decir, el valor en la posición k es $\text{sum}_q(0, k)$. El arreglo de sumas de prefijos puede construirse en tiempo $O(n)$.

Por ejemplo, considera el siguiente arreglo:

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

El arreglo de sumas de prefijos correspondiente es el siguiente:

0	1	2	3	4	5	6	7
1	4	8	16	22	23	27	29

Como el arreglo de sumas de prefijos contiene todos los valores de $\text{sum}_q(0, k)$, podemos calcular cualquier valor de $\text{sum}_q(a, b)$ en tiempo $O(1)$ de la siguiente manera:

$$\text{sum}_q(a, b) = \text{sum}_q(0, b) - \text{sum}_q(0, a - 1)$$

Definiendo $\text{sum}_q(0, -1) = 0$, la fórmula anterior también se cumple cuando $a = 0$.

Por ejemplo, considera el rango $[3, 6]$:

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

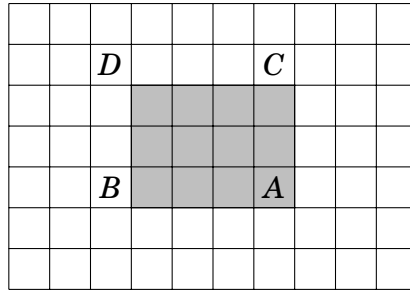
En este caso $\text{sum}_q(3, 6) = 8 + 6 + 1 + 4 = 19$. Esta suma puede calcularse a partir de dos valores del arreglo de sumas de prefijos:

0	1	2	3	4	5	6	7
1	4	8	16	22	23	27	29

Así, $\text{sum}_q(3, 6) = \text{sum}_q(0, 6) - \text{sum}_q(0, 2) = 27 - 8 = 19$.

También es posible generalizar esta idea a dimensiones más altas. Por ejemplo, podemos construir un arreglo bidimensional de sumas de prefijos que puede usarse para calcular la suma de cualquier subarreglo rectangular en tiempo $O(1)$. Cada suma en tal arreglo corresponde a un subarreglo que comienza en la esquina superior izquierda del arreglo.

La siguiente figura ilustra la idea:



La suma del subarreglo gris puede calcularse usando la fórmula

$$S(A) - S(B) - S(C) + S(D),$$

donde $S(X)$ denota la suma de los valores en un subarreglo rectangular desde la esquina superior izquierda hasta la posición de X .

Consultas de mínimo

Las consultas de mínimo son más difíciles de procesar que las consultas de suma. Aun así, hay un método de preprocesamiento bastante simple de tiempo $O(n \log n)$ después del cual podemos responder cualquier consulta de mínimo en tiempo $O(1)$ ¹. Nota que como las consultas de mínimo y máximo pueden procesarse de forma similar, podemos centrarnos en las consultas de mínimo.

La idea es precalcular todos los valores de $\min_q(a, b)$ donde $b - a + 1$ (la longitud del rango) es una potencia de dos. Por ejemplo, para el arreglo

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

se calculan los siguientes valores:

a	b	$\min_q(a, b)$	a	b	$\min_q(a, b)$	a	b	$\min_q(a, b)$
0	0	1	0	1	1	0	3	1
1	1	3	1	2	3	1	4	3
2	2	4	2	3	4	2	5	1
3	3	8	3	4	6	3	6	1
4	4	6	4	5	1	4	7	1
5	5	1	5	6	1	0	7	1
6	6	4	6	7	2			
7	7	2						

¹Esta técnica se presentó en [7] y a veces se llama el método de la **tabla dispersa** (sparse table). También hay técnicas más sofisticadas [22] donde el tiempo de preprocesamiento es solo $O(n)$, pero tales algoritmos no se necesitan en la programación competitiva.

El número de valores precalculados es $O(n \log n)$, porque hay $O(\log n)$ longitudes de rango que son potencias de dos. Los valores pueden calcularse de forma eficiente usando la fórmula recursiva

$$\min_q(a, b) = \min(\min_q(a, a + w - 1), \min_q(a + w, b)),$$

donde $b - a + 1$ es una potencia de dos y $w = (b - a + 1)/2$. Calcular todos esos valores toma $O(n \log n)$ de tiempo.

Después de esto, cualquier valor de $\min_q(a, b)$ puede calcularse en tiempo $O(1)$ como el mínimo de dos valores precalculados. Sea k la mayor potencia de dos que no excede $b - a + 1$. Podemos calcular el valor de $\min_q(a, b)$ usando la fórmula

$$\min_q(a, b) = \min(\min_q(a, a + k - 1), \min_q(b - k + 1, b)).$$

En la fórmula anterior, el rango $[a, b]$ se representa como la unión de los rangos $[a, a + k - 1]$ y $[b - k + 1, b]$, ambos de longitud k .

Como ejemplo, considera el rango $[1, 6]$:

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

La longitud del rango es 6, y la mayor potencia de dos que no excede 6 es 4. Así, el rango $[1, 6]$ es la unión de los rangos $[1, 4]$ y $[3, 6]$:

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

Como $\min_q(1, 4) = 3$ y $\min_q(3, 6) = 1$, concluimos que $\min_q(1, 6) = 1$.

9.2. Árbol indexado binario

Un **árbol indexado binario** o **árbol de Fenwick**² puede verse como una variante dinámica de un arreglo de sumas de prefijos. Admite dos operaciones de tiempo $O(\log n)$ sobre un arreglo: procesar una consulta de suma de rango y actualizar un valor.

La ventaja de un árbol indexado binario es que nos permite actualizar de forma eficiente los valores del arreglo entre consultas de suma. Esto no sería posible usando un arreglo de sumas de prefijos, porque después de cada actualización, sería necesario construir de nuevo todo el arreglo de sumas de prefijos en tiempo $O(n)$.

²La estructura del árbol indexado binario fue presentada por P. M. Fenwick en 1994 [21].

Estructura

Aunque el nombre de la estructura es *árbol* indexado binario, normalmente se representa como un arreglo. En esta sección suponemos que todos los arreglos están indexados desde 1, porque hace la implementación más fácil.

Sea $p(k)$ la mayor potencia de dos que divide a k . Almacenamos un árbol indexado binario como un arreglo *tree* tal que

$$\text{tree}[k] = \text{sum}_q(k - p(k) + 1, k),$$

es decir, cada posición k contiene la suma de los valores en un rango del arreglo original cuya longitud es $p(k)$ y que termina en la posición k . Por ejemplo, como $p(6) = 2$, $\text{tree}[6]$ contiene el valor de $\text{sum}_q(5, 6)$.

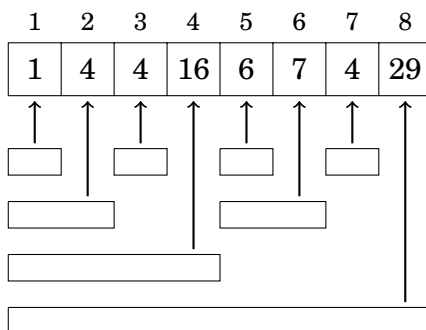
Por ejemplo, considera el siguiente arreglo:

1	2	3	4	5	6	7	8
1	3	4	8	6	1	4	2

El árbol indexado binario correspondiente es el siguiente:

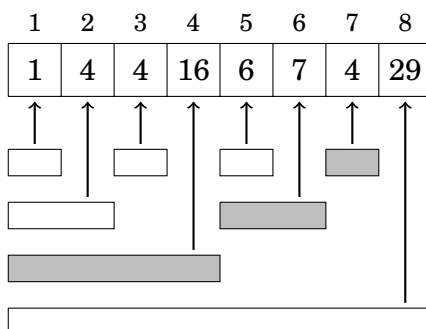
1	2	3	4	5	6	7	8
1	4	4	16	6	7	4	29

La siguiente figura muestra más claramente cómo cada valor del árbol indexado binario corresponde a un rango del arreglo original:



Usando un árbol indexado binario, cualquier valor de $\text{sum}_q(1, k)$ puede calcularse en tiempo $O(\log n)$, porque un rango $[1, k]$ siempre puede dividirse en $O(\log n)$ rangos cuyas sumas están almacenadas en el árbol.

Por ejemplo, el rango $[1, 7]$ consta de los siguientes rangos:



Así, podemos calcular la suma correspondiente de la siguiente manera:

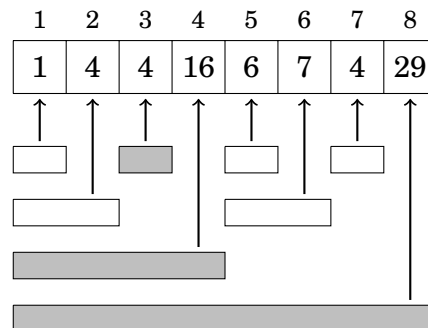
$$\text{sum}_q(1, 7) = \text{sum}_q(1, 4) + \text{sum}_q(5, 6) + \text{sum}_q(7, 7) = 16 + 7 + 4 = 27$$

Para calcular el valor de $\text{sum}_q(a, b)$ donde $a > 1$, podemos usar el mismo truco que usamos con los arreglos de sumas de prefijos:

$$\text{sum}_q(a, b) = \text{sum}_q(1, b) - \text{sum}_q(1, a - 1).$$

Como podemos calcular tanto $\text{sum}_q(1, b)$ como $\text{sum}_q(1, a - 1)$ en tiempo $O(\log n)$, la complejidad temporal total es $O(\log n)$.

Luego, después de actualizar un valor en el arreglo original, deberían actualizarse varios valores en el árbol indexado binario. Por ejemplo, si el valor en la posición 3 cambia, las sumas de los siguientes rangos cambian:



Como cada elemento del arreglo pertenece a $O(\log n)$ rangos en el árbol indexado binario, basta con actualizar $O(\log n)$ valores en el árbol.

Implementación

Las operaciones de un árbol indexado binario pueden implementarse de forma eficiente usando operaciones de bits. El hecho clave que se necesita es que podemos calcular cualquier valor de $p(k)$ usando la fórmula

$$p(k) = k \& -k.$$

La siguiente función calcula el valor de $\text{sum}_q(1, k)$:

```
int sum(int k) {
    int s = 0;
    while (k >= 1) {
        s += tree[k];
        k -= k&-k;
    }
    return s;
}
```

La siguiente función incrementa el valor del arreglo en la posición k en x (x puede ser positivo o negativo):

```

void add(int k, int x) {
    while (k <= n) {
        tree[k] += x;
        k += k&-k;
    }
}

```

La complejidad temporal de ambas funciones es $O(\log n)$, porque las funciones acceden a $O(\log n)$ valores en el árbol indexado binario, y cada movimiento a la siguiente posición toma $O(1)$ de tiempo.

9.3. Árbol de segmentos

Un **árbol de segmentos**³ es una estructura de datos que admite dos operaciones: procesar una consulta de rango y actualizar un valor del arreglo. Los árboles de segmentos pueden admitir consultas de suma, consultas de mínimo y máximo y muchas otras consultas de modo que ambas operaciones funcionen en tiempo $O(\log n)$.

Comparado con un árbol indexado binario, la ventaja de un árbol de segmentos es que es una estructura de datos más general. Mientras que los árboles indexados binarios solo admiten consultas de suma⁴, los árboles de segmentos también admiten otras consultas. Por otro lado, un árbol de segmentos requiere más memoria y es un poco más difícil de implementar.

Estructura

Un árbol de segmentos es un árbol binario tal que los nodos del nivel inferior del árbol corresponden a los elementos del arreglo, y los otros nodos contienen información necesaria para procesar consultas de rango.

En esta sección, suponemos que el tamaño del arreglo es una potencia de dos y se usa indexación desde cero, porque es conveniente construir un árbol de segmentos para tal arreglo. Si el tamaño del arreglo no es una potencia de dos, siempre podemos agregarle elementos adicionales.

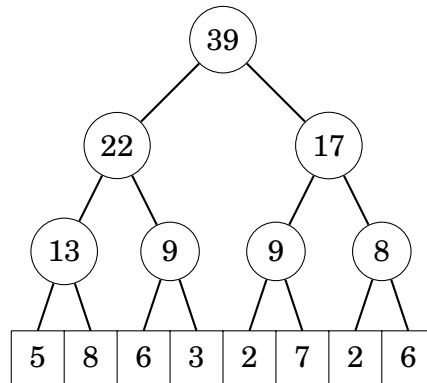
Primero discutiremos árboles de segmentos que admiten consultas de suma. Como ejemplo, considera el siguiente arreglo:

0	1	2	3	4	5	6	7
5	8	6	3	2	7	2	6

El árbol de segmentos correspondiente es el siguiente:

³La implementación de abajo hacia arriba de este capítulo corresponde a la de [62]. Estructuras similares se usaron a finales de los años 70 para resolver problemas geométricos [9].

⁴De hecho, usando *dos* árboles indexados binarios es posible admitir consultas de mínimo [16], pero esto es más complicado que usar un árbol de segmentos.

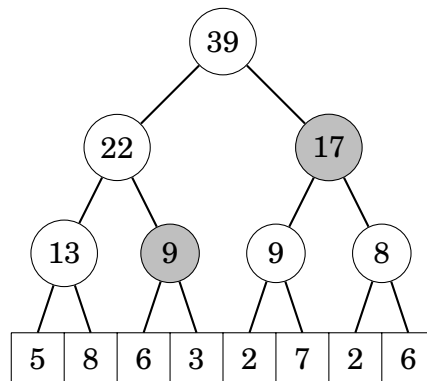


Cada nodo interno del árbol corresponde a un rango del arreglo cuyo tamaño es una potencia de dos. En el árbol anterior, el valor de cada nodo interno es la suma de los valores correspondientes del arreglo, y puede calcularse como la suma de los valores de sus nodos hijos izquierdo y derecho.

Resulta que cualquier rango $[a, b]$ puede dividirse en $O(\log n)$ rangos cuyos valores están almacenados en nodos del árbol. Por ejemplo, considera el rango $[2, 7]$:

0	1	2	3	4	5	6	7
5	8	6	3	2	7	2	6

Aquí $\text{sum}_q(2, 7) = 6 + 3 + 2 + 7 + 2 + 6 = 26$. En este caso, los siguientes dos nodos del árbol corresponden al rango:

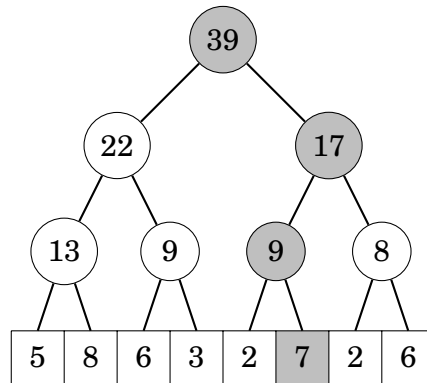


Así, otra forma de calcular la suma es $9 + 17 = 26$.

Cuando la suma se calcula usando nodos ubicados lo más alto posible en el árbol, se necesitan a lo sumo dos nodos en cada nivel del árbol. Por lo tanto, el número total de nodos es $O(\log n)$.

Después de una actualización del arreglo, deberíamos actualizar todos los nodos cuyo valor dependa del valor actualizado. Esto puede hacerse recorriendo el camino desde el elemento actualizado del arreglo hasta el nodo superior y actualizando los nodos a lo largo del camino.

La siguiente figura muestra qué nodos del árbol cambian si el valor 7 del arreglo cambia:

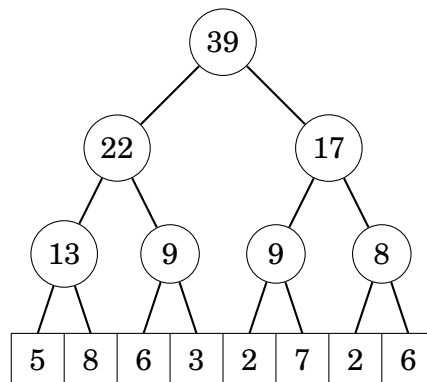


El camino de abajo hacia arriba siempre consta de $O(\log n)$ nodos, así que cada actualización cambia $O(\log n)$ nodos en el árbol.

Implementación

Almacenamos un árbol de segmentos como un arreglo de $2n$ elementos donde n es el tamaño del arreglo original y una potencia de dos. Los nodos del árbol se almacenan de arriba hacia abajo: $\text{tree}[1]$ es el nodo superior, $\text{tree}[2]$ y $\text{tree}[3]$ son sus hijos, y así sucesivamente. Finalmente, los valores desde $\text{tree}[n]$ hasta $\text{tree}[2n - 1]$ corresponden a los valores del arreglo original en el nivel inferior del árbol.

Por ejemplo, el árbol de segmentos



se almacena de la siguiente manera:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
39	22	17	13	9	9	8	5	8	6	3	2	7	2	6

Usando esta representación, el padre de $\text{tree}[k]$ es $\text{tree}[\lfloor k/2 \rfloor]$, y sus hijos son $\text{tree}[2k]$ y $\text{tree}[2k + 1]$. Nota que esto implica que la posición de un nodo es par si es un hijo izquierdo e impar si es un hijo derecho.

La siguiente función calcula el valor de $\text{sum}_q(a, b)$:

```

int sum(int a, int b) {
    a += n; b += n;
    int s = 0;

```

```

while (a <= b) {
    if (a%2 == 1) s += tree[a++];
    if (b%2 == 0) s += tree[b--];
    a /= 2; b /= 2;
}
return s;
}

```

La función mantiene un rango que inicialmente es $[a + n, b + n]$. Luego, en cada paso, el rango se mueve un nivel más arriba en el árbol, y antes de eso, los valores de los nodos que no pertenecen al rango superior se suman a la suma.

La siguiente función incrementa el valor del arreglo en la posición k en x :

```

void add(int k, int x) {
    k += n;
    tree[k] += x;
    for (k /= 2; k >= 1; k /= 2) {
        tree[k] = tree[2*k] + tree[2*k+1];
    }
}

```

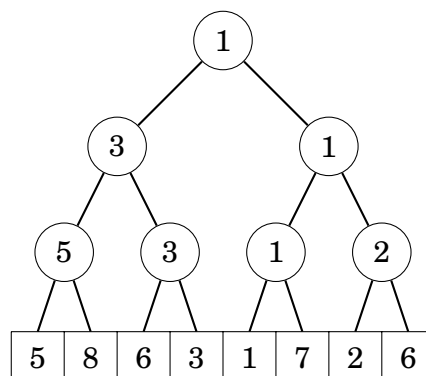
Primero la función actualiza el valor en el nivel inferior del árbol. Después de esto, la función actualiza los valores de todos los nodos internos del árbol, hasta que llega al nodo superior del árbol.

Ambas funciones anteriores funcionan en tiempo $O(\log n)$, porque un árbol de segmentos de n elementos consta de $O(\log n)$ niveles, y las funciones se mueven un nivel más arriba en el árbol en cada paso.

Otras consultas

Los árboles de segmentos pueden admitir todas las consultas de rango donde es posible dividir un rango en dos partes, calcular la respuesta por separado para ambas partes y luego combinar de forma eficiente las respuestas. Ejemplos de tales consultas son el mínimo y el máximo, el máximo común divisor, y las operaciones de bits and, or y xor.

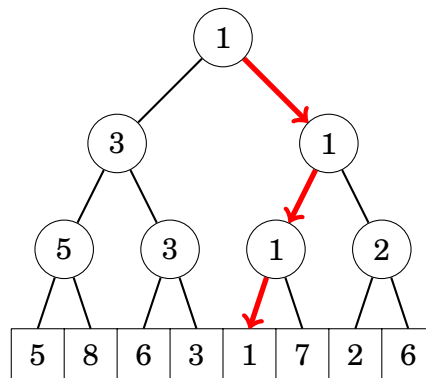
Por ejemplo, el siguiente árbol de segmentos admite consultas de mínimo:



En este caso, cada nodo del árbol contiene el valor más pequeño del rango correspondiente del arreglo. El nodo superior del árbol contiene el valor más pequeño de todo el arreglo. Las operaciones pueden implementarse como antes, pero en lugar de sumas, se calculan mínimos.

La estructura de un árbol de segmentos también nos permite usar búsqueda binaria para localizar elementos del arreglo. Por ejemplo, si el árbol admite consultas de mínimo, podemos encontrar la posición de un elemento con el valor más pequeño en tiempo $O(\log n)$.

Por ejemplo, en el árbol anterior, un elemento con el valor más pequeño 1 puede encontrarse recorriendo un camino hacia abajo desde el nodo superior:



9.4. Técnicas adicionales

Compresión de índices

Una limitación en las estructuras de datos que se construyen sobre un arreglo es que los elementos se indexan usando enteros consecutivos. Surgen dificultades cuando se necesitan índices grandes. Por ejemplo, si deseamos usar el índice 10^9 , el arreglo debería contener 10^9 elementos, lo que requeriría demasiada memoria.

Sin embargo, a menudo podemos sortear esta limitación usando **compresión de índices**, donde los índices originales se reemplazan con los índices 1, 2, 3, etc. Esto puede hacerse si conocemos de antemano todos los índices necesarios durante el algoritmo.

La idea es reemplazar cada índice original x con $c(x)$ donde c es una función que comprime los índices. Requerimos que el orden de los índices no cambie, así que si $a < b$, entonces $c(a) < c(b)$. Esto nos permite realizar consultas cómodamente aunque los índices estén comprimidos.

Por ejemplo, si los índices originales son 555, 10^9 y 8, los nuevos índices son:

$$\begin{aligned} c(8) &= 1 \\ c(555) &= 2 \\ c(10^9) &= 3 \end{aligned}$$

Actualizaciones de rango

Hasta ahora, hemos implementado estructuras de datos que admiten consultas de rango y actualizaciones de valores individuales. Consideremos ahora una situación opuesta, donde debemos actualizar rangos y recuperar valores individuales. Nos centramos en una operación que incrementa todos los elementos de un rango $[a, b]$ en x .

Sorprendentemente, podemos usar las estructuras de datos presentadas en este capítulo también en esta situación. Para hacer esto, construimos un **arreglo de diferencias** cuyos valores indican las diferencias entre valores consecutivos del arreglo original. Así, el arreglo original es el arreglo de sumas de prefijos del arreglo de diferencias. Por ejemplo, considera el siguiente arreglo:

0	1	2	3	4	5	6	7
3	3	1	1	1	5	2	2

El arreglo de diferencias para el arreglo anterior es el siguiente:

0	1	2	3	4	5	6	7
3	0	-2	0	0	4	-3	0

Por ejemplo, el valor 2 en la posición 6 del arreglo original corresponde a la suma $3 - 2 + 4 - 3 = 2$ en el arreglo de diferencias.

La ventaja del arreglo de diferencias es que podemos actualizar un rango en el arreglo original cambiando solo dos elementos en el arreglo de diferencias. Por ejemplo, si queremos incrementar en 5 los valores del arreglo original entre las posiciones 1 y 4, basta con incrementar el valor del arreglo de diferencias en la posición 1 en 5 y disminuir el valor en la posición 5 en 5. El resultado es el siguiente:

0	1	2	3	4	5	6	7
3	5	-2	0	0	-1	-3	0

Más en general, para incrementar los valores en el rango $[a, b]$ en x , incrementamos el valor en la posición a en x y disminuimos el valor en la posición $b + 1$ en x . Así, solo se necesita actualizar valores individuales y procesar consultas de suma, por lo que podemos usar un árbol indexado binario o un árbol de segmentos.

Un problema más difícil es admitir tanto consultas de rango como actualizaciones de rango. En el Capítulo 28 veremos que incluso esto es posible.

Bit manipulation

10.1. Bit representation

Here is the bit representation of the int number 43:

The bits in the representation are indexed from right to left. To convert a bit representation $b_k \cdots b_2 b_1 b_0$ into a number, we can use the formula

For example,

The bit representation of a number is either **signed** or **unsigned**. Usually a signed representation is used, which means that both negative and positive numbers can be represented. A signed variable of n bits can contain any integer between -2^{n-1} and $2^{n-1} - 1$. For example, the `int` type in C++ is a signed type, so an `int` variable can contain any integer between -2^{31} and $2^{31} - 1$.

For example, the bit representation of the int number `-43` is

103

In an unsigned representation, only nonnegative numbers can be used, but the upper bound for the values is larger. An unsigned variable of n bits can contain any integer between 0 and $2^n - 1$. For example, in C++, an unsigned int variable can contain any integer between 0 and $2^{32} - 1$.

There is a connection between the representations: a signed number $-x$ equals an unsigned number $2^n - x$. For example, the following code shows that the signed number $x = -43$ equals the unsigned number $y = 2^{32} - 43$:

```
int x = -43;
unsigned int y = x;
cout << x << "\n"; // -43
cout << y << "\n"; // 4294967253
```

If a number is larger than the upper bound of the bit representation, the number will overflow. In a signed representation, the next number after $2^{n-1} - 1$ is -2^{n-1} , and in an unsigned representation, the next number after $2^n - 1$ is 0. For example, consider the following code:

```
int x = 2147483647
cout << x << "\n"; // 2147483647
x++;
cout << x << "\n"; // -2147483648
```

Initially, the value of x is $2^{31} - 1$. This is the largest value that can be stored in an int variable, so the next number after $2^{31} - 1$ is -2^{31} .

10.2. Bit operations

And operation

The **and** operation $x \& y$ produces a number that has one bits in positions where both x and y have one bits. For example, $22 \& 26 = 18$, because

$$\begin{array}{r} 10110 \quad (22) \\ \& \quad 11010 \quad (26) \\ \hline = \quad 10010 \quad (18) \end{array}$$

Using the and operation, we can check if a number x is even because $x \& 1 = 0$ if x is even, and $x \& 1 = 1$ if x is odd. More generally, x is divisible by 2^k exactly when $x \& (2^k - 1) = 0$.

Or operation

The **or** operation $x \mid y$ produces a number that has one bits in positions where at least one of x and y have one bits. For example, $22 \mid 26 = 30$, because

$$\begin{array}{r} 10110 \quad (22) \\ \mid \quad 11010 \quad (26) \\ \hline = \quad 11110 \quad (30) \end{array}$$

Xor operation

The **xor** operation $x \wedge y$ produces a number that has one bits in positions where exactly one of x and y have one bits. For example, $22 \wedge 26 = 12$, because

$$\begin{array}{r} 10110 \quad (22) \\ \wedge \quad 11010 \quad (26) \\ \hline = \quad 01100 \quad (12) \end{array}$$

Not operation

The **not** operation $\sim x$ produces a number where all the bits of x have been inverted. The formula $\sim x = -x - 1$ holds, for example, $\sim 29 = -30$.

The result of the not operation at the bit level depends on the length of the bit representation, because the operation inverts all bits. For example, if the numbers are 32-bit int numbers, the result is as follows:

$$\begin{array}{rcl} x & = & 29 \quad 000000000000000000000000011101 \\ \sim x & = & -30 \quad 111111111111111111111111110010 \end{array}$$

Bit shifts

The left bit shift $x \ll k$ appends k zero bits to the number, and the right bit shift $x \gg k$ removes the k last bits from the number. For example, $14 \ll 2 = 56$, because 14 and 56 correspond to 1110 and 111000. Similarly, $49 \gg 3 = 6$, because 49 and 6 correspond to 110001 and 110.

Note that $x \ll k$ corresponds to multiplying x by 2^k , and $x \gg k$ corresponds to dividing x by 2^k rounded down to an integer.

Applications

A number of the form $1 \ll k$ has a one bit in position k and all other bits are zero, so we can use such numbers to access single bits of numbers. In particular, the k th bit of a number is one exactly when $x \& (1 \ll k)$ is not zero. The following code prints the bit representation of an int number x :

```
for (int i = 31; i >= 0; i--) {
    if (x & (1 << i)) cout << "1";
    else cout << "0";
}
```

It is also possible to modify single bits of numbers using similar ideas. For example, the formula $x \mid (1 \ll k)$ sets the k th bit of x to one, the formula $x \& \sim(1 \ll k)$ sets the k th bit of x to zero, and the formula $x \wedge (1 \ll k)$ inverts the k th bit of x .

The formula $x \& (x - 1)$ sets the last one bit of x to zero, and the formula $x \& -x$ sets all the one bits to zero, except for the last one bit. The formula $x \mid (x - 1)$ inverts all the bits after the last one bit. Also note that a positive number x is a power of two exactly when $x \& (x - 1) = 0$.

Additional functions

The g++ compiler provides the following functions for counting bits:

- `__builtin_clz(x)`: the number of zeros at the beginning of the number
- `__builtin_ctz(x)`: the number of zeros at the end of the number
- `__builtin_popcount(x)`: the number of ones in the number
- `__builtin_parity(x)`: the parity (even or odd) of the number of ones

The functions can be used as follows:

```
int x = 5328; // 000000000000000000001010011010000
cout << __builtin_clz(x) << "\n"; // 19
cout << __builtin_ctz(x) << "\n"; // 4
cout << __builtin_popcount(x) << "\n"; // 5
cout << __builtin_parity(x) << "\n"; // 1
```

While the above functions only support `int` numbers, there are also `long` versions of the functions available with the suffix `ll`.

10.3. Representing sets

Every subset of a set $\{0, 1, 2, \dots, n-1\}$ can be represented as an n bit integer whose one bits indicate which elements belong to the subset. This is an efficient way to represent sets, because every element requires only one bit of memory, and set operations can be implemented as bit operations.

For example, since `int` is a 32-bit type, an `int` number can represent any subset of the set $\{0, 1, 2, \dots, 31\}$. The bit representation of the set $\{1, 3, 4, 8\}$ is

000000000000000000000000100011010,

which corresponds to the number $2^8 + 2^4 + 2^3 + 2^1 = 282$.

Set implementation

The following code declares an `int` variable `x` that can contain a subset of $\{0, 1, 2, \dots, 31\}$. After this, the code adds the elements 1, 3, 4 and 8 to the set and prints the size of the set.

```
int x = 0;
x |= (1<<1);
x |= (1<<3);
x |= (1<<4);
x |= (1<<8);
cout << __builtin_popcount(x) << "\n"; // 4
```


Then, the following code prints all elements that belong to the set:

```
for (int i = 0; i < 32; i++) {
    if (x & (1 << i)) cout << i << " ";
}
// output: 1 3 4 8
```

Set operations

Set operations can be implemented as follows as bit operations:

	set syntax	bit syntax
intersection	$a \cap b$	$a \& b$
union	$a \cup b$	$a \mid b$
complement	$\bar{a}$	$\sim a$
difference	$a \setminus b$	$a \& (\sim b)$

For example, the following code first constructs the sets $x = \{1, 3, 4, 8\}$ and $y = \{3, 6, 8, 9\}$, and then constructs the set $z = x \cup y = \{1, 3, 4, 6, 8, 9\}$:

```
int x = (1 << 1) | (1 << 3) | (1 << 4) | (1 << 8);
int y = (1 << 3) | (1 << 6) | (1 << 8) | (1 << 9);
int z = x | y;
cout << __builtin_popcount(z) << "\n"; // 6
```

Iterating through subsets

The following code goes through the subsets of $\{0, 1, \dots, n - 1\}$:

```
for (int b = 0; b < (1 << n); b++) {
    // process subset b
}
```

The following code goes through the subsets with exactly k elements:

```
for (int b = 0; b < (1 << n); b++) {
    if (__builtin_popcount(b) == k) {
        // process subset b
    }
}
```

The following code goes through the subsets of a set x :

```
int b = 0;
do {
    // process subset b
} while (b = (b - x) & x);
```

10.4. Bit optimizations

Many algorithms can be optimized using bit operations. Such optimizations do not change the time complexity of the algorithm, but they may have a large impact on the actual running time of the code. In this section we discuss examples of such situations.

Hamming distances

The **Hamming distance** $\text{hamming}(a, b)$ between two strings a and b of equal length is the number of positions where the strings differ. For example,

$$\text{hamming}(01101, 11001) = 2.$$

Consider the following problem: Given a list of n bit strings, each of length k , calculate the minimum Hamming distance between two strings in the list. For example, the answer for $[00111, 01101, 11110]$ is 2, because

- $\text{hamming}(00111, 01101) = 2$,
- $\text{hamming}(00111, 11110) = 3$, and
- $\text{hamming}(01101, 11110) = 3$.

A straightforward way to solve the problem is to go through all pairs of strings and calculate their Hamming distances, which yields an $O(n^2k)$ time algorithm. The following function can be used to calculate distances:

```
int hamming(string a, string b) {  
    int d = 0;  
    for (int i = 0; i < k; i++) {  
        if (a[i] != b[i]) d++;  
    }  
    return d;  
}
```

However, if k is small, we can optimize the code by storing the bit strings as integers and calculating the Hamming distances using bit operations. In particular, if $k \leq 32$, we can just store the strings as `int` values and use the following function to calculate distances:

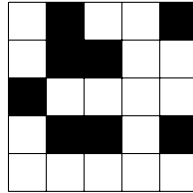
```
int hamming(int a, int b) {  
    return __builtin_popcount(a^b);  
}
```

In the above function, the xor operation constructs a bit string that has one bits in positions where a and b differ. Then, the number of bits is calculated using the `__builtin_popcount` function.

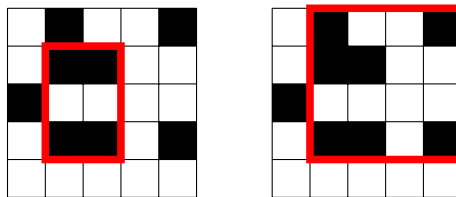
To compare the implementations, we generated a list of 10000 random bit strings of length 30. Using the first approach, the search took 13.5 seconds, and after the bit optimization, it only took 0.5 seconds. Thus, the bit optimized code was almost 30 times faster than the original code.

Counting subgrids

As another example, consider the following problem: Given an $n \times n$ grid whose each square is either black (1) or white (0), calculate the number of subgrids whose all corners are black. For example, the grid



contains two such subgrids:



There is an $O(n^3)$ time algorithm for solving the problem: go through all $O(n^2)$ pairs of rows and for each pair (a, b) calculate the number of columns that contain a black square in both rows in $O(n)$ time. The following code assumes that `color[y][x]` denotes the color in row y and column x :

```
int count = 0;
for (int i = 0; i < n; i++) {
    if (color[a][i] == 1 && color[b][i] == 1) count++;
}
```

Then, those columns account for $\text{count}(\text{count} - 1)/2$ subgrids with black corners, because we can choose any two of them to form a subgrid.

To optimize this algorithm, we divide the grid into blocks of columns such that each block consists of N consecutive columns. Then, each row is stored as a list of N -bit numbers that describe the colors of the squares. Now we can process N columns at the same time using bit operations. In the following code, `color[y][k]` represents a block of N colors as bits.

```
int count = 0;
for (int i = 0; i <= n/N; i++) {
    count += __builtin_popcount(color[a][i]&color[b][i]);
}
```

The resulting algorithm works in $O(n^3/N)$ time.

We generated a random grid of size 2500×2500 and compared the original and bit optimized implementation. While the original code took 29.6 seconds, the bit optimized version only took 3.1 seconds with $N = 32$ (int numbers) and 1.7 seconds with $N = 64$ (long long numbers).

10.5. Dynamic programming

Bit operations provide an efficient and convenient way to implement dynamic programming algorithms whose states contain subsets of elements, because such states can be stored as integers. Next we discuss examples of combining bit operations and dynamic programming.

Optimal selection

As a first example, consider the following problem: We are given the prices of k products over n days, and we want to buy each product exactly once. However, we are allowed to buy at most one product in a day. What is the minimum total price? For example, consider the following scenario ($k = 3$ and $n = 8$):

	0	1	2	3	4	5	6	7
product 0	6	9	5	2	8	9	1	6
product 1	8	2	6	2	7	5	7	2
product 2	5	3	9	7	3	5	1	4

In this scenario, the minimum total price is 5:

	0	1	2	3	4	5	6	7
product 0	6	9	5	2	8	9	1	6
product 1	8	2	6	2	7	5	7	2
product 2	5	3	9	7	3	5	1	4

Let $\text{price}[x][d]$ denote the price of product x on day d . For example, in the above scenario $\text{price}[2][3] = 7$. Then, let $\text{total}(S, d)$ denote the minimum total price for buying a subset S of products by day d . Using this function, the solution to the problem is $\text{total}(\{0 \dots k-1\}, n-1)$.

First, $\text{total}(\emptyset, d) = 0$, because it does not cost anything to buy an empty set, and $\text{total}(\{x\}, 0) = \text{price}[x][0]$, because there is one way to buy one product on the first day. Then, the following recurrence can be used:

$$\text{total}(S, d) = \min(\text{total}(S, d-1), \min_{x \in S} (\text{total}(S \setminus x, d-1) + \text{price}[x][d]))$$

This means that we either do not buy any product on day d or buy a product x that belongs to S . In the latter case, we remove x from S and add the price of x to the total price.

The next step is to calculate the values of the function using dynamic programming. To store the function values, we declare an array

```
int total[1<<K][N];
```

where K and N are suitably large constants. The first dimension of the array corresponds to a bit representation of a subset.

First, the cases where $d = 0$ can be processed as follows:

```
for (int x = 0; x < k; x++) {
    total[1<<x][0] = price[x][0];
}
```

Then, the recurrence translates into the following code:

```
for (int d = 1; d < n; d++) {
    for (int s = 0; s < (1<<k); s++) {
        total[s][d] = total[s][d-1];
        for (int x = 0; x < k; x++) {
            if (s & (1<<x)) {
                total[s][d] = min(total[s][d],
                                   total[s^(1<<x)][d-1] + price[x][d]);
            }
        }
    }
}
```

The time complexity of the algorithm is $O(n2^k k)$.

From permutations to subsets

Using dynamic programming, it is often possible to change an iteration over permutations into an iteration over subsets¹. The benefit of this is that $n!$, the number of permutations, is much larger than 2^n , the number of subsets. For example, if $n = 20$, then $n! \approx 2.4 \cdot 10^{18}$ and $2^n \approx 10^6$. Thus, for certain values of n , we can efficiently go through the subsets but not through the permutations.

As an example, consider the following problem: There is an elevator with maximum weight x , and n people with known weights who want to get from the ground floor to the top floor. What is the minimum number of rides needed if the people enter the elevator in an optimal order?

For example, suppose that $x = 10$, $n = 5$ and the weights are as follows:

person	weight
0	2
1	3
2	3
3	5
4	6

In this case, the minimum number of rides is 2. One optimal order is {0, 2, 3, 1, 4}, which partitions the people into two rides: first {0, 2, 3} (total weight 10), and then {1, 4} (total weight 9).

¹This technique was introduced in 1962 by M. Held and R. M. Karp [34].

The problem can be easily solved in $O(n!n)$ time by testing all possible permutations of n people. However, we can use dynamic programming to get a more efficient $O(2^n n)$ time algorithm. The idea is to calculate for each subset of people two values: the minimum number of rides needed and the minimum weight of people who ride in the last group.

Let $\text{weight}[p]$ denote the weight of person p . We define two functions: $\text{rides}(S)$ is the minimum number of rides for a subset S , and $\text{last}(S)$ is the minimum weight of the last ride. For example, in the above scenario

$$\text{rides}(\{1,3,4\}) = 2 \quad \text{and} \quad \text{last}(\{1,3,4\}) = 5,$$

because the optimal rides are $\{1,4\}$ and $\{3\}$, and the second ride has weight 5. Of course, our final goal is to calculate the value of $\text{rides}(\{0 \dots n-1\})$.

We can calculate the values of the functions recursively and then apply dynamic programming. The idea is to go through all people who belong to S and optimally choose the last person p who enters the elevator. Each such choice yields a subproblem for a smaller subset of people. If $\text{last}(S \setminus p) + \text{weight}[p] \leq x$, we can add p to the last ride. Otherwise, we have to reserve a new ride that initially only contains p .

To implement dynamic programming, we declare an array

```
pair<int,int> best[1<<N];
```

that contains for each subset S a pair $(\text{rides}(S), \text{last}(S))$. We set the value for an empty group as follows:

```
best[0] = {1,0};
```

Then, we can fill the array as follows:

```
for (int s = 1; s < (1<<n); s++) {
    // initial value: n+1 rides are needed
    best[s] = {n+1,0};
    for (int p = 0; p < n; p++) {
        if (s&(1<<p)) {
            auto option = best[s^(1<<p)];
            if (option.second+weight[p] <= x) {
                // add p to an existing ride
                option.second += weight[p];
            } else {
                // reserve a new ride for p
                option.first++;
                option.second = weight[p];
            }
            best[s] = min(best[s], option);
        }
    }
}
```

Note that the above loop guarantees that for any two subsets S_1 and S_2 such that $S_1 \subset S_2$, we process S_1 before S_2 . Thus, the dynamic programming values are calculated in the correct order.

Counting subsets

Our last problem in this chapter is as follows: Let $X = \{0 \dots n-1\}$, and each subset $S \subset X$ is assigned an integer $\text{value}[S]$. Our task is to calculate for each S

$$\text{sum}(S) = \sum_{A \subset S} \text{value}[A],$$

i.e., the sum of values of subsets of S .

For example, suppose that $n = 3$ and the values are as follows:

- | | |
|---------------------------------|-----------------------------------|
| ▪ $\text{value}[\emptyset] = 3$ | ▪ $\text{value}[\{2\}] = 5$ |
| ▪ $\text{value}[\{0\}] = 1$ | ▪ $\text{value}[\{0, 2\}] = 1$ |
| ▪ $\text{value}[\{1\}] = 4$ | ▪ $\text{value}[\{1, 2\}] = 3$ |
| ▪ $\text{value}[\{0, 1\}] = 5$ | ▪ $\text{value}[\{0, 1, 2\}] = 3$ |

In this case, for example,

$$\begin{aligned} \text{sum}(\{0, 2\}) &= \text{value}[\emptyset] + \text{value}[\{0\}] + \text{value}[\{2\}] + \text{value}[\{0, 2\}] \\ &= 3 + 1 + 5 + 1 = 10. \end{aligned}$$

Because there are a total of 2^n subsets, one possible solution is to go through all pairs of subsets in $O(2^{2n})$ time. However, using dynamic programming, we can solve the problem in $O(2^n n)$ time. The idea is to focus on sums where the elements that may be removed from S are restricted.

Let $\text{partial}(S, k)$ denote the sum of values of subsets of S with the restriction that only elements $0 \dots k$ may be removed from S . For example,

$$\text{partial}(\{0, 2\}, 1) = \text{value}[\{2\}] + \text{value}[\{0, 2\}],$$

because we may only remove elements $0 \dots 1$. We can calculate values of sum using values of partial , because

$$\text{sum}(S) = \text{partial}(S, n-1).$$

The base cases for the function are

$$\text{partial}(S, -1) = \text{value}[S],$$

because in this case no elements can be removed from S . Then, in the general case we can use the following recurrence:

$$\text{partial}(S, k) = \begin{cases} \text{partial}(S, k-1) & k \notin S \\ \text{partial}(S, k-1) + \text{partial}(S \setminus \{k\}, k-1) & k \in S \end{cases}$$

Here we focus on the element k . If $k \in S$, we have two options: we may either keep k in S or remove it from S .

There is a particularly clever way to implement the calculation of sums. We can declare an array

```
int sum[1<<N];
```

that will contain the sum of each subset. The array is initialized as follows:

```
for (int s = 0; s < (1<<n); s++) {  
    sum[s] = value[s];  
}
```

Then, we can fill the array as follows:

```
for (int k = 0; k < n; k++) {  
    for (int s = 0; s < (1<<n); s++) {  
        if (s & (1<<k)) sum[s] += sum[s^(1<<k)];  
    }  
}
```

This code calculates the values of $\text{partial}(S, k)$ for $k = 0 \dots n - 1$ to the array `sum`. Since $\text{partial}(S, k)$ is always based on $\text{partial}(S, k - 1)$, we can reuse the array `sum`, which yields a very efficient implementation.

Parte II

Algoritmos de grafos

Capítulo 11

Basics of graphs

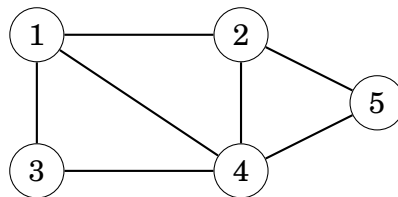
Many programming problems can be solved by modeling the problem as a graph problem and using an appropriate graph algorithm. A typical example of a graph is a network of roads and cities in a country. Sometimes, though, the graph is hidden in the problem and it may be difficult to detect it.

This part of the book discusses graph algorithms, especially focusing on topics that are important in competitive programming. In this chapter, we go through concepts related to graphs, and study different ways to represent graphs in algorithms.

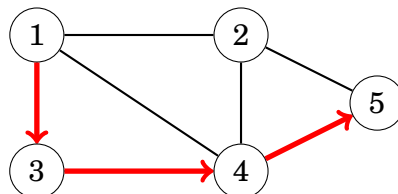
11.1. Graph terminology

A **graph** consists of **nodes** and **edges**. In this book, the variable n denotes the number of nodes in a graph, and the variable m denotes the number of edges. The nodes are numbered using integers $1, 2, \dots, n$.

For example, the following graph consists of 5 nodes and 7 edges:



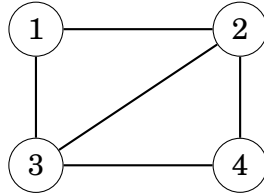
A **path** leads from node a to node b through edges of the graph. The **length** of a path is the number of edges in it. For example, the above graph contains a path $1 \rightarrow 3 \rightarrow 4 \rightarrow 5$ of length 3 from node 1 to node 5:



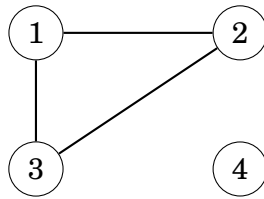
A path is a **cycle** if the first and last node is the same. For example, the above graph contains a cycle $1 \rightarrow 3 \rightarrow 4 \rightarrow 1$. A path is **simple** if each node appears at most once in the path.

Connectivity

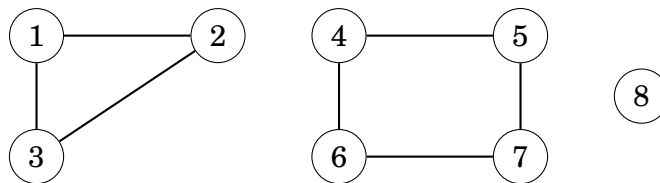
A graph is **connected** if there is a path between any two nodes. For example, the following graph is connected:



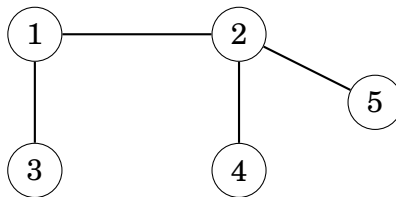
The following graph is not connected, because it is not possible to get from node 4 to any other node:



The connected parts of a graph are called its **components**. For example, the following graph contains three components: {1, 2, 3}, {4, 5, 6, 7} and {8}.

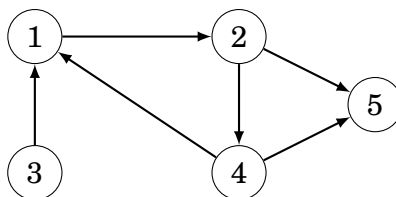


A **tree** is a connected graph that consists of n nodes and $n - 1$ edges. There is a unique path between any two nodes of a tree. For example, the following graph is a tree:



Edge directions

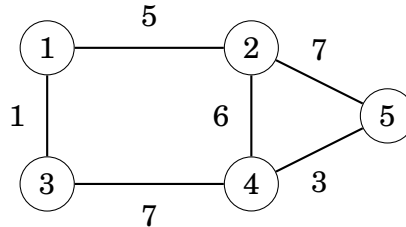
A graph is **directed** if the edges can be traversed in one direction only. For example, the following graph is directed:



The above graph contains a path $3 \rightarrow 1 \rightarrow 2 \rightarrow 5$ from node 3 to node 5, but there is no path from node 5 to node 3.

Edge weights

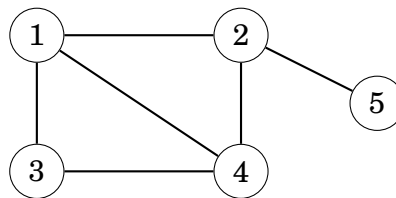
In a **weighted** graph, each edge is assigned a **weight**. The weights are often interpreted as edge lengths. For example, the following graph is weighted:



The length of a path in a weighted graph is the sum of the edge weights on the path. For example, in the above graph, the length of the path $1 \rightarrow 2 \rightarrow 5$ is 12, and the length of the path $1 \rightarrow 3 \rightarrow 4 \rightarrow 5$ is 11. The latter path is the **shortest** path from node 1 to node 5.

Neighbors and degrees

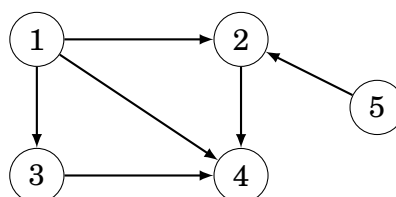
Two nodes are **neighbors** or **adjacent** if there is an edge between them. The **degree** of a node is the number of its neighbors. For example, in the following graph, the neighbors of node 2 are 1, 4 and 5, so its degree is 3.



The sum of degrees in a graph is always $2m$, where m is the number of edges, because each edge increases the degree of exactly two nodes by one. For this reason, the sum of degrees is always even.

A graph is **regular** if the degree of every node is a constant d . A graph is **complete** if the degree of every node is $n - 1$, i.e., the graph contains all possible edges between the nodes.

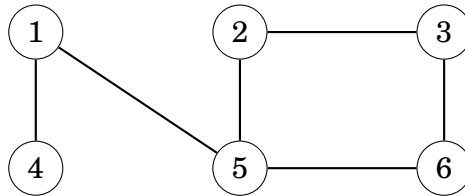
In a directed graph, the **indegree** of a node is the number of edges that end at the node, and the **outdegree** of a node is the number of edges that start at the node. For example, in the following graph, the indegree of node 2 is 2, and the outdegree of node 2 is 1.



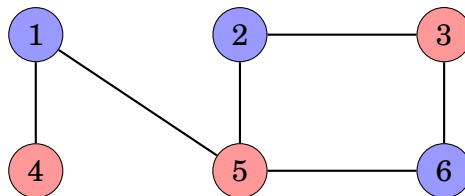
Colorings

In a **coloring** of a graph, each node is assigned a color so that no adjacent nodes have the same color.

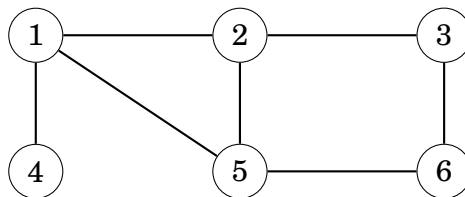
A graph is **bipartite** if it is possible to color it using two colors. It turns out that a graph is bipartite exactly when it does not contain a cycle with an odd number of edges. For example, the graph



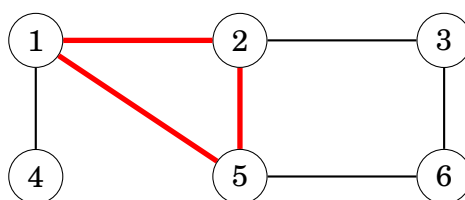
is bipartite, because it can be colored as follows:



However, the graph

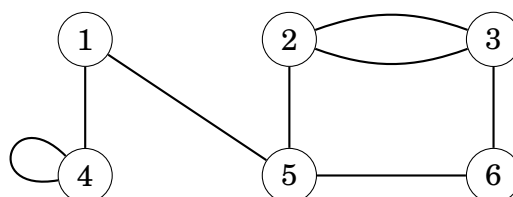


is not bipartite, because it is not possible to color the following cycle of three nodes using two colors:



Simplicity

A graph is **simple** if no edge starts and ends at the same node, and there are no multiple edges between two nodes. Often we assume that graphs are simple. For example, the following graph is *not* simple:



11.2. Graph representation

There are several ways to represent graphs in algorithms. The choice of a data structure depends on the size of the graph and the way the algorithm processes it. Next we will go through three common representations.

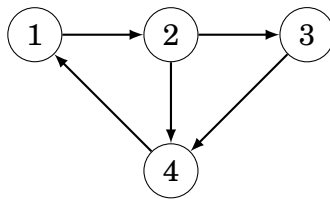
Adjacency list representation

In the adjacency list representation, each node x in the graph is assigned an **adjacency list** that consists of nodes to which there is an edge from x . Adjacency lists are the most popular way to represent graphs, and most algorithms can be efficiently implemented using them.

A convenient way to store the adjacency lists is to declare an array of vectors as follows:

```
vector<int> adj[N];
```

The constant N is chosen so that all adjacency lists can be stored. For example, the graph



can be stored as follows:

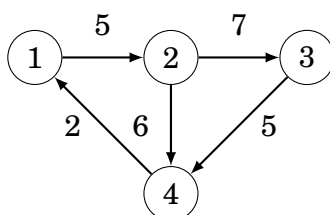
```
adj[1].push_back(2);  
adj[2].push_back(3);  
adj[2].push_back(4);  
adj[3].push_back(4);  
adj[4].push_back(1);
```

If the graph is undirected, it can be stored in a similar way, but each edge is added in both directions.

For a weighted graph, the structure can be extended as follows:

```
vector<pair<int,int>> adj[N];
```

In this case, the adjacency list of node a contains the pair (b, w) always when there is an edge from node a to node b with weight w . For example, the graph



can be stored as follows:

```
adj[1].push_back({2,5});  
adj[2].push_back({3,7});  
adj[2].push_back({4,6});  
adj[3].push_back({4,5});  
adj[4].push_back({1,2});
```

The benefit of using adjacency lists is that we can efficiently find the nodes to which we can move from a given node through an edge. For example, the following loop goes through all nodes to which we can move from node s :

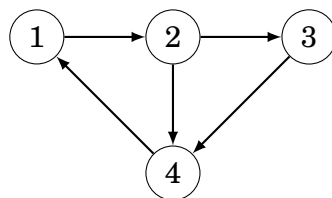
```
for (auto u : adj[s]) {  
    // process node u  
}
```

Adjacency matrix representation

An **adjacency matrix** is a two-dimensional array that indicates which edges the graph contains. We can efficiently check from an adjacency matrix if there is an edge between two nodes. The matrix can be stored as an array

```
int adj[N][N];
```

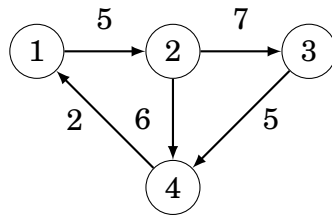
where each value $\text{adj}[a][b]$ indicates whether the graph contains an edge from node a to node b . If the edge is included in the graph, then $\text{adj}[a][b] = 1$, and otherwise $\text{adj}[a][b] = 0$. For example, the graph



can be represented as follows:

	1	2	3	4
1	0	1	0	0
2	0	0	1	1
3	0	0	0	1
4	1	0	0	0

If the graph is weighted, the adjacency matrix representation can be extended so that the matrix contains the weight of the edge if the edge exists. Using this representation, the graph



corresponds to the following matrix:

	1	2	3	4
1	0	5	0	0
2	0	0	7	6
3	0	0	0	5
4	2	0	0	0

The drawback of the adjacency matrix representation is that the matrix contains n^2 elements, and usually most of them are zero. For this reason, the representation cannot be used if the graph is large.

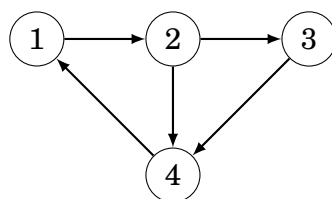
Edge list representation

An **edge list** contains all edges of a graph in some order. This is a convenient way to represent a graph if the algorithm processes all edges of the graph and it is not needed to find edges that start at a given node.

The edge list can be stored in a vector

```
vector<pair<int,int>> edges;
```

where each pair (a,b) denotes that there is an edge from node a to node b . Thus, the graph



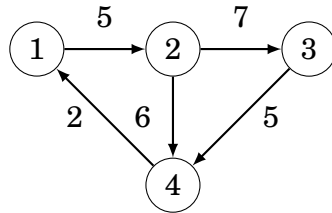
can be represented as follows:

```
edges.push_back({1,2});
edges.push_back({2,3});
edges.push_back({2,4});
edges.push_back({3,4});
edges.push_back({4,1});
```

If the graph is weighted, the structure can be extended as follows:

```
vector<tuple<int,int,int>> edges;
```

Each element in this list is of the form (a,b,w) , which means that there is an edge from node a to node b with weight w . For example, the graph



can be represented as follows¹:

```
edges.push_back({1,2,5});  
edges.push_back({2,3,7});  
edges.push_back({2,4,6});  
edges.push_back({3,4,5});  
edges.push_back({4,1,2});
```

¹In some older compilers, the function `make_tuple` must be used instead of the braces (for example, `make_tuple(1,2,5)` instead of `{1,2,5}`).

Capítulo 12

Graph traversal

This chapter discusses two fundamental graph algorithms: depth-first search and breadth-first search. Both algorithms are given a starting node in the graph, and they visit all nodes that can be reached from the starting node. The difference in the algorithms is the order in which they visit the nodes.

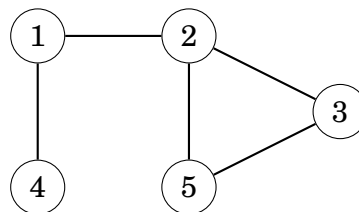
12.1. Depth-first search

Depth-first search (DFS) is a straightforward graph traversal technique. The algorithm begins at a starting node, and proceeds to all other nodes that are reachable from the starting node using the edges of the graph.

Depth-first search always follows a single path in the graph as long as it finds new nodes. After this, it returns to previous nodes and begins to explore other parts of the graph. The algorithm keeps track of visited nodes, so that it processes each node only once.

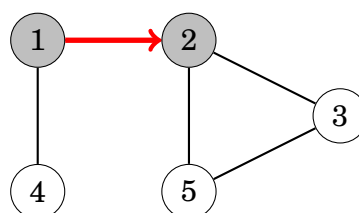
Example

Let us consider how depth-first search processes the following graph:

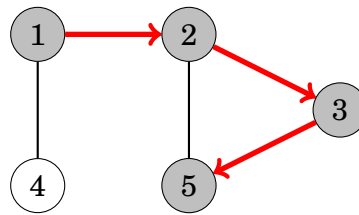


We may begin the search at any node of the graph; now we will begin the search at node 1.

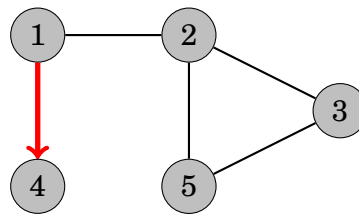
The search first proceeds to node 2:



After this, nodes 3 and 5 will be visited:



The neighbors of node 5 are 2 and 3, but the search has already visited both of them, so it is time to return to the previous nodes. Also the neighbors of nodes 3 and 2 have been visited, so we next move from node 1 to node 4:



After this, the search terminates because it has visited all nodes.

The time complexity of depth-first search is $O(n + m)$ where n is the number of nodes and m is the number of edges, because the algorithm processes each node and edge once.

Implementation

Depth-first search can be conveniently implemented using recursion. The following function `dfs` begins a depth-first search at a given node. The function assumes that the graph is stored as adjacency lists in an array

```
vector<int> adj[N];
```

and also maintains an array

```
bool visited[N];
```

that keeps track of the visited nodes. Initially, each array value is false, and when the search arrives at node s , the value of `visited[s]` becomes true. The function can be implemented as follows:

```
void dfs(int s) {
    if (visited[s]) return;
    visited[s] = true;
    // process node s
    for (auto u: adj[s]) {
        dfs(u);
    }
}
```

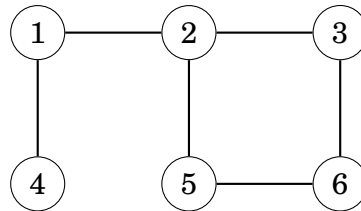
12.2. Breadth-first search

Breadth-first search (BFS) visits the nodes in increasing order of their distance from the starting node. Thus, we can calculate the distance from the starting node to all other nodes using breadth-first search. However, breadth-first search is more difficult to implement than depth-first search.

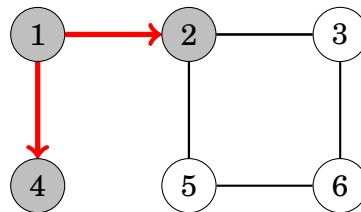
Breadth-first search goes through the nodes one level after another. First the search explores the nodes whose distance from the starting node is 1, then the nodes whose distance is 2, and so on. This process continues until all nodes have been visited.

Example

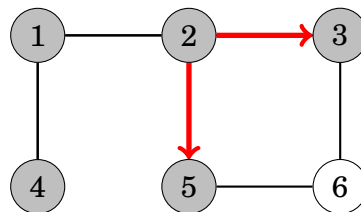
Let us consider how breadth-first search processes the following graph:



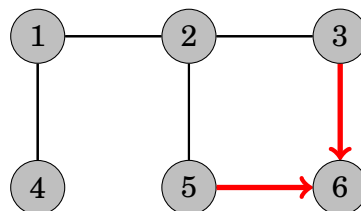
Suppose that the search begins at node 1. First, we process all nodes that can be reached from node 1 using a single edge:



After this, we proceed to nodes 3 and 5:



Finally, we visit node 6:



Now we have calculated the distances from the starting node to all nodes of the graph. The distances are as follows:

node	distance
1	0
2	1
3	2
4	1
5	2
6	3

Like in depth-first search, the time complexity of breadth-first search is $O(n + m)$, where n is the number of nodes and m is the number of edges.

Implementation

Breadth-first search is more difficult to implement than depth-first search, because the algorithm visits nodes in different parts of the graph. A typical implementation is based on a queue that contains nodes. At each step, the next node in the queue will be processed.

The following code assumes that the graph is stored as adjacency lists and maintains the following data structures:

```
queue<int> q;  
bool visited[N];  
int distance[N];
```

The queue q contains nodes to be processed in increasing order of their distance. New nodes are always added to the end of the queue, and the node at the beginning of the queue is the next node to be processed. The array `visited` indicates which nodes the search has already visited, and the array `distance` will contain the distances from the starting node to all nodes of the graph.

The search can be implemented as follows, starting at node x :

```
visited[x] = true;  
distance[x] = 0;  
q.push(x);  
while (!q.empty()) {  
    int s = q.front(); q.pop();  
    // process node s  
    for (auto u : adj[s]) {  
        if (visited[u]) continue;  
        visited[u] = true;  
        distance[u] = distance[s]+1;  
        q.push(u);  
    }  
}
```

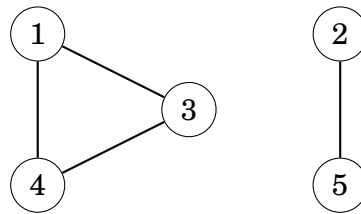
12.3. Applications

Using the graph traversal algorithms, we can check many properties of graphs. Usually, both depth-first search and breadth-first search may be used, but in practice, depth-first search is a better choice, because it is easier to implement. In the following applications we will assume that the graph is undirected.

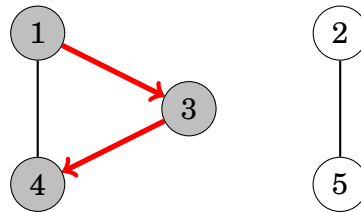
Connectivity check

A graph is connected if there is a path between any two nodes of the graph. Thus, we can check if a graph is connected by starting at an arbitrary node and finding out if we can reach all other nodes.

For example, in the graph



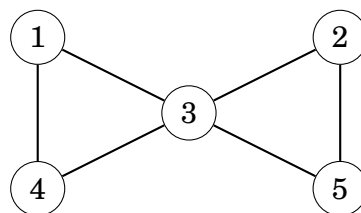
a depth-first search from node 1 visits the following nodes:



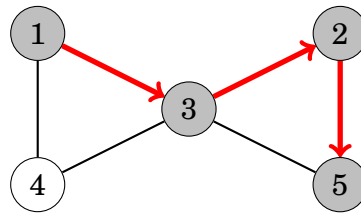
Since the search did not visit all the nodes, we can conclude that the graph is not connected. In a similar way, we can also find all connected components of a graph by iterating through the nodes and always starting a new depth-first search if the current node does not belong to any component yet.

Finding cycles

A graph contains a cycle if during a graph traversal, we find a node whose neighbor (other than the previous node in the current path) has already been visited. For example, the graph



contains two cycles and we can find one of them as follows:



After moving from node 2 to node 5 we notice that the neighbor 3 of node 5 has already been visited. Thus, the graph contains a cycle that goes through node 3, for example, $3 \rightarrow 2 \rightarrow 5 \rightarrow 3$.

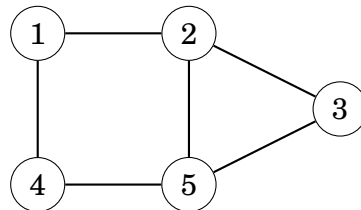
Another way to find out whether a graph contains a cycle is to simply calculate the number of nodes and edges in every component. If a component contains c nodes and no cycle, it must contain exactly $c - 1$ edges (so it has to be a tree). If there are c or more edges, the component surely contains a cycle.

Bipartiteness check

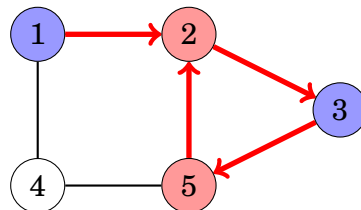
A graph is bipartite if its nodes can be colored using two colors so that there are no adjacent nodes with the same color. It is surprisingly easy to check if a graph is bipartite using graph traversal algorithms.

The idea is to color the starting node blue, all its neighbors red, all their neighbors blue, and so on. If at some point of the search we notice that two adjacent nodes have the same color, this means that the graph is not bipartite. Otherwise the graph is bipartite and one coloring has been found.

For example, the graph



is not bipartite, because a search from node 1 proceeds as follows:



We notice that the color of both nodes 2 and 5 is red, while they are adjacent nodes in the graph. Thus, the graph is not bipartite.

This algorithm always works, because when there are only two colors available, the color of the starting node in a component determines the colors of all other nodes in the component. It does not make any difference whether the starting node is red or blue.

Note that in the general case, it is difficult to find out if the nodes in a graph can be colored using k colors so that no adjacent nodes have the same color. Even when $k = 3$, no efficient algorithm is known but the problem is NP-hard.

Capítulo 13

Shortest paths

Finding a shortest path between two nodes of a graph is an important problem that has many practical applications. For example, a natural problem related to a road network is to calculate the shortest possible length of a route between two cities, given the lengths of the roads.

In an unweighted graph, the length of a path equals the number of its edges, and we can simply use breadth-first search to find a shortest path. However, in this chapter we focus on weighted graphs where more sophisticated algorithms are needed for finding shortest paths.

13.1. Bellman–Ford algorithm

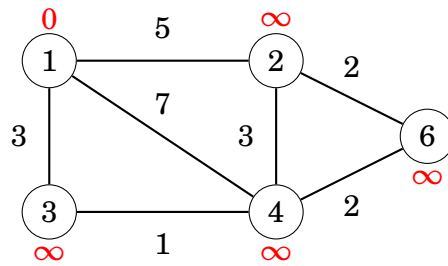
The **Bellman–Ford algorithm**¹ finds shortest paths from a starting node to all nodes of the graph. The algorithm can process all kinds of graphs, provided that the graph does not contain a cycle with negative length. If the graph contains a negative cycle, the algorithm can detect this.

The algorithm keeps track of distances from the starting node to all nodes of the graph. Initially, the distance to the starting node is 0 and the distance to all other nodes is infinite. The algorithm reduces the distances by finding edges that shorten the paths until it is not possible to reduce any distance.

Example

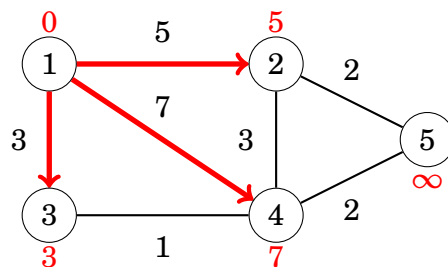
Let us consider how the Bellman–Ford algorithm works in the following graph:

¹The algorithm is named after R. E. Bellman and L. R. Ford who published it independently in 1958 and 1956, respectively [5, 24].

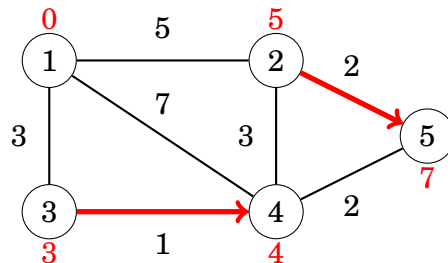


Each node of the graph is assigned a distance. Initially, the distance to the starting node is 0, and the distance to all other nodes is infinite.

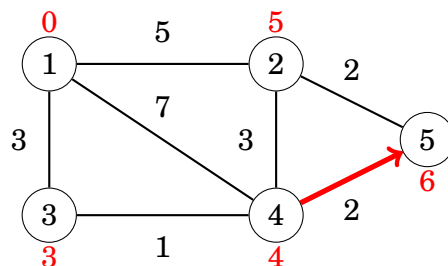
The algorithm searches for edges that reduce distances. First, all edges from node 1 reduce distances:



After this, edges 2 → 5 and 3 → 4 reduce distances:

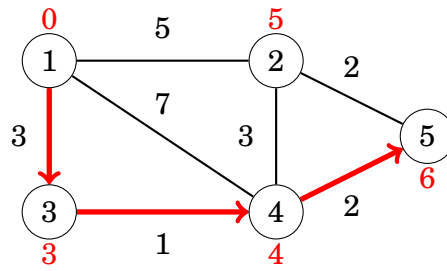


Finally, there is one more change:



After this, no edge can reduce any distance. This means that the distances are final, and we have successfully calculated the shortest distances from the starting node to all nodes of the graph.

For example, the shortest distance 3 from node 1 to node 5 corresponds to the following path:



Implementation

The following implementation of the Bellman–Ford algorithm determines the shortest distances from a node x to all nodes of the graph. The code assumes that the graph is stored as an edge list edges that consists of tuples of the form (a, b, w) , meaning that there is an edge from node a to node b with weight w .

The algorithm consists of $n - 1$ rounds, and on each round the algorithm goes through all edges of the graph and tries to reduce the distances. The algorithm constructs an array distance that will contain the distances from x to all nodes of the graph. The constant INF denotes an infinite distance.

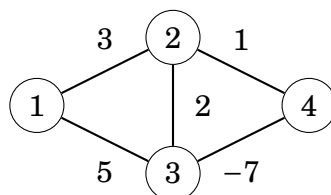
```
for (int i = 1; i <= n; i++) distance[i] = INF;
distance[x] = 0;
for (int i = 1; i <= n-1; i++) {
    for (auto e : edges) {
        int a, b, w;
        tie(a, b, w) = e;
        distance[b] = min(distance[b], distance[a]+w);
    }
}
```

The time complexity of the algorithm is $O(nm)$, because the algorithm consists of $n - 1$ rounds and iterates through all m edges during a round. If there are no negative cycles in the graph, all distances are final after $n - 1$ rounds, because each shortest path can contain at most $n - 1$ edges.

In practice, the final distances can usually be found faster than in $n - 1$ rounds. Thus, a possible way to make the algorithm more efficient is to stop the algorithm if no distance can be reduced during a round.

Negative cycles

The Bellman–Ford algorithm can also be used to check if the graph contains a cycle with negative length. For example, the graph



contains a negative cycle $2 \rightarrow 3 \rightarrow 4 \rightarrow 2$ with length -4 .

If the graph contains a negative cycle, we can shorten infinitely many times any path that contains the cycle by repeating the cycle again and again. Thus, the concept of a shortest path is not meaningful in this situation.

A negative cycle can be detected using the Bellman–Ford algorithm by running the algorithm for n rounds. If the last round reduces any distance, the graph contains a negative cycle. Note that this algorithm can be used to search for a negative cycle in the whole graph regardless of the starting node.

SPFA algorithm

The **SPFA algorithm** (“Shortest Path Faster Algorithm”) [20] is a variant of the Bellman–Ford algorithm, that is often more efficient than the original algorithm. The SPFA algorithm does not go through all the edges on each round, but instead, it chooses the edges to be examined in a more intelligent way.

The algorithm maintains a queue of nodes that might be used for reducing the distances. First, the algorithm adds the starting node x to the queue. Then, the algorithm always processes the first node in the queue, and when an edge $a \rightarrow b$ reduces a distance, node b is added to the queue.

The efficiency of the SPFA algorithm depends on the structure of the graph: the algorithm is often efficient, but its worst case time complexity is still $O(nm)$ and it is possible to create inputs that make the algorithm as slow as the original Bellman–Ford algorithm.

13.2. Dijkstra’s algorithm

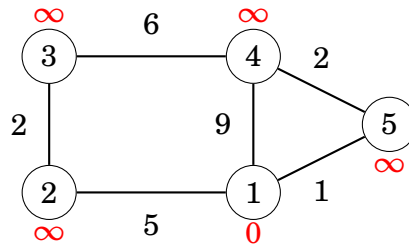
Dijkstra’s algorithm² finds shortest paths from the starting node to all nodes of the graph, like the Bellman–Ford algorithm. The benefit of Dijkstra’s algorithm is that it is more efficient and can be used for processing large graphs. However, the algorithm requires that there are no negative weight edges in the graph.

Like the Bellman–Ford algorithm, Dijkstra’s algorithm maintains distances to the nodes and reduces them during the search. Dijkstra’s algorithm is efficient, because it only processes each edge in the graph once, using the fact that there are no negative edges.

Example

Let us consider how Dijkstra’s algorithm works in the following graph when the starting node is node 1:

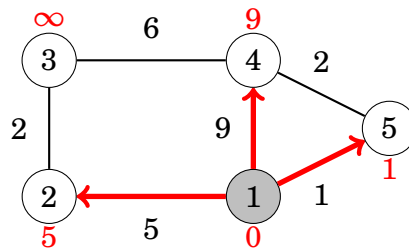
²E. W. Dijkstra published the algorithm in 1959 [14]; however, his original paper does not mention how to implement the algorithm efficiently.



Like in the Bellman–Ford algorithm, initially the distance to the starting node is 0 and the distance to all other nodes is infinite.

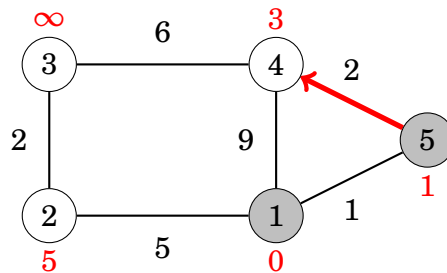
At each step, Dijkstra's algorithm selects a node that has not been processed yet and whose distance is as small as possible. The first such node is node 1 with distance 0.

When a node is selected, the algorithm goes through all edges that start at the node and reduces the distances using them:

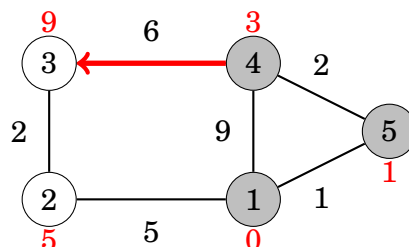


In this case, the edges from node 1 reduced the distances of nodes 2, 4 and 5, whose distances are now 5, 9 and 1.

The next node to be processed is node 5 with distance 1. This reduces the distance to node 4 from 9 to 3:

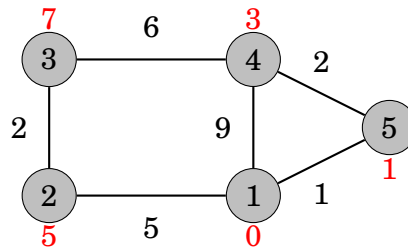


After this, the next node is node 4, which reduces the distance to node 3 to 9:



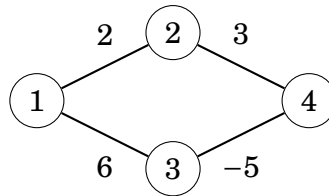
A remarkable property in Dijkstra's algorithm is that whenever a node is selected, its distance is final. For example, at this point of the algorithm, the distances 0, 1 and 3 are the final distances to nodes 1, 5 and 4.

After this, the algorithm processes the two remaining nodes, and the final distances are as follows:



Negative edges

The efficiency of Dijkstra's algorithm is based on the fact that the graph does not contain negative edges. If there is a negative edge, the algorithm may give incorrect results. As an example, consider the following graph:



The shortest path from node 1 to node 4 is $1 \rightarrow 3 \rightarrow 4$ and its length is 1. However, Dijkstra's algorithm finds the path $1 \rightarrow 2 \rightarrow 4$ by following the minimum weight edges. The algorithm does not take into account that on the other path, the weight -5 compensates the previous large weight 6.

Implementation

The following implementation of Dijkstra's algorithm calculates the minimum distances from a node x to other nodes of the graph. The graph is stored as adjacency lists so that $\text{adj}[a]$ contains a pair (b, w) always when there is an edge from node a to node b with weight w .

An efficient implementation of Dijkstra's algorithm requires that it is possible to efficiently find the minimum distance node that has not been processed. An appropriate data structure for this is a priority queue that contains the nodes ordered by their distances. Using a priority queue, the next node to be processed can be retrieved in logarithmic time.

In the following code, the priority queue q contains pairs of the form $(-d, x)$, meaning that the current distance to node x is d . The array distance contains the distance to each node, and the array processed indicates whether a node has been processed. Initially the distance is 0 to x and ∞ to all other nodes.

```
for (int i = 1; i <= n; i++) distance[i] = INF;
distance[x] = 0;
q.push({0, x});
```

```

while (!q.empty()) {
    int a = q.top().second; q.pop();
    if (processed[a]) continue;
    processed[a] = true;
    for (auto u : adj[a]) {
        int b = u.first, w = u.second;
        if (distance[a]+w < distance[b]) {
            distance[b] = distance[a]+w;
            q.push({-distance[b],b});
        }
    }
}

```

Note that the priority queue contains *negative* distances to nodes. The reason for this is that the default version of the C++ priority queue finds maximum elements, while we want to find minimum elements. By using negative distances, we can directly use the default priority queue³. Also note that there may be several instances of the same node in the priority queue; however, only the instance with the minimum distance will be processed.

The time complexity of the above implementation is $O(n + m \log m)$, because the algorithm goes through all nodes of the graph and adds for each edge at most one distance to the priority queue.

13.3. Floyd–Warshall algorithm

The **Floyd–Warshall algorithm**⁴ provides an alternative way to approach the problem of finding shortest paths. Unlike the other algorithms of this chapter, it finds all shortest paths between the nodes in a single run.

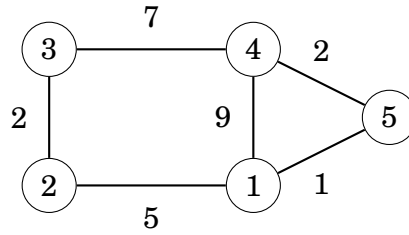
The algorithm maintains a two-dimensional array that contains distances between the nodes. First, distances are calculated only using direct edges between the nodes, and after this, the algorithm reduces distances by using intermediate nodes in paths.

Example

Let us consider how the Floyd–Warshall algorithm works in the following graph:

³Of course, we could also declare the priority queue as in Chapter 4.5 and use positive distances, but the implementation would be a bit longer.

⁴The algorithm is named after R. W. Floyd and S. Warshall who published it independently in 1962 [23, 70].



Initially, the distance from each node to itself is 0, and the distance between nodes a and b is x if there is an edge between nodes a and b with weight x . All other distances are infinite.

In this graph, the initial array is as follows:

	1	2	3	4	5
1	0	5	∞	9	1
2	5	0	2	∞	∞
3	∞	2	0	7	∞
4	9	∞	7	0	2
5	1	∞	∞	2	0

The algorithm consists of consecutive rounds. On each round, the algorithm selects a new node that can act as an intermediate node in paths from now on, and distances are reduced using this node.

On the first round, node 1 is the new intermediate node. There is a new path between nodes 2 and 4 with length 14, because node 1 connects them. There is also a new path between nodes 2 and 5 with length 6.

	1	2	3	4	5
1	0	5	∞	9	1
2	5	0	2	14	6
3	∞	2	0	7	∞
4	9	14	7	0	2
5	1	6	∞	2	0

On the second round, node 2 is the new intermediate node. This creates new paths between nodes 1 and 3 and between nodes 3 and 5:

	1	2	3	4	5
1	0	5	7	9	1
2	5	0	2	14	6
3	7	2	0	7	8
4	9	14	7	0	2
5	1	6	8	2	0

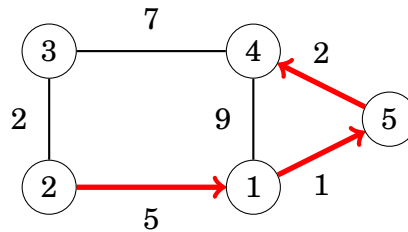
On the third round, node 3 is the new intermediate round. There is a new path between nodes 2 and 4:

	1	2	3	4	5
1	0	5	7	9	1
2	5	0	2	9	6
3	7	2	0	7	8
4	9	9	7	0	2
5	1	6	8	2	0

The algorithm continues like this, until all nodes have been appointed intermediate nodes. After the algorithm has finished, the array contains the minimum distances between any two nodes:

	1	2	3	4	5
1	0	5	7	3	1
2	5	0	2	8	6
3	7	2	0	7	8
4	3	8	7	0	2
5	1	6	8	2	0

For example, the array tells us that the shortest distance between nodes 2 and 4 is 8. This corresponds to the following path:



Implementation

The advantage of the Floyd–Warshall algorithm is that it is easy to implement. The following code constructs a distance matrix where $\text{distance}[a][b]$ is the shortest distance between nodes a and b . First, the algorithm initializes distance using the adjacency matrix adj of the graph:

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        if (i == j) distance[i][j] = 0;
        else if (adj[i][j]) distance[i][j] = adj[i][j];
        else distance[i][j] = INF;
    }
}
```

After this, the shortest distances can be found as follows:

```
for (int k = 1; k <= n; k++) {
    for (int i = 1; i <= n; i++) {
```

```
    for (int j = 1; j <= n; j++) {  
        distance[i][j] = min(distance[i][j],  
                             distance[i][k]+distance[k][j]);  
    }  
}  
}
```

The time complexity of the algorithm is $O(n^3)$, because it contains three nested loops that go through the nodes of the graph.

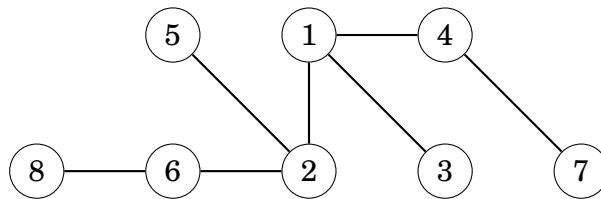
Since the implementation of the Floyd–Warshall algorithm is simple, the algorithm can be a good choice even if it is only needed to find a single shortest path in the graph. However, the algorithm can only be used when the graph is so small that a cubic time complexity is fast enough.

Capítulo 14

Tree algorithms

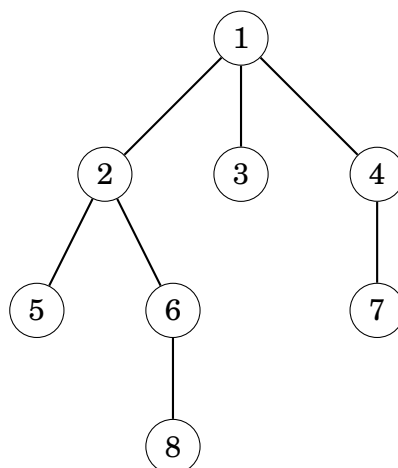
A **tree** is a connected, acyclic graph that consists of n nodes and $n - 1$ edges. Removing any edge from a tree divides it into two components, and adding any edge to a tree creates a cycle. Moreover, there is always a unique path between any two nodes of a tree.

For example, the following tree consists of 8 nodes and 7 edges:



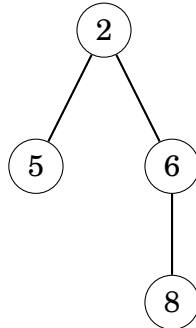
The **leaves** of a tree are the nodes with degree 1, i.e., with only one neighbor. For example, the leaves of the above tree are nodes 3, 5, 7 and 8.

In a **rooted** tree, one of the nodes is appointed the **root** of the tree, and all other nodes are placed underneath the root. For example, in the following tree, node 1 is the root node.



In a rooted tree, the **children** of a node are its lower neighbors, and the **parent** of a node is its upper neighbor. Each node has exactly one parent, except for the root that does not have a parent. For example, in the above tree, the children of node 2 are nodes 5 and 6, and its parent is node 1.

The structure of a rooted tree is *recursive*: each node of the tree acts as the root of a **subtree** that contains the node itself and all nodes that are in the subtrees of its children. For example, in the above tree, the subtree of node 2 consists of nodes 2, 5, 6 and 8:



14.1. Tree traversal

General graph traversal algorithms can be used to traverse the nodes of a tree. However, the traversal of a tree is easier to implement than that of a general graph, because there are no cycles in the tree and it is not possible to reach a node from multiple directions.

The typical way to traverse a tree is to start a depth-first search at an arbitrary node. The following recursive function can be used:

```
void dfs(int s, int e) {  
    // process node s  
    for (auto u : adj[s]) {  
        if (u != e) dfs(u, s);  
    }  
}
```

The function is given two parameters: the current node s and the previous node e . The purpose of the parameter e is to make sure that the search only moves to nodes that have not been visited yet.

The following function call starts the search at node x :

```
dfs(x, 0);
```

In the first call $e = 0$, because there is no previous node, and it is allowed to proceed to any direction in the tree.

Dynamic programming

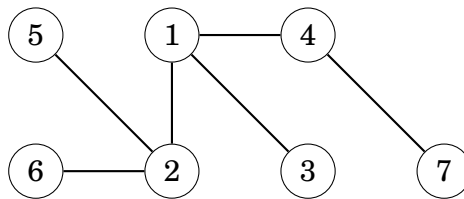
Dynamic programming can be used to calculate some information during a tree traversal. Using dynamic programming, we can, for example, calculate in $O(n)$ time for each node of a rooted tree the number of nodes in its subtree or the length of the longest path from the node to a leaf.

As an example, let us calculate for each node s a value $\text{count}[s]$: the number of nodes in its subtree. The subtree contains the node itself and all nodes in the subtrees of its children, so we can calculate the number of nodes recursively using the following code:

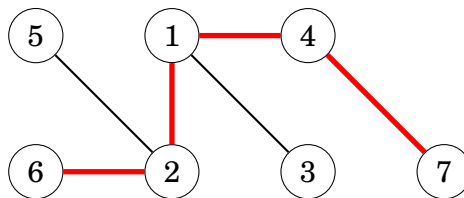
```
void dfs(int s, int e) {
    count[s] = 1;
    for (auto u : adj[s]) {
        if (u == e) continue;
        dfs(u, s);
        count[s] += count[u];
    }
}
```

14.2. Diameter

The **diameter** of a tree is the maximum length of a path between two nodes. For example, consider the following tree:



The diameter of this tree is 4, which corresponds to the following path:



Note that there may be several maximum-length paths. In the above path, we could replace node 6 with node 5 to obtain another path with length 4.

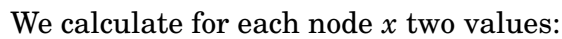
Next we will discuss two $O(n)$ time algorithms for calculating the diameter of a tree. The first algorithm is based on dynamic programming, and the second algorithm uses two depth-first searches.

Algorithm 1

A general way to approach many tree problems is to first root the tree arbitrarily. After this, we can try to solve the problem separately for each subtree. Our first algorithm for calculating the diameter is based on this idea.

An important observation is that every path in a rooted tree has a *highest point*: the highest node that belongs to the path. Thus, we can calculate for each

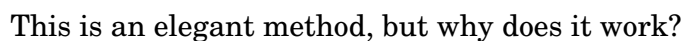
For example, in the following tree, node 1 is the highest point on the path that corresponds to the diameter:



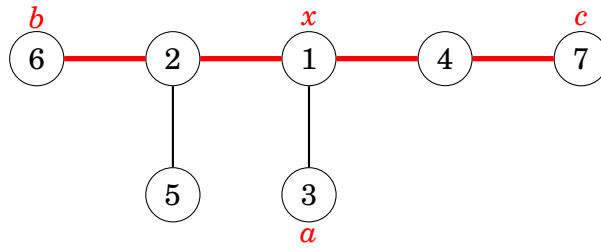
- For example, in the above tree, $\text{toLeaf}(1) = 2$, because there is a path $1 \rightarrow 2 \rightarrow 6$, and $\text{maxLength}(1) = 4$, because there is a path $6 \rightarrow 2 \rightarrow 1 \rightarrow 4 \rightarrow 7$. In this case, $\text{maxLength}(1)$ equals the diameter.

Algorithm 2

In the following graph, a , b and c could be:



144

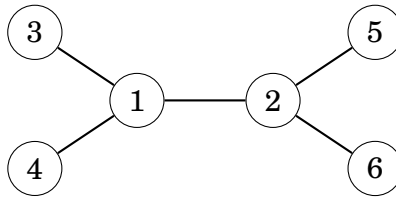


Node x indicates the place where the path from node a joins the path that corresponds to the diameter. The farthest node from a is node b , node c or some other node that is at least as far from node x . Thus, this node is always a valid choice for an endpoint of a path that corresponds to the diameter.

14.3. All longest paths

Our next problem is to calculate for every node in the tree the maximum length of a path that begins at the node. This can be seen as a generalization of the tree diameter problem, because the largest of those lengths equals the diameter of the tree. Also this problem can be solved in $O(n)$ time.

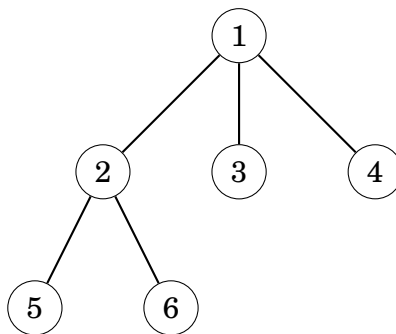
As an example, consider the following tree:



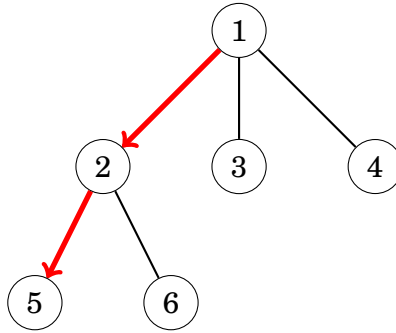
Let $\text{maxLength}(x)$ denote the maximum length of a path that begins at node x . For example, in the above tree, $\text{maxLength}(4) = 3$, because there is a path $4 \rightarrow 1 \rightarrow 2 \rightarrow 6$. Here is a complete table of the values:

node x	1	2	3	4	5	6
$\text{maxLength}(x)$	2	2	3	3	3	3

Also in this problem, a good starting point for solving the problem is to root the tree arbitrarily:

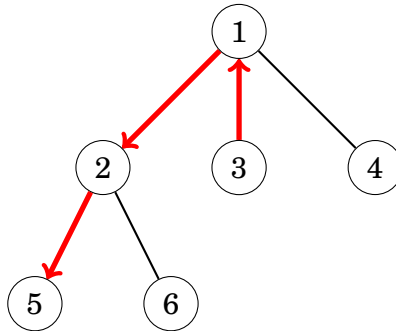


The first part of the problem is to calculate for every node x the maximum length of a path that goes through a child of x . For example, the longest path from node 1 goes through its child 2:

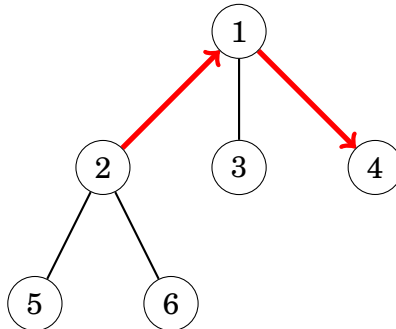


This part is easy to solve in $O(n)$ time, because we can use dynamic programming as we have done previously.

Then, the second part of the problem is to calculate for every node x the maximum length of a path through its parent p . For example, the longest path from node 3 goes through its parent 1:



At first glance, it seems that we should choose the longest path from p . However, this *does not* always work, because the longest path from p may go through x . Here is an example of this situation:



Still, we can solve the second part in $O(n)$ time by storing *two* maximum lengths for each node x :

- $\text{maxLength}_1(x)$: the maximum length of a path from x
- $\text{maxLength}_2(x)$ the maximum length of a path from x in another direction than the first path

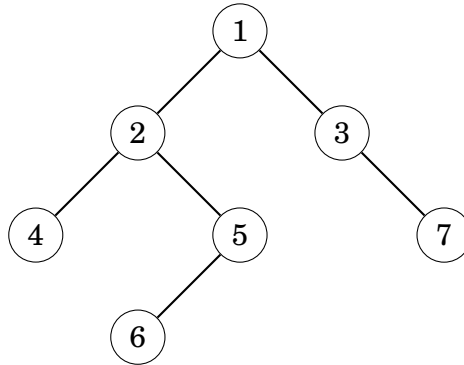
For example, in the above graph, $\text{maxLength}_1(1) = 2$ using the path $1 \rightarrow 2 \rightarrow 5$, and $\text{maxLength}_2(1) = 1$ using the path $1 \rightarrow 3$.

Finally, if the path that corresponds to $\text{maxLength}_1(p)$ goes through x , we conclude that the maximum length is $\text{maxLength}_2(p) + 1$, and otherwise the maximum length is $\text{maxLength}_1(p) + 1$.

14.4. Binary trees

A **binary tree** is a rooted tree where each node has a left and right subtree. It is possible that a subtree of a node is empty. Thus, every node in a binary tree has zero, one or two children.

For example, the following tree is a binary tree:



The nodes of a binary tree have three natural orderings that correspond to different ways to recursively traverse the tree:

- **pre-order**: first process the root, then traverse the left subtree, then traverse the right subtree
- **in-order**: first traverse the left subtree, then process the root, then traverse the right subtree
- **post-order**: first traverse the left subtree, then traverse the right subtree, then process the root

For the above tree, the nodes in pre-order are [1,2,4,5,6,3,7], in in-order [4,2,6,5,1,3,7] and in post-order [4,6,5,2,7,3,1].

If we know the pre-order and in-order of a tree, we can reconstruct the exact structure of the tree. For example, the above tree is the only possible tree with pre-order [1,2,4,5,6,3,7] and in-order [4,2,6,5,1,3,7]. In a similar way, the post-order and in-order also determine the structure of a tree.

However, the situation is different if we only know the pre-order and post-order of a tree. In this case, there may be more than one tree that match the orderings. For example, in both of the trees



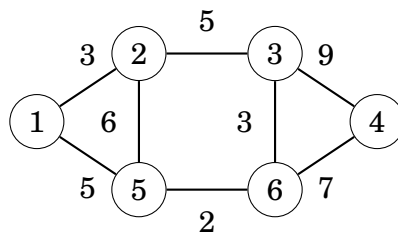
the pre-order is [1,2] and the post-order is [2,1], but the structures of the trees are different.

Capítulo 15

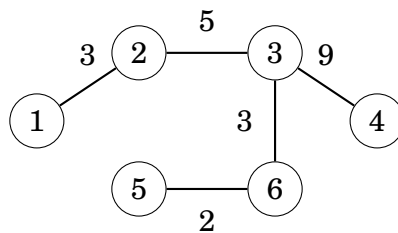
Spanning trees

A **spanning tree** of a graph consists of all nodes of the graph and some of the edges of the graph so that there is a path between any two nodes. Like trees in general, spanning trees are connected and acyclic. Usually there are several ways to construct a spanning tree.

For example, consider the following graph:

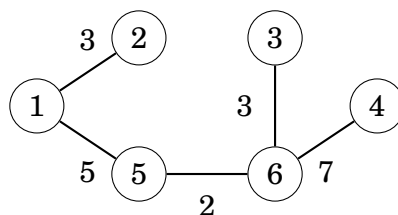


One spanning tree for the graph is as follows:

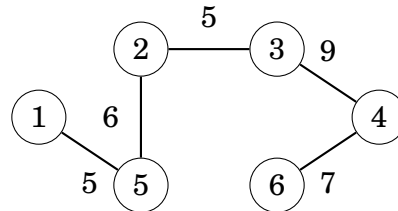


The weight of a spanning tree is the sum of its edge weights. For example, the weight of the above spanning tree is $3 + 5 + 9 + 3 + 2 = 22$.

A **minimum spanning tree** is a spanning tree whose weight is as small as possible. The weight of a minimum spanning tree for the example graph is 20, and such a tree can be constructed as follows:



In a similar way, a **maximum spanning tree** is a spanning tree whose weight is as large as possible. The weight of a maximum spanning tree for the example graph is 32:



Note that a graph may have several minimum and maximum spanning trees, so the trees are not unique.

It turns out that several greedy methods can be used to construct minimum and maximum spanning trees. In this chapter, we discuss two algorithms that process the edges of the graph ordered by their weights. We focus on finding minimum spanning trees, but the same algorithms can find maximum spanning trees by processing the edges in reverse order.

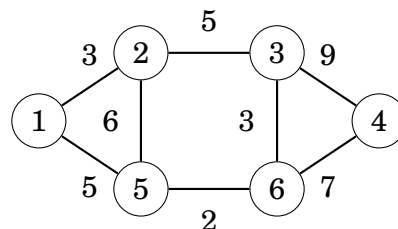
15.1. Kruskal's algorithm

In **Kruskal's algorithm**¹, the initial spanning tree only contains the nodes of the graph and does not contain any edges. Then the algorithm goes through the edges ordered by their weights, and always adds an edge to the tree if it does not create a cycle.

The algorithm maintains the components of the tree. Initially, each node of the graph belongs to a separate component. Always when an edge is added to the tree, two components are joined. Finally, all nodes belong to the same component, and a minimum spanning tree has been found.

Example

Let us consider how Kruskal's algorithm processes the following graph:



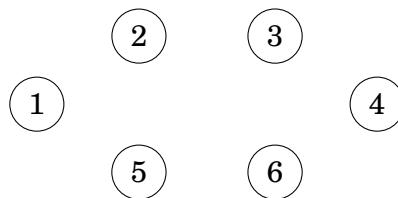
The first step of the algorithm is to sort the edges in increasing order of their weights. The result is the following list:

¹The algorithm was published in 1956 by J. B. Kruskal [48].

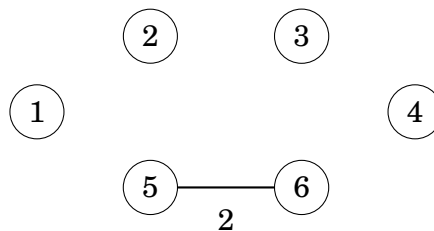
edge	weight
5-6	2
1-2	3
3-6	3
1-5	5
2-3	5
2-5	6
4-6	7
3-4	9

After this, the algorithm goes through the list and adds each edge to the tree if it joins two separate components.

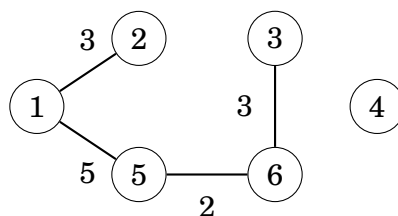
Initially, each node is in its own component:



The first edge to be added to the tree is the edge 5-6 that creates a component {5,6} by joining the components {5} and {6}:



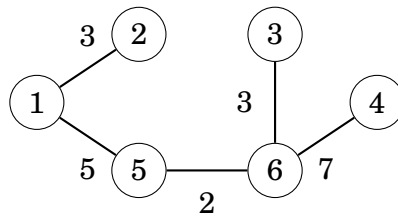
After this, the edges 1-2, 3-6 and 1-5 are added in a similar way:



After those steps, most components have been joined and there are two components in the tree: {1,2,3,5,6} and {4}.

The next edge in the list is the edge 2-3, but it will not be included in the tree, because nodes 2 and 3 are already in the same component. For the same reason, the edge 2-5 will not be included in the tree.

Finally, the edge 4–6 will be included in the tree:

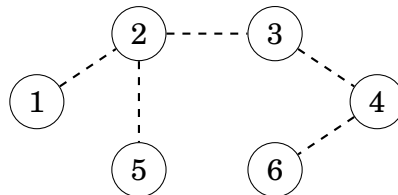


After this, the algorithm will not add any new edges, because the graph is connected and there is a path between any two nodes. The resulting graph is a minimum spanning tree with weight $2 + 3 + 3 + 5 + 7 = 20$.

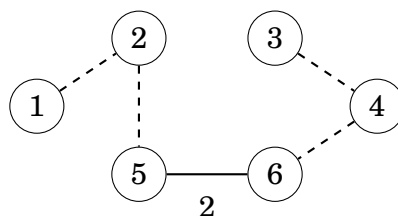
Why does this work?

It is a good question why Kruskal's algorithm works. Why does the greedy strategy guarantee that we will find a minimum spanning tree?

Let us see what happens if the minimum weight edge of the graph is *not* included in the spanning tree. For example, suppose that a spanning tree for the previous graph would not contain the minimum weight edge 5–6. We do not know the exact structure of such a spanning tree, but in any case it has to contain some edges. Assume that the tree would be as follows:



However, it is not possible that the above tree would be a minimum spanning tree for the graph. The reason for this is that we can remove an edge from the tree and replace it with the minimum weight edge 5–6. This produces a spanning tree whose weight is *smaller*:



For this reason, it is always optimal to include the minimum weight edge in the tree to produce a minimum spanning tree. Using a similar argument, we can show that it is also optimal to add the next edge in weight order to the tree, and so on. Hence, Kruskal's algorithm works correctly and always produces a minimum spanning tree.

Implementation

When implementing Kruskal's algorithm, it is convenient to use the edge list representation of the graph. The first phase of the algorithm sorts the edges in the list in $O(m \log m)$ time. After this, the second phase of the algorithm builds the minimum spanning tree as follows:

```
for (...) {  
    if (!same(a,b)) unite(a,b);  
}
```

The loop goes through the edges in the list and always processes an edge $a-b$ where a and b are two nodes. Two functions are needed: the function `same` determines if a and b are in the same component, and the function `unite` joins the components that contain a and b .

The problem is how to efficiently implement the functions `same` and `unite`. One possibility is to implement the function `same` as a graph traversal and check if we can get from node a to node b . However, the time complexity of such a function would be $O(n + m)$ and the resulting algorithm would be slow, because the function `same` will be called for each edge in the graph.

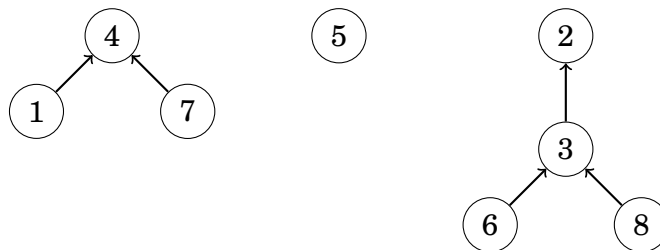
We will solve the problem using a union-find structure that implements both functions in $O(\log n)$ time. Thus, the time complexity of Kruskal's algorithm will be $O(m \log n)$ after sorting the edge list.

15.2. Union-find structure

A **union-find structure** maintains a collection of sets. The sets are disjoint, so no element belongs to more than one set. Two $O(\log n)$ time operations are supported: the `unite` operation joins two sets, and the `find` operation finds the representative of the set that contains a given element².

Structure

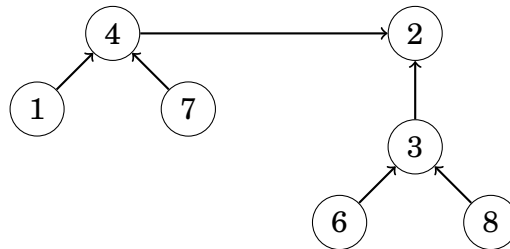
In a union-find structure, one element in each set is the representative of the set, and there is a chain from any other element of the set to the representative. For example, assume that the sets are $\{1, 4, 7\}$, $\{5\}$ and $\{2, 3, 6, 8\}$:



²The structure presented here was introduced in 1971 by J. D. Hopcroft and J. D. Ullman [38]. Later, in 1975, R. E. Tarjan studied a more sophisticated variant of the structure [64] that is discussed in many algorithm textbooks nowadays.

In this case the representatives of the sets are 4, 5 and 2. We can find the representative of any element by following the chain that begins at the element. For example, the element 2 is the representative for the element 6, because we follow the chain $6 \rightarrow 3 \rightarrow 2$. Two elements belong to the same set exactly when their representatives are the same.

Two sets can be joined by connecting the representative of one set to the representative of the other set. For example, the sets $\{1, 4, 7\}$ and $\{2, 3, 6, 8\}$ can be joined as follows:



The resulting set contains the elements $\{1, 2, 3, 4, 6, 7, 8\}$. From this on, the element 2 is the representative for the entire set and the old representative 4 points to the element 2.

The efficiency of the union-find structure depends on how the sets are joined. It turns out that we can follow a simple strategy: always connect the representative of the *smaller* set to the representative of the *larger* set (or if the sets are of equal size, we can make an arbitrary choice). Using this strategy, the length of any chain will be $O(\log n)$, so we can find the representative of any element efficiently by following the corresponding chain.

Implementation

The union-find structure can be implemented using arrays. In the following implementation, the array `link` contains for each element the next element in the chain or the element itself if it is a representative, and the array `size` indicates for each representative the size of the corresponding set.

Initially, each element belongs to a separate set:

```
for (int i = 1; i <= n; i++) link[i] = i;
for (int i = 1; i <= n; i++) size[i] = 1;
```

The function `find` returns the representative for an element x . The representative can be found by following the chain that begins at x .

```
int find(int x) {
    while (x != link[x]) x = link[x];
    return x;
}
```

The function `same` checks whether elements a and b belong to the same set. This can easily be done by using the function `find`:


```

bool same(int a, int b) {
    return find(a) == find(b);
}

```

The function `unite` joins the sets that contain elements a and b (the elements have to be in different sets). The function first finds the representatives of the sets and then connects the smaller set to the larger set.

```

void unite(int a, int b) {
    a = find(a);
    b = find(b);
    if (size[a] < size[b]) swap(a,b);
    size[a] += size[b];
    link[b] = a;
}

```

The time complexity of the function `find` is $O(\log n)$ assuming that the length of each chain is $O(\log n)$. In this case, the functions `same` and `unite` also work in $O(\log n)$ time. The function `unite` makes sure that the length of each chain is $O(\log n)$ by connecting the smaller set to the larger set.

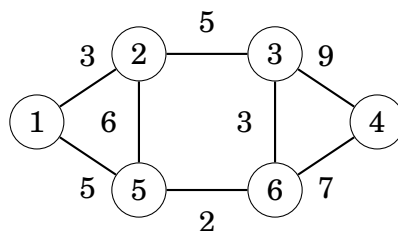
15.3. Prim's algorithm

Prim's algorithm³ is an alternative method for finding a minimum spanning tree. The algorithm first adds an arbitrary node to the tree. After this, the algorithm always chooses a minimum-weight edge that adds a new node to the tree. Finally, all nodes have been added to the tree and a minimum spanning tree has been found.

Prim's algorithm resembles Dijkstra's algorithm. The difference is that Dijkstra's algorithm always selects an edge whose distance from the starting node is minimum, but Prim's algorithm simply selects the minimum weight edge that adds a new node to the tree.

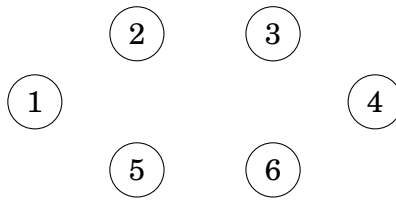
Example

Let us consider how Prim's algorithm works in the following graph:

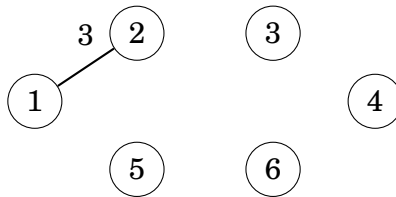


³The algorithm is named after R. C. Prim who published it in 1957 [54]. However, the same algorithm was discovered already in 1930 by V. Jarník.

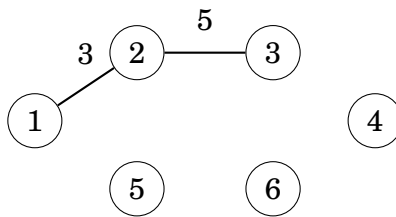
Initially, there are no edges between the nodes:



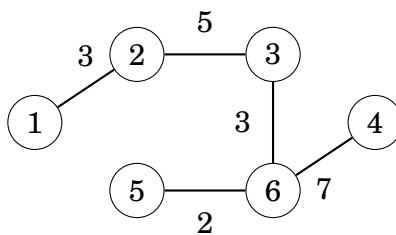
An arbitrary node can be the starting node, so let us choose node 1. First, we add node 2 that is connected by an edge of weight 3:



After this, there are two edges with weight 5, so we can add either node 3 or node 5 to the tree. Let us add node 3 first:



The process continues until all nodes have been included in the tree:



Implementation

Like Dijkstra's algorithm, Prim's algorithm can be efficiently implemented using a priority queue. The priority queue should contain all nodes that can be connected to the current component using a single edge, in increasing order of the weights of the corresponding edges.

The time complexity of Prim's algorithm is $O(n + m \log m)$ that equals the time complexity of Dijkstra's algorithm. In practice, Prim's and Kruskal's algorithms are both efficient, and the choice of the algorithm is a matter of taste. Still, most competitive programmers use Kruskal's algorithm.

Capítulo 16

Directed graphs

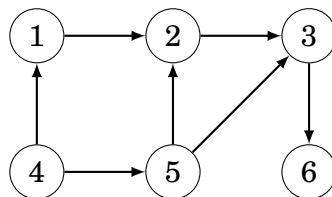
In this chapter, we focus on two classes of directed graphs:

- **Acyclic graphs:** There are no cycles in the graph, so there is no path from any node to itself¹.
- **Successor graphs:** The outdegree of each node is 1, so each node has a unique successor.

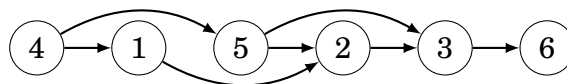
It turns out that in both cases, we can design efficient algorithms that are based on the special properties of the graphs.

16.1. Topological sorting

A **topological sort** is an ordering of the nodes of a directed graph such that if there is a path from node a to node b , then node a appears before node b in the ordering. For example, for the graph



one topological sort is [4, 1, 5, 2, 3, 6]:



An acyclic graph always has a topological sort. However, if the graph contains a cycle, it is not possible to form a topological sort, because no node of the cycle can appear before the other nodes of the cycle in the ordering. It turns out that depth-first search can be used to both check if a directed graph contains a cycle and, if it does not contain a cycle, to construct a topological sort.

¹Directed acyclic graphs are sometimes called DAGs.

Algorithm

The idea is to go through the nodes of the graph and always begin a depth-first search at the current node if it has not been processed yet. During the searches, the nodes have three possible states:

- state 0: the node has not been processed (white)
- state 1: the node is under processing (light gray)
- state 2: the node has been processed (dark gray)

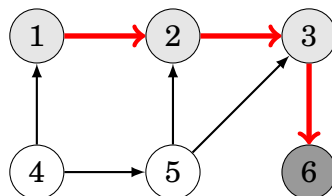
Initially, the state of each node is 0. When a search reaches a node for the first time, its state becomes 1. Finally, after all successors of the node have been processed, its state becomes 2.

If the graph contains a cycle, we will find this out during the search, because sooner or later we will arrive at a node whose state is 1. In this case, it is not possible to construct a topological sort.

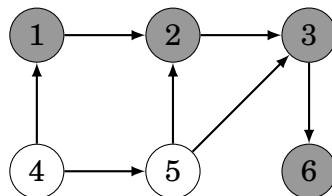
If the graph does not contain a cycle, we can construct a topological sort by adding each node to a list when the state of the node becomes 2. This list in reverse order is a topological sort.

Example 1

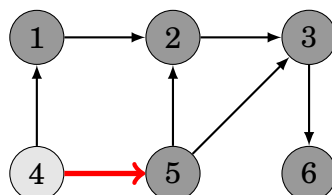
In the example graph, the search first proceeds from node 1 to node 6:



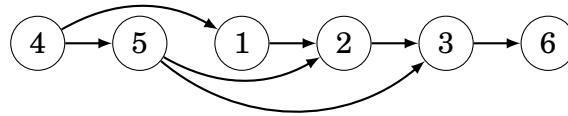
Now node 6 has been processed, so it is added to the list. After this, also nodes 3, 2 and 1 are added to the list:



At this point, the list is [6, 3, 2, 1]. The next search begins at node 4:



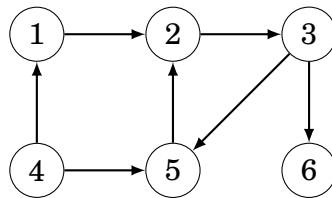
Thus, the final list is [6, 3, 2, 1, 5, 4]. We have processed all nodes, so a topological sort has been found. The topological sort is the reverse list [4, 5, 1, 2, 3, 6]:



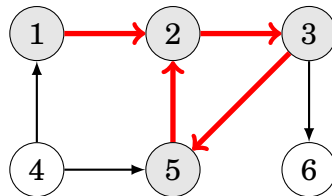
Note that a topological sort is not unique, and there can be several topological sorts for a graph.

Example 2

Let us now consider a graph for which we cannot construct a topological sort, because the graph contains a cycle:



The search proceeds as follows:



The search reaches node 2 whose state is 1, which means that the graph contains a cycle. In this example, there is a cycle $2 \rightarrow 3 \rightarrow 5 \rightarrow 2$.

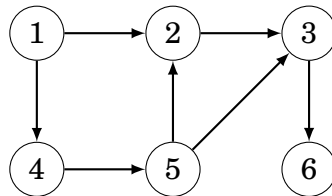
16.2. Dynamic programming

If a directed graph is acyclic, dynamic programming can be applied to it. For example, we can efficiently solve the following problems concerning paths from a starting node to an ending node:

- how many different paths are there?
- what is the shortest/longest path?
- what is the minimum/maximum number of edges in a path?
- which nodes certainly appear in any path?

Counting the number of paths

As an example, let us calculate the number of paths from node 1 to node 6 in the following graph:



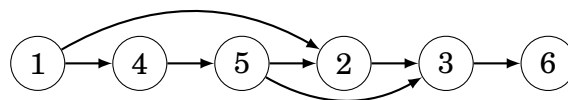
There are a total of three such paths:

- $1 \rightarrow 2 \rightarrow 3 \rightarrow 6$
- $1 \rightarrow 4 \rightarrow 5 \rightarrow 2 \rightarrow 3 \rightarrow 6$
- $1 \rightarrow 4 \rightarrow 5 \rightarrow 3 \rightarrow 6$

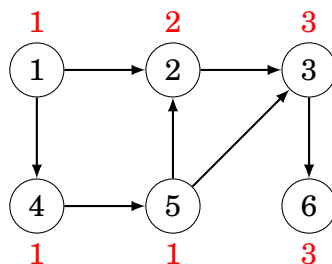
Let $\text{paths}(x)$ denote the number of paths from node 1 to node x . As a base case, $\text{paths}(1) = 1$. Then, to calculate other values of $\text{paths}(x)$, we may use the recursion

$$\text{paths}(x) = \text{paths}(a_1) + \text{paths}(a_2) + \dots + \text{paths}(a_k)$$

where $a_1, a_2, \dots, a_k$ are the nodes from which there is an edge to x . Since the graph is acyclic, the values of $\text{paths}(x)$ can be calculated in the order of a topological sort. A topological sort for the above graph is as follows:



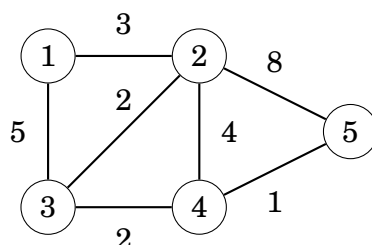
Hence, the numbers of paths are as follows:



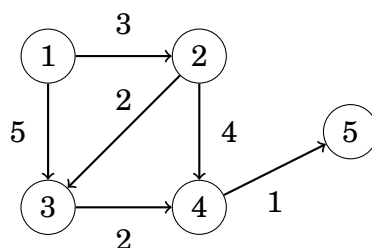
For example, to calculate the value of $\text{paths}(3)$, we can use the formula $\text{paths}(2) + \text{paths}(5)$, because there are edges from nodes 2 and 5 to node 3. Since $\text{paths}(2) = 2$ and $\text{paths}(5) = 1$, we conclude that $\text{paths}(3) = 3$.

Extending Dijkstra's algorithm

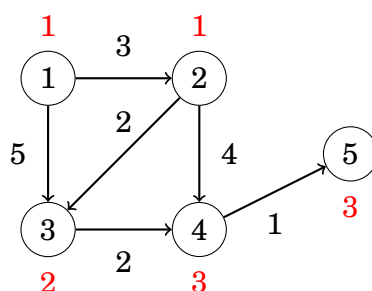
A by-product of Dijkstra's algorithm is a directed, acyclic graph that indicates for each node of the original graph the possible ways to reach the node using a shortest path from the starting node. Dynamic programming can be applied to that graph. For example, in the graph



the shortest paths from node 1 may use the following edges:



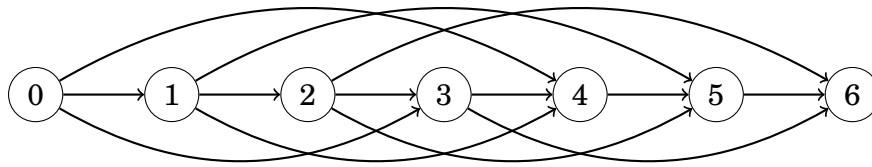
Now we can, for example, calculate the number of shortest paths from node 1 to node 5 using dynamic programming:



Representing problems as graphs

Actually, any dynamic programming problem can be represented as a directed, acyclic graph. In such a graph, each node corresponds to a dynamic programming state and the edges indicate how the states depend on each other.

As an example, consider the problem of forming a sum of money n using coins $\{c_1, c_2, \dots, c_k\}$. In this problem, we can construct a graph where each node corresponds to a sum of money, and the edges show how the coins can be chosen. For example, for coins $\{1, 3, 4\}$ and $n = 6$, the graph is as follows:



Using this representation, the shortest path from node 0 to node n corresponds to a solution with the minimum number of coins, and the total number of paths from node 0 to node n equals the total number of solutions.

16.3. Successor paths

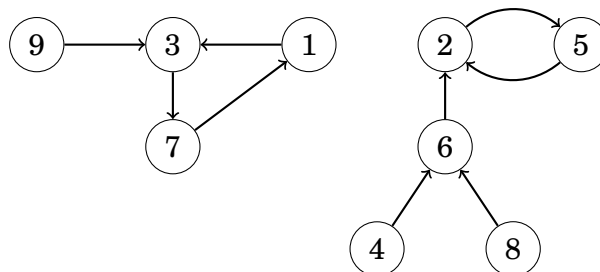
For the rest of the chapter, we will focus on **successor graphs**. In those graphs, the outdegree of each node is 1, i.e., exactly one edge starts at each node. A successor graph consists of one or more components, each of which contains one cycle and some paths that lead to it.

Successor graphs are sometimes called **functional graphs**. The reason for this is that any successor graph corresponds to a function that defines the edges of the graph. The parameter for the function is a node of the graph, and the function gives the successor of that node.

For example, the function

x	1	2	3	4	5	6	7	8	9
$\text{succ}(x)$	3	5	7	6	2	2	1	6	3

defines the following graph:



Since each node of a successor graph has a unique successor, we can also define a function $\text{succ}(x, k)$ that gives the node that we will reach if we begin at node x and walk k steps forward. For example, in the above graph $\text{succ}(4, 6) = 2$, because we will reach node 2 by walking 6 steps from node 4:



A straightforward way to calculate a value of $\text{succ}(x, k)$ is to start at node x and walk k steps forward, which takes $O(k)$ time. However, using preprocessing, any value of $\text{succ}(x, k)$ can be calculated in only $O(\log k)$ time.

The idea is to precalculate all values of $\text{succ}(x, k)$ where k is a power of two and at most u , where u is the maximum number of steps we will ever walk. This can be efficiently done, because we can use the following recursion:

$$\text{succ}(x, k) = \begin{cases} \text{succ}(x) & k = 1 \\ \text{succ}(\text{succ}(x, k/2), k/2) & k > 1 \end{cases}$$

Precalculating the values takes $O(n \log u)$ time, because $O(\log u)$ values are calculated for each node. In the above graph, the first values are as follows:

x	1	2	3	4	5	6	7	8	9
$\text{succ}(x, 1)$	3	5	7	6	2	2	1	6	3
$\text{succ}(x, 2)$	7	2	1	2	5	5	3	2	7
$\text{succ}(x, 4)$	3	2	7	2	5	5	1	2	3
$\text{succ}(x, 8)$	7	2	1	2	5	5	3	2	7
...									

After this, any value of $\text{succ}(x, k)$ can be calculated by presenting the number of steps k as a sum of powers of two. For example, if we want to calculate the value of $\text{succ}(x, 11)$, we first form the representation $11 = 8 + 2 + 1$. Using that,

$$\text{succ}(x, 11) = \text{succ}(\text{succ}(\text{succ}(x, 8), 2), 1).$$

For example, in the previous graph

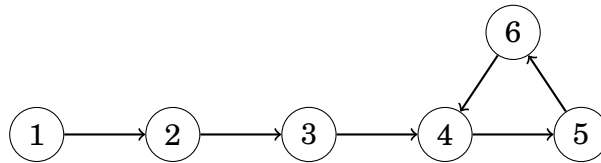
$$\text{succ}(4, 11) = \text{succ}(\text{succ}(\text{succ}(4, 8), 2), 1) = 5.$$

Such a representation always consists of $O(\log k)$ parts, so calculating a value of $\text{succ}(x, k)$ takes $O(\log k)$ time.

16.4. Cycle detection

Consider a successor graph that only contains a path that ends in a cycle. We may ask the following questions: if we begin our walk at the starting node, what is the first node in the cycle and how many nodes does the cycle contain?

For example, in the graph



we begin our walk at node 1, the first node that belongs to the cycle is node 4, and the cycle consists of three nodes (4, 5 and 6).

A simple way to detect the cycle is to walk in the graph and keep track of all nodes that have been visited. Once a node is visited for the second time, we can conclude that the node is the first node in the cycle. This method works in $O(n)$ time and also uses $O(n)$ memory.

However, there are better algorithms for cycle detection. The time complexity of such algorithms is still $O(n)$, but they only use $O(1)$ memory. This is an important improvement if n is large. Next we will discuss Floyd's algorithm that achieves these properties.

Floyd's algorithm

Floyd's algorithm² walks forward in the graph using two pointers a and b . Both pointers begin at a node x that is the starting node of the graph. Then, on each turn, the pointer a walks one step forward and the pointer b walks two steps forward. The process continues until the pointers meet each other:

```
a = succ(x);
b = succ(succ(x));
while (a != b) {
    a = succ(a);
    b = succ(succ(b));
}
```

At this point, the pointer a has walked k steps and the pointer b has walked $2k$ steps, so the length of the cycle divides k . Thus, the first node that belongs to the cycle can be found by moving the pointer a to node x and advancing the pointers step by step until they meet again.

```
a = x;
while (a != b) {
    a = succ(a);
    b = succ(b);
}
first = a;
```

After this, the length of the cycle can be calculated as follows:

```
b = succ(a);
length = 1;
while (a != b) {
    b = succ(b);
    length++;
}
```

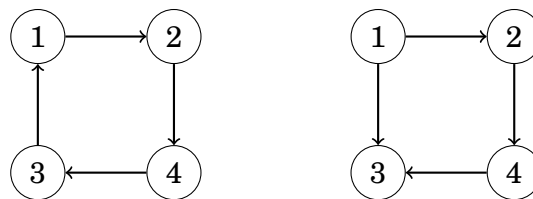
²The idea of the algorithm is mentioned in [46] and attributed to R. W. Floyd; however, it is not known if Floyd actually discovered the algorithm.

Capítulo 17

Strong connectivity

In a directed graph, the edges can be traversed in one direction only, so even if the graph is connected, this does not guarantee that there would be a path from a node to another node. For this reason, it is meaningful to define a new concept that requires more than connectivity.

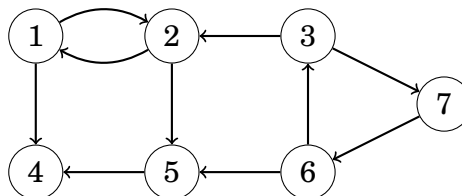
A graph is **strongly connected** if there is a path from any node to all other nodes in the graph. For example, in the following picture, the left graph is strongly connected while the right graph is not.



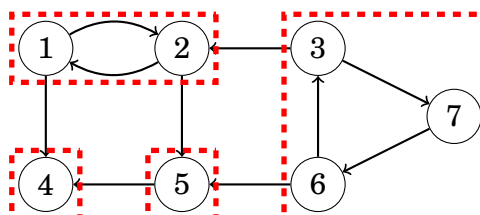
The right graph is not strongly connected because, for example, there is no path from node 2 to node 1.

The **strongly connected components** of a graph divide the graph into strongly connected parts that are as large as possible. The strongly connected components form an acyclic **component graph** that represents the deep structure of the original graph.

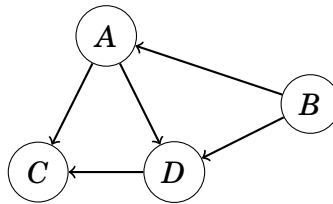
For example, for the graph



the strongly connected components are as follows:



The corresponding component graph is as follows:



The components are $A = \{1, 2\}$, $B = \{3, 6, 7\}$, $C = \{4\}$ and $D = \{5\}$.

A component graph is an acyclic, directed graph, so it is easier to process than the original graph. Since the graph does not contain cycles, we can always construct a topological sort and use dynamic programming techniques like those presented in Chapter 16.

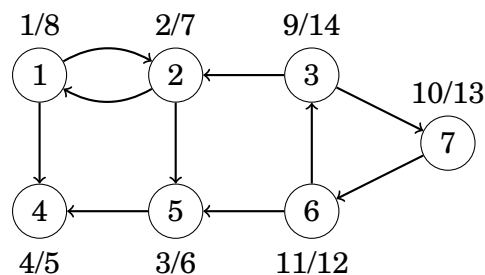
17.1. Kosaraju's algorithm

Kosaraju's algorithm¹ is an efficient method for finding the strongly connected components of a directed graph. The algorithm performs two depth-first searches: the first search constructs a list of nodes according to the structure of the graph, and the second search forms the strongly connected components.

Search 1

The first phase of Kosaraju's algorithm constructs a list of nodes in the order in which a depth-first search processes them. The algorithm goes through the nodes, and begins a depth-first search at each unprocessed node. Each node will be added to the list after it has been processed.

In the example graph, the nodes are processed in the following order:



The notation x/y means that processing the node started at time x and finished at time y . Thus, the corresponding list is as follows:

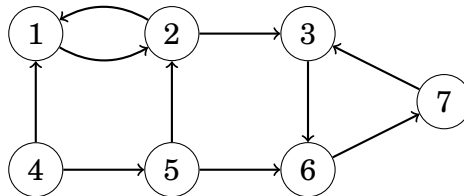
¹According to [1], S. R. Kosaraju invented this algorithm in 1978 but did not publish it. In 1981, the same algorithm was rediscovered and published by M. Sharir [57].

node	processing time
4	5
5	6
2	7
1	8
6	12
7	13
3	14

Search 2

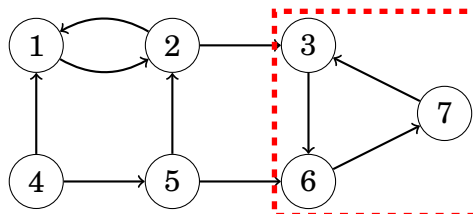
The second phase of the algorithm forms the strongly connected components of the graph. First, the algorithm reverses every edge in the graph. This guarantees that during the second search, we will always find strongly connected components that do not have extra nodes.

After reversing the edges, the example graph is as follows:



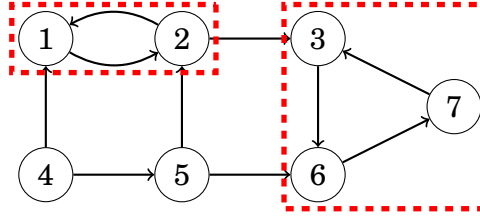
After this, the algorithm goes through the list of nodes created by the first search, in *reverse* order. If a node does not belong to a component, the algorithm creates a new component and starts a depth-first search that adds all new nodes found during the search to the new component.

In the example graph, the first component begins at node 3:

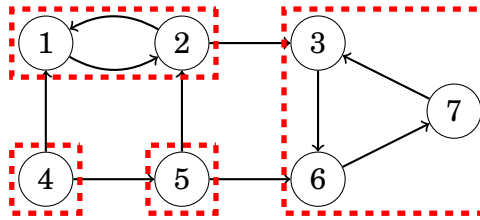


Note that since all edges are reversed, the component does not "leak" to other parts in the graph.

The next nodes in the list are nodes 7 and 6, but they already belong to a component, so the next new component begins at node 1:



Finally, the algorithm processes nodes 5 and 4 that create the remaining strongly connected components:



The time complexity of the algorithm is $O(n + m)$, because the algorithm performs two depth-first searches.

17.2. 2SAT problem

Strong connectivity is also linked with the **2SAT problem**². In this problem, we are given a logical formula

$$(a_1 \vee b_1) \wedge (a_2 \vee b_2) \wedge \cdots \wedge (a_m \vee b_m),$$

where each a_i and b_i is either a logical variable ($x_1, x_2, \dots, x_n$) or a negation of a logical variable ($\neg x_1, \neg x_2, \dots, \neg x_n$). The symbols " $\wedge$ " and " $\vee$ " denote logical operators "and" and "or". Our task is to assign each variable a value so that the formula is true, or state that this is not possible.

For example, the formula

$$L_1 = (x_2 \vee \neg x_1) \wedge (\neg x_1 \vee \neg x_2) \wedge (x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3) \wedge (x_1 \vee x_4)$$

is true when the variables are assigned as follows:

$$\begin{cases} x_1 = \text{false} \\ x_2 = \text{false} \\ x_3 = \text{true} \\ x_4 = \text{true} \end{cases}$$

²The algorithm presented here was introduced in [4]. There is also another well-known linear-time algorithm [19] that is based on backtracking.

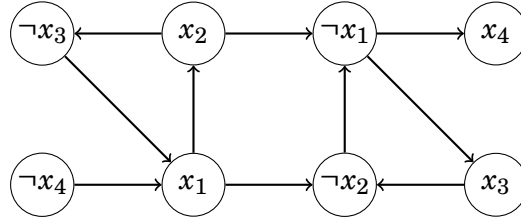
However, the formula

$$L_2 = (x_1 \vee x_2) \wedge (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_1 \vee \neg x_3)$$

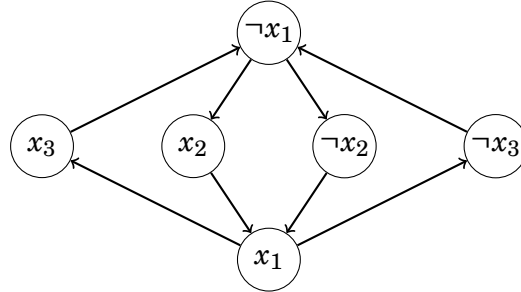
is always false, regardless of how we assign the values. The reason for this is that we cannot choose a value for x_1 without creating a contradiction. If x_1 is false, both x_2 and $\neg x_2$ should be true which is impossible, and if x_1 is true, both x_3 and $\neg x_3$ should be true which is also impossible.

The 2SAT problem can be represented as a graph whose nodes correspond to variables x_i and negations $\neg x_i$, and edges determine the connections between the variables. Each pair $(a_i \vee b_i)$ generates two edges: $\neg a_i \rightarrow b_i$ and $\neg b_i \rightarrow a_i$. This means that if a_i does not hold, b_i must hold, and vice versa.

The graph for the formula L_1 is:



And the graph for the formula L_2 is:



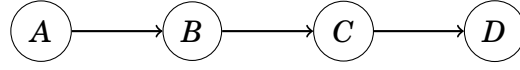
The structure of the graph tells us whether it is possible to assign the values of the variables so that the formula is true. It turns out that this can be done exactly when there are no nodes x_i and $\neg x_i$ such that both nodes belong to the same strongly connected component. If there are such nodes, the graph contains a path from x_i to $\neg x_i$ and also a path from $\neg x_i$ to x_i , so both x_i and $\neg x_i$ should be true which is not possible.

In the graph of the formula L_1 there are no nodes x_i and $\neg x_i$ such that both nodes belong to the same strongly connected component, so a solution exists. In the graph of the formula L_2 all nodes belong to the same strongly connected component, so a solution does not exist.

If a solution exists, the values for the variables can be found by going through the nodes of the component graph in a reverse topological sort order. At each step, we process a component that does not contain edges that lead to an unprocessed component. If the variables in the component have not been assigned values, their values will be determined according to the values in the component, and if

they already have values, they remain unchanged. The process continues until each variable has been assigned a value.

The component graph for the formula L_1 is as follows:



The components are $A = \{\neg x_4\}$, $B = \{x_1, x_2, \neg x_3\}$, $C = \{\neg x_1, \neg x_2, x_3\}$ and $D = \{x_4\}$. When constructing the solution, we first process the component D where x_4 becomes true. After this, we process the component C where x_1 and x_2 become false and x_3 becomes true. All variables have been assigned values, so the remaining components A and B do not change the variables.

Note that this method works, because the graph has a special structure: if there are paths from node x_i to node x_j and from node x_j to node $\neg x_j$, then node x_i never becomes true. The reason for this is that there is also a path from node $\neg x_j$ to node $\neg x_i$, and both x_i and x_j become false.

A more difficult problem is the **3SAT problem**, where each part of the formula is of the form $(a_i \vee b_i \vee c_i)$. This problem is NP-hard, so no efficient algorithm for solving the problem is known.

Capítulo 18

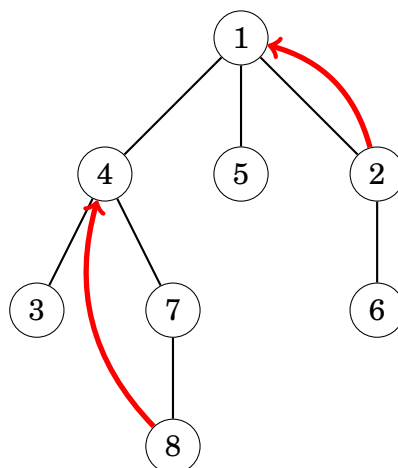
Tree queries

This chapter discusses techniques for processing queries on subtrees and paths of a rooted tree. For example, such queries are:

- what is the k th ancestor of a node?
- what is the sum of values in the subtree of a node?
- what is the sum of values on a path between two nodes?
- what is the lowest common ancestor of two nodes?

18.1. Finding ancestors

The k th **ancestor** of a node x in a rooted tree is the node that we will reach if we move k levels up from x . Let $\text{ancestor}(x, k)$ denote the k th ancestor of a node x (or 0 if there is no such an ancestor). For example, in the following tree, $\text{ancestor}(2, 1) = 1$ and $\text{ancestor}(8, 2) = 4$.



An easy way to calculate any value of $\text{ancestor}(x, k)$ is to perform a sequence of k moves in the tree. However, the time complexity of this method is $O(k)$, which may be slow, because a tree of n nodes may have a chain of n nodes.

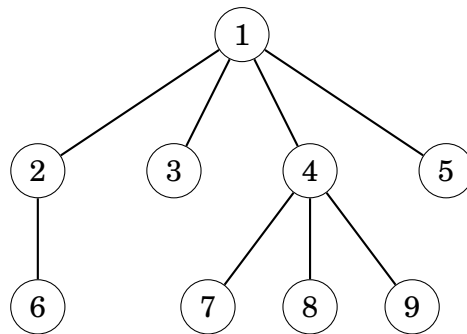
Fortunately, using a technique similar to that used in Chapter 16.3, any value of $\text{ancestor}(x, k)$ can be efficiently calculated in $O(\log k)$ time after preprocessing. The idea is to precalculate all values $\text{ancestor}(x, k)$ where $k \leq n$ is a power of two. For example, the values for the above tree are as follows:

x	1	2	3	4	5	6	7	8
$\text{ancestor}(x, 1)$	0	1	4	1	1	2	4	7
$\text{ancestor}(x, 2)$	0	0	1	0	0	1	1	4
$\text{ancestor}(x, 4)$	0	0	0	0	0	0	0	0
...								

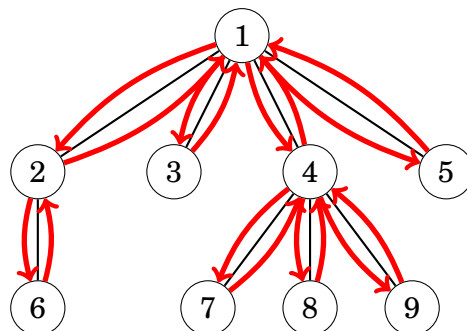
The preprocessing takes $O(n \log n)$ time, because $O(\log n)$ values are calculated for each node. After this, any value of $\text{ancestor}(x, k)$ can be calculated in $O(\log k)$ time by representing k as a sum where each term is a power of two.

18.2. Subtrees and paths

A **tree traversal array** contains the nodes of a rooted tree in the order in which a depth-first search from the root node visits them. For example, in the tree



a depth-first search proceeds as follows:



Hence, the corresponding tree traversal array is as follows:

1	2	6	3	4	7	8	9	5
---	---	---	---	---	---	---	---	---

Subtree queries

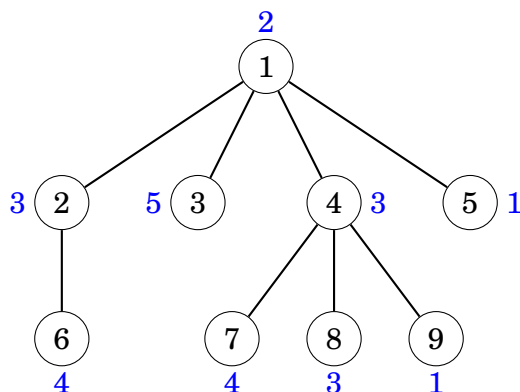
Each subtree of a tree corresponds to a subarray of the tree traversal array such that the first element of the subarray is the root node. For example, the following subarray contains the nodes of the subtree of node 4:

1	2	6	3	4	7	8	9	5
---	---	---	---	---	---	---	---	---

Using this fact, we can efficiently process queries that are related to subtrees of a tree. As an example, consider a problem where each node is assigned a value, and our task is to support the following queries:

- update the value of a node
- calculate the sum of values in the subtree of a node

Consider the following tree where the blue numbers are the values of the nodes. For example, the sum of the subtree of node 4 is $3 + 4 + 3 + 1 = 11$.



The idea is to construct a tree traversal array that contains three values for each node: the identifier of the node, the size of the subtree, and the value of the node. For example, the array for the above tree is as follows:

node id	1	2	6	3	4	7	8	9	5
subtree size	9	2	1	1	4	1	1	1	1
node value	2	3	4	5	3	4	3	1	1

Using this array, we can calculate the sum of values in any subtree by first finding out the size of the subtree and then the values of the corresponding nodes. For example, the values in the subtree of node 4 can be found as follows:

node id	1	2	6	3	4	7	8	9	5
subtree size	9	2	1	1	4	1	1	1	1
node value	2	3	4	5	3	4	3	1	1

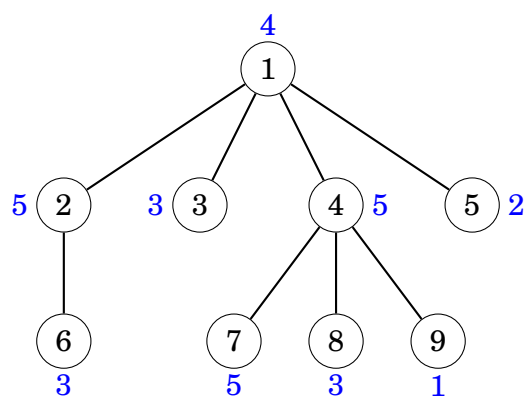
To answer the queries efficiently, it suffices to store the values of the nodes in a binary indexed or segment tree. After this, we can both update a value and calculate the sum of values in $O(\log n)$ time.

Path queries

Using a tree traversal array, we can also efficiently calculate sums of values on paths from the root node to any node of the tree. Consider a problem where our task is to support the following queries:

- change the value of a node
- calculate the sum of values on a path from the root to a node

For example, in the following tree, the sum of values from the root node to node 7 is $4 + 5 + 5 = 14$:



We can solve this problem like before, but now each value in the last row of the array is the sum of values on a path from the root to the node. For example, the following array corresponds to the above tree:

node id	1	2	6	3	4	7	8	9	5
subtree size	9	2	1	1	4	1	1	1	1
path sum	4	9	12	7	9	14	12	10	6

When the value of a node increases by x , the sums of all nodes in its subtree increase by x . For example, if the value of node 4 increases by 1, the array changes as follows:

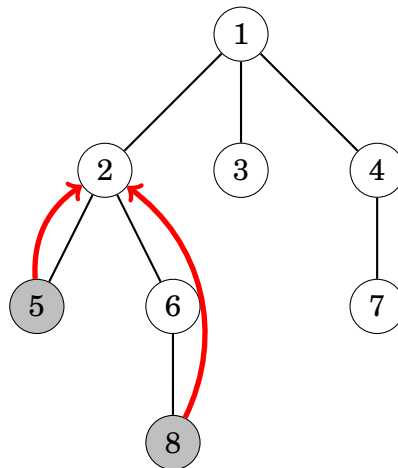
node id	1	2	6	3	4	7	8	9	5
subtree size	9	2	1	1	4	1	1	1	1
path sum	4	9	12	7	10	15	13	11	6

Thus, to support both the operations, we should be able to increase all values in a range and retrieve a single value. This can be done in $O(\log n)$ time using a binary indexed or segment tree (see Chapter 9.4).

18.3. Lowest common ancestor

The **lowest common ancestor** of two nodes of a rooted tree is the lowest node whose subtree contains both the nodes. A typical problem is to efficiently process queries that ask to find the lowest common ancestor of two nodes.

For example, in the following tree, the lowest common ancestor of nodes 5 and 8 is node 2:



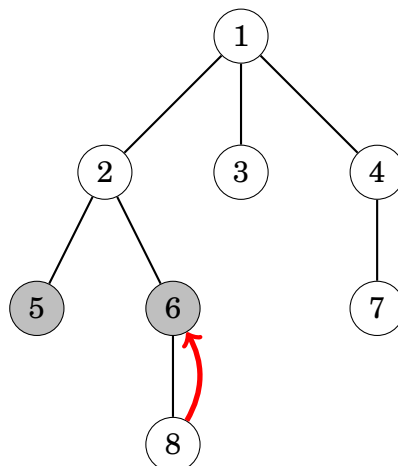
Next we will discuss two efficient techniques for finding the lowest common ancestor of two nodes.

Method 1

One way to solve the problem is to use the fact that we can efficiently find the k th ancestor of any node in the tree. Using this, we can divide the problem of finding the lowest common ancestor into two parts.

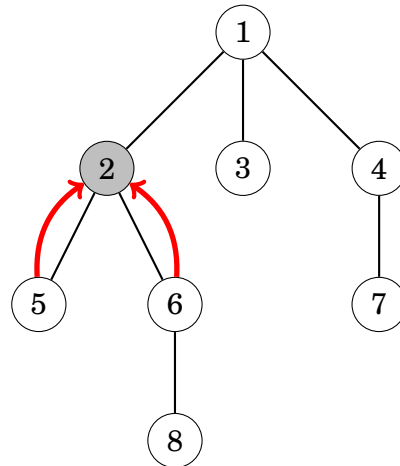
We use two pointers that initially point to the two nodes whose lowest common ancestor we should find. First, we move one of the pointers upwards so that both pointers point to nodes at the same level.

In the example scenario, we move the second pointer one level up so that it points to node 6 which is at the same level with node 5:



After this, we determine the minimum number of steps needed to move both pointers upwards so that they will point to the same node. The node to which the pointers point after this is the lowest common ancestor.

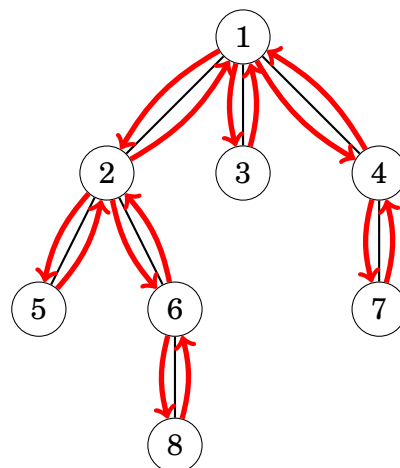
In the example scenario, it suffices to move both pointers one step upwards to node 2, which is the lowest common ancestor:



Since both parts of the algorithm can be performed in $O(\log n)$ time using precomputed information, we can find the lowest common ancestor of any two nodes in $O(\log n)$ time.

Method 2

Another way to solve the problem is based on a tree traversal array¹. Once again, the idea is to traverse the nodes using a depth-first search:



However, we use a different tree traversal array than before: we add each node to the array *always* when the depth-first search walks through the node, and not only at the first visit. Hence, a node that has k children appears $k + 1$ times in the array and there are a total of $2n - 1$ nodes in the array.

¹This lowest common ancestor algorithm was presented in [7]. This technique is sometimes called the **Euler tour technique** [66].

We store two values in the array: the identifier of the node and the depth of the node in the tree. The following array corresponds to the above tree:

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
node id	1	2	5	2	6	8	6	2	1	3	1	4	7	4	1
depth	1	2	3	2	3	4	3	2	1	2	1	2	3	2	1

Now we can find the lowest common ancestor of nodes a and b by finding the node with the *minimum* depth between nodes a and b in the array. For example, the lowest common ancestor of nodes 5 and 8 can be found as follows:

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
node id	1	2	5	2	6	8	6	2	1	3	1	4	7	4	1
depth	1	2	3	2	3	4	3	2	1	2	1	2	3	2	1

↑

Node 5 is at position 2, node 8 is at position 5, and the node with minimum depth between positions 2...5 is node 2 at position 3 whose depth is 2. Thus, the lowest common ancestor of nodes 5 and 8 is node 2.

Thus, to find the lowest common ancestor of two nodes it suffices to process a range minimum query. Since the array is static, we can process such queries in $O(1)$ time after an $O(n \log n)$ time preprocessing.

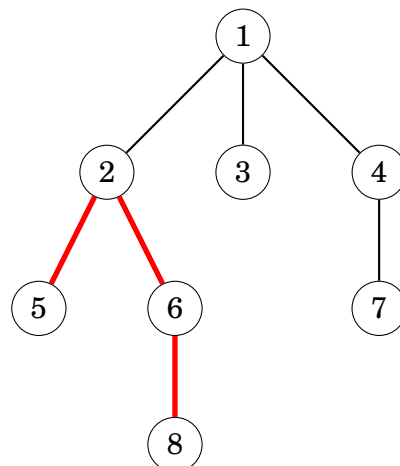
Distances of nodes

The distance between nodes a and b equals the length of the path from a to b . It turns out that the problem of calculating the distance between nodes reduces to finding their lowest common ancestor.

First, we root the tree arbitrarily. After this, the distance of nodes a and b can be calculated using the formula

$$\text{depth}(a) + \text{depth}(b) - 2 \cdot \text{depth}(c),$$

where c is the lowest common ancestor of a and b and $\text{depth}(s)$ denotes the depth of node s . For example, consider the distance of nodes 5 and 8:



The lowest common ancestor of nodes 5 and 8 is node 2. The depths of the nodes are $\text{depth}(5) = 3$, $\text{depth}(8) = 4$ and $\text{depth}(2) = 2$, so the distance between nodes 5 and 8 is $3 + 4 - 2 \cdot 2 = 3$.

18.4. Offline algorithms

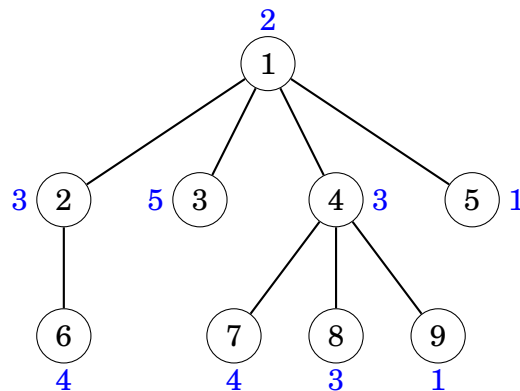
So far, we have discussed *online* algorithms for tree queries. Those algorithms are able to process queries one after another so that each query is answered before receiving the next query.

However, in many problems, the online property is not necessary. In this section, we focus on *offline* algorithms. Those algorithms are given a set of queries which can be answered in any order. It is often easier to design an offline algorithm compared to an online algorithm.

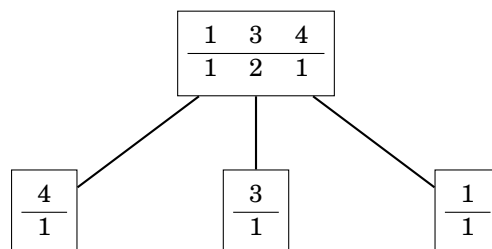
Merging data structures

One method to construct an offline algorithm is to perform a depth-first tree traversal and maintain data structures in nodes. At each node s , we create a data structure $d[s]$ that is based on the data structures of the children of s . Then, using this data structure, all queries related to s are processed.

As an example, consider the following problem: We are given a tree where each node has some value. Our task is to process queries of the form "calculate the number of nodes with value x in the subtree of node s ". For example, in the following tree, the subtree of node 4 contains two nodes whose value is 3.



In this problem, we can use map structures to answer the queries. For example, the maps for node 4 and its children are as follows:



If we create such a data structure for each node, we can easily process all given queries, because we can handle all queries related to a node immediately after creating its data structure. For example, the above map structure for node 4 tells us that its subtree contains two nodes whose value is 3.

However, it would be too slow to create all data structures from scratch. Instead, at each node s , we create an initial data structure $d[s]$ that only contains the value of s . After this, we go through the children of s and *merge* $d[s]$ and all data structures $d[u]$ where u is a child of s .

For example, in the above tree, the map for node 4 is created by merging the following maps:

$\frac{3}{1}$	$\frac{4}{1}$	$\frac{3}{1}$	$\frac{1}{1}$
---------------	---------------	---------------	---------------

Here the first map is the initial data structure for node 4, and the other three maps correspond to nodes 7, 8 and 9.

The merging at node s can be done as follows: We go through the children of s and at each child u merge $d[s]$ and $d[u]$. We always copy the contents from $d[u]$ to $d[s]$. However, before this, we *swap* the contents of $d[s]$ and $d[u]$ if $d[s]$ is smaller than $d[u]$. By doing this, each value is copied only $O(\log n)$ times during the tree traversal, which ensures that the algorithm is efficient.

To swap the contents of two data structures a and b efficiently, we can just use the following code:

```
swap(a,b);
```

It is guaranteed that the above code works in constant time when a and b are C++ standard library data structures.

Lowest common ancestors

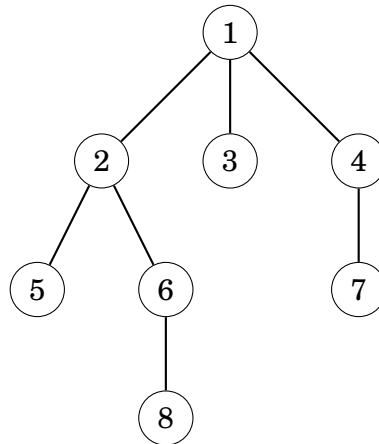
There is also an offline algorithm for processing a set of lowest common ancestor queries². The algorithm is based on the union-find data structure (see Chapter 15.2), and the benefit of the algorithm is that it is easier to implement than the algorithms discussed earlier in this chapter.

The algorithm is given as input a set of pairs of nodes, and it determines for each such pair the lowest common ancestor of the nodes. The algorithm performs a depth-first tree traversal and maintains disjoint sets of nodes. Initially, each node belongs to a separate set. For each set, we also store the highest node in the tree that belongs to the set.

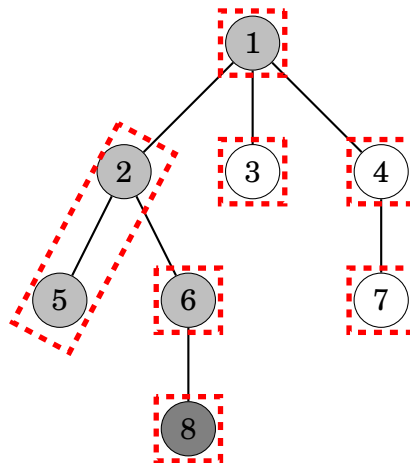
When the algorithm visits a node x , it goes through all nodes y such that the lowest common ancestor of x and y has to be found. If y has already been visited, the algorithm reports that the lowest common ancestor of x and y is the highest node in the set of y . Then, after processing node x , the algorithm joins the sets of x and its parent.

²This algorithm was published by R. E. Tarjan in 1979 [65].

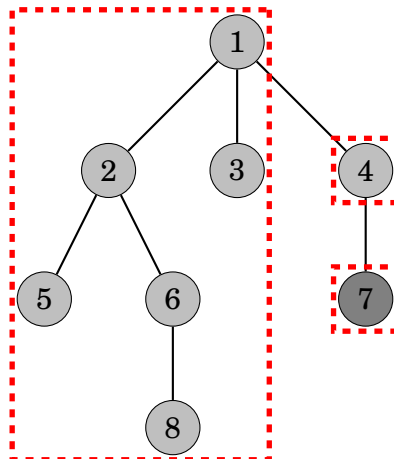
For example, suppose that we want to find the lowest common ancestors of node pairs (5,8) and (2,7) in the following tree:



In the following trees, gray nodes denote visited nodes and dashed groups of nodes belong to the same set. When the algorithm visits node 8, it notices that node 5 has been visited and the highest node in its set is 2. Thus, the lowest common ancestor of nodes 5 and 8 is 2:



Later, when visiting node 7, the algorithm determines that the lowest common ancestor of nodes 2 and 7 is 1:



Capítulo 19

Paths and circuits

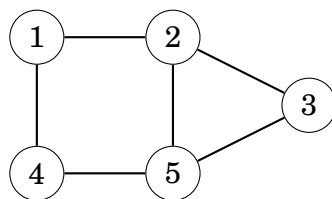
This chapter focuses on two types of paths in graphs:

- An **Eulerian path** is a path that goes through each edge exactly once.
- A **Hamiltonian path** is a path that visits each node exactly once.

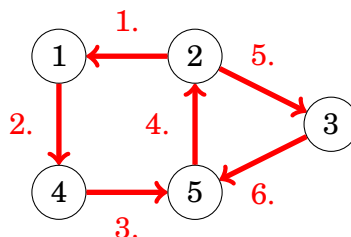
While Eulerian and Hamiltonian paths look like similar concepts at first glance, the computational problems related to them are very different. It turns out that there is a simple rule that determines whether a graph contains an Eulerian path, and there is also an efficient algorithm to find such a path if it exists. On the contrary, checking the existence of a Hamiltonian path is a NP-hard problem, and no efficient algorithm is known for solving the problem.

19.1. Eulerian paths

An **Eulerian path**¹ is a path that goes exactly once through each edge of the graph. For example, the graph

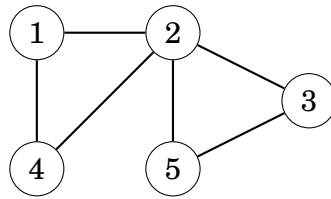


has an Eulerian path from node 2 to node 5:

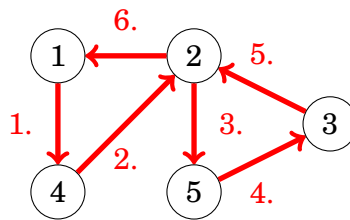


¹L. Euler studied such paths in 1736 when he solved the famous Königsberg bridge problem. This was the birth of graph theory.

An **Eulerian circuit** is an Eulerian path that starts and ends at the same node. For example, the graph



has an Eulerian circuit that starts and ends at node 1:



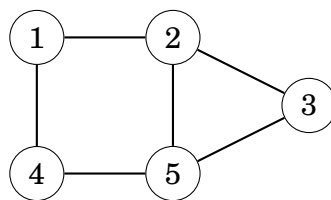
Existence

The existence of Eulerian paths and circuits depends on the degrees of the nodes. First, an undirected graph has an Eulerian path exactly when all the edges belong to the same connected component and

- the degree of each node is even *or*
- the degree of exactly two nodes is odd, and the degree of all other nodes is even.

In the first case, each Eulerian path is also an Eulerian circuit. In the second case, the odd-degree nodes are the starting and ending nodes of an Eulerian path which is not an Eulerian circuit.

For example, in the graph



nodes 1, 3 and 4 have a degree of 2, and nodes 2 and 5 have a degree of 3. Exactly two nodes have an odd degree, so there is an Eulerian path between nodes 2 and 5, but the graph does not contain an Eulerian circuit.

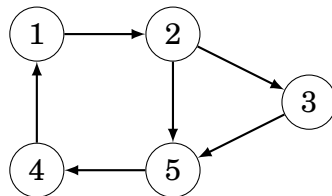
In a directed graph, we focus on indegrees and outdegrees of the nodes. A directed graph contains an Eulerian path exactly when all the edges belong to the same connected component and

- in each node, the indegree equals the outdegree, *or*

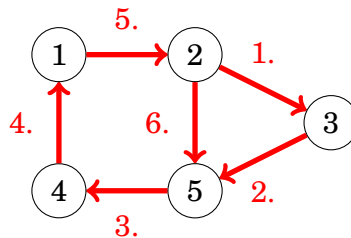
- in one node, the indegree is one larger than the outdegree, in another node, the outdegree is one larger than the indegree, and in all other nodes, the indegree equals the outdegree.

In the first case, each Eulerian path is also an Eulerian circuit, and in the second case, the graph contains an Eulerian path that begins at the node whose outdegree is larger and ends at the node whose indegree is larger.

For example, in the graph



nodes 1, 3 and 4 have both indegree 1 and outdegree 1, node 2 has indegree 1 and outdegree 2, and node 5 has indegree 2 and outdegree 1. Hence, the graph contains an Eulerian path from node 2 to node 5:



Hierholzer's algorithm

Hierholzer's algorithm² is an efficient method for constructing an Eulerian circuit. The algorithm consists of several rounds, each of which adds new edges to the circuit. Of course, we assume that the graph contains an Eulerian circuit; otherwise Hierholzer's algorithm cannot find it.

First, the algorithm constructs a circuit that contains some (not necessarily all) of the edges of the graph. After this, the algorithm extends the circuit step by step by adding subcircuits to it. The process continues until all edges have been added to the circuit.

The algorithm extends the circuit by always finding a node x that belongs to the circuit but has an outgoing edge that is not included in the circuit. The algorithm constructs a new path from node x that only contains edges that are not yet in the circuit. Sooner or later, the path will return to node x , which creates a subcircuit.

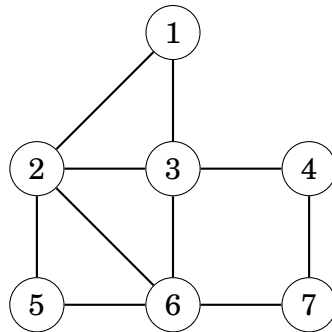
If the graph only contains an Eulerian path, we can still use Hierholzer's algorithm to find it by adding an extra edge to the graph and removing the edge after the circuit has been constructed. For example, in an undirected graph, we add the extra edge between the two odd-degree nodes.

Next we will see how Hierholzer's algorithm constructs an Eulerian circuit for an undirected graph.

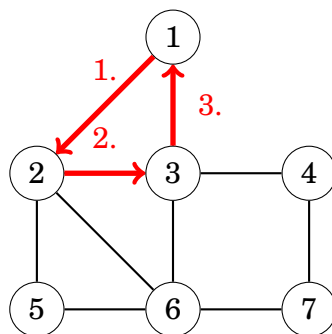
²The algorithm was published in 1873 after Hierholzer's death [35].

Example

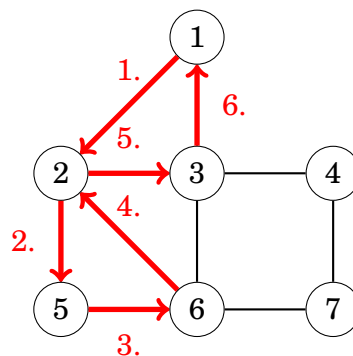
Let us consider the following graph:



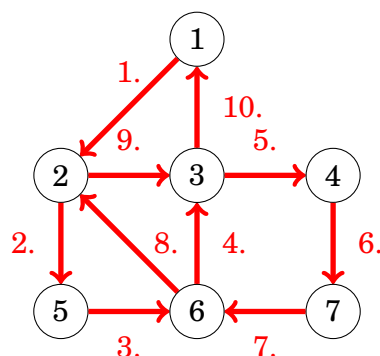
Suppose that the algorithm first creates a circuit that begins at node 1. A possible circuit is $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$:



After this, the algorithm adds the subcircuit $2 \rightarrow 5 \rightarrow 6 \rightarrow 2$ to the circuit:



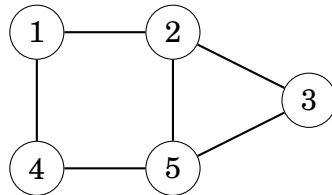
Finally, the algorithm adds the subcircuit $6 \rightarrow 3 \rightarrow 4 \rightarrow 7 \rightarrow 6$ to the circuit:



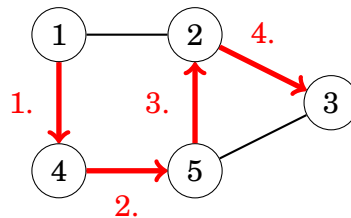
Now all edges are included in the circuit, so we have successfully constructed an Eulerian circuit.

19.2. Hamiltonian paths

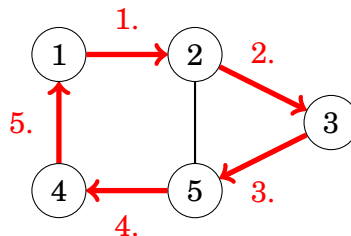
A **Hamiltonian path** is a path that visits each node of the graph exactly once. For example, the graph



contains a Hamiltonian path from node 1 to node 3:



If a Hamiltonian path begins and ends at the same node, it is called a **Hamiltonian circuit**. The graph above also has an Hamiltonian circuit that begins and ends at node 1:



Existence

No efficient method is known for testing if a graph contains a Hamiltonian path, and the problem is NP-hard. Still, in some special cases, we can be certain that a graph contains a Hamiltonian path.

A simple observation is that if the graph is complete, i.e., there is an edge between all pairs of nodes, it also contains a Hamiltonian path. Also stronger results have been achieved:

- **Dirac's theorem:** If the degree of each node is at least $n/2$, the graph contains a Hamiltonian path.
- **Ore's theorem:** If the sum of degrees of each non-adjacent pair of nodes is at least n , the graph contains a Hamiltonian path.

A common property in these theorems and other results is that they guarantee the existence of a Hamiltonian path if the graph has *a large number* of edges. This makes sense, because the more edges the graph contains, the more possibilities there is to construct a Hamiltonian path.

Construction

Since there is no efficient way to check if a Hamiltonian path exists, it is clear that there is also no method to efficiently construct the path, because otherwise we could just try to construct the path and see whether it exists.

A simple way to search for a Hamiltonian path is to use a backtracking algorithm that goes through all possible ways to construct the path. The time complexity of such an algorithm is at least $O(n!)$, because there are $n!$ different ways to choose the order of n nodes.

A more efficient solution is based on dynamic programming (see Chapter 10.5). The idea is to calculate values of a function $\text{possible}(S, x)$, where S is a subset of nodes and x is one of the nodes. The function indicates whether there is a Hamiltonian path that visits the nodes of S and ends at node x . It is possible to implement this solution in $O(2^n n^2)$ time.

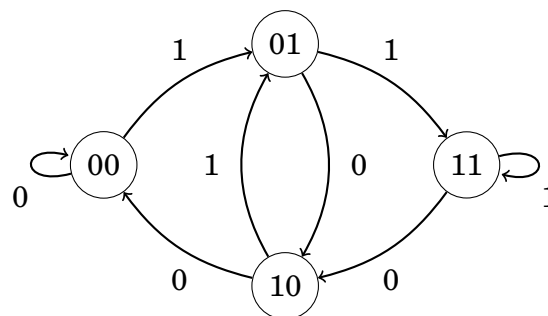
19.3. De Bruijn sequences

A **De Bruijn sequence** is a string that contains every string of length n exactly once as a substring, for a fixed alphabet of k characters. The length of such a string is $k^n + n - 1$ characters. For example, when $n = 3$ and $k = 2$, an example of a De Bruijn sequence is

0001011100.

The substrings of this string are all combinations of three bits: 000, 001, 010, 011, 100, 101, 110 and 111.

It turns out that each De Bruijn sequence corresponds to an Eulerian path in a graph. The idea is to construct a graph where each node contains a string of $n - 1$ characters and each edge adds one character to the string. The following graph corresponds to the above scenario:



An Eulerian path in this graph corresponds to a string that contains all strings of length n . The string contains the characters of the starting node and

all characters of the edges. The starting node has $n - 1$ characters and there are k^n characters in the edges, so the length of the string is $k^n + n - 1$.

19.4. Knight's tours

A **knight's tour** is a sequence of moves of a knight on an $n \times n$ chessboard following the rules of chess such that the knight visits each square exactly once. A knight's tour is called a *closed* tour if the knight finally returns to the starting square and otherwise it is called an *open* tour.

For example, here is an open knight's tour on a 5×5 board:

1	4	11	16	25
12	17	2	5	10
3	20	7	24	15
18	13	22	9	6
21	8	19	14	23

A knight's tour corresponds to a Hamiltonian path in a graph whose nodes represent the squares of the board, and two nodes are connected with an edge if a knight can move between the squares according to the rules of chess.

A natural way to construct a knight's tour is to use backtracking. The search can be made more efficient by using *heuristics* that attempt to guide the knight so that a complete tour will be found quickly.

Warnsdorf's rule

Warnsdorf's rule is a simple and effective heuristic for finding a knight's tour³. Using the rule, it is possible to efficiently construct a tour even on a large board. The idea is to always move the knight so that it ends up in a square where the number of possible moves is as *small* as possible.

For example, in the following situation, there are five possible squares to which the knight can move (squares $a \dots e$):

1				a
		2		
b				e
	c		d	

In this situation, Warnsdorf's rule moves the knight to square a , because after this choice, there is only a single possible move. The other choices would move the knight to squares where there would be three moves available.

³This heuristic was proposed in Warnsdorf's book [69] in 1823. There are also polynomial algorithms for finding knight's tours [52], but they are more complicated.

Capítulo 20

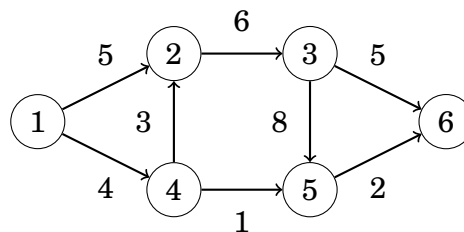
Flows and cuts

In this chapter, we focus on the following two problems:

- **Finding a maximum flow:** What is the maximum amount of flow we can send from a node to another node?
- **Finding a minimum cut:** What is a minimum-weight set of edges that separates two nodes of the graph?

The input for both these problems is a directed, weighted graph that contains two special nodes: the *source* is a node with no incoming edges, and the *sink* is a node with no outgoing edges.

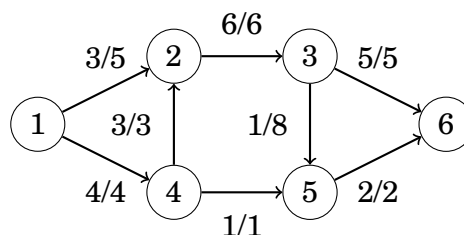
As an example, we will use the following graph where node 1 is the source and node 6 is the sink:



Maximum flow

In the **maximum flow** problem, our task is to send as much flow as possible from the source to the sink. The weight of each edge is a capacity that restricts the flow that can go through the edge. In each intermediate node, the incoming and outgoing flow has to be equal.

For example, the maximum size of a flow in the example graph is 7. The following picture shows how we can route the flow:

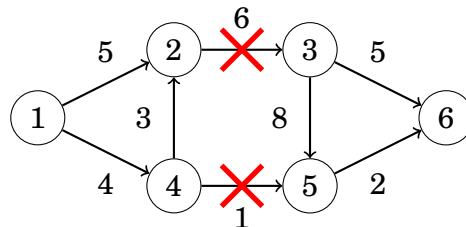


The notation v/k means that a flow of v units is routed through an edge whose capacity is k units. The size of the flow is 7, because the source sends $3 + 4$ units of flow and the sink receives $5 + 2$ units of flow. It is easy to see that this flow is maximum, because the total capacity of the edges leading to the sink is 7.

Minimum cut

In the **minimum cut** problem, our task is to remove a set of edges from the graph such that there will be no path from the source to the sink after the removal and the total weight of the removed edges is minimum.

The minimum size of a cut in the example graph is 7. It suffices to remove the edges $2 \rightarrow 3$ and $4 \rightarrow 5$:



After removing the edges, there will be no path from the source to the sink. The size of the cut is 7, because the weights of the removed edges are 6 and 1. The cut is minimum, because there is no valid way to remove edges from the graph such that their total weight would be less than 7.

It is not a coincidence that the maximum size of a flow and the minimum size of a cut are the same in the above example. It turns out that a maximum flow and a minimum cut are *always* equally large, so the concepts are two sides of the same coin.

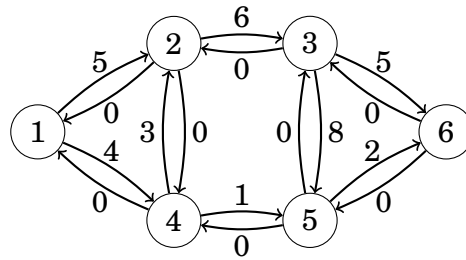
Next we will discuss the Ford–Fulkerson algorithm that can be used to find the maximum flow and minimum cut of a graph. The algorithm also helps us to understand *why* they are equally large.

20.1. Ford–Fulkerson algorithm

The **Ford–Fulkerson algorithm** [25] finds the maximum flow in a graph. The algorithm begins with an empty flow, and at each step finds a path from the source to the sink that generates more flow. Finally, when the algorithm cannot increase the flow anymore, the maximum flow has been found.

The algorithm uses a special representation of the graph where each original edge has a reverse edge in another direction. The weight of each edge indicates how much more flow we could route through it. At the beginning of the algorithm, the weight of each original edge equals the capacity of the edge and the weight of each reverse edge is zero.

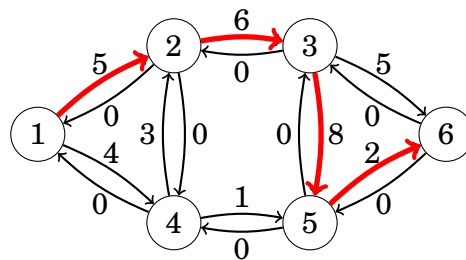
The new representation for the example graph is as follows:



Algorithm description

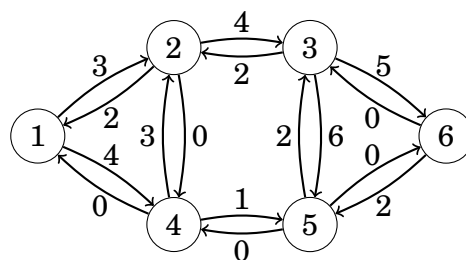
The Ford–Fulkerson algorithm consists of several rounds. On each round, the algorithm finds a path from the source to the sink such that each edge on the path has a positive weight. If there is more than one possible path available, we can choose any of them.

For example, suppose we choose the following path:



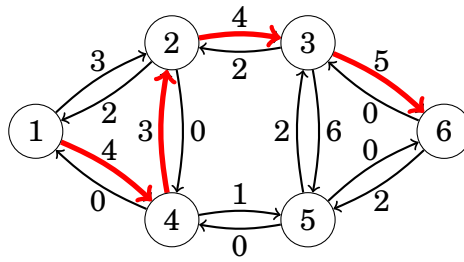
After choosing the path, the flow increases by x units, where x is the smallest edge weight on the path. In addition, the weight of each edge on the path decreases by x and the weight of each reverse edge increases by x .

In the above path, the weights of the edges are 5, 6, 8 and 2. The smallest weight is 2, so the flow increases by 2 and the new graph is as follows:



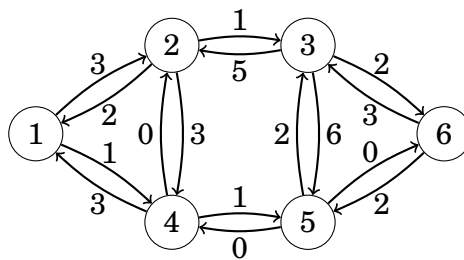
The idea is that increasing the flow decreases the amount of flow that can go through the edges in the future. On the other hand, it is possible to cancel flow later using the reverse edges of the graph if it turns out that it would be beneficial to route the flow in another way.

The algorithm increases the flow as long as there is a path from the source to the sink through positive-weight edges. In the present example, our next path can be as follows:

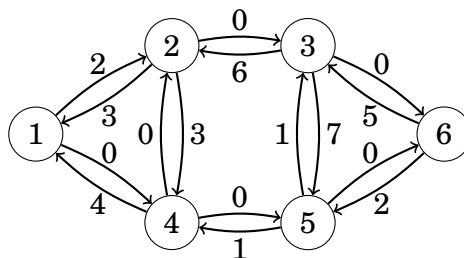


The minimum edge weight on this path is 3, so the path increases the flow by 3, and the total flow after processing the path is 5.

The new graph will be as follows:



We still need two more rounds before reaching the maximum flow. For example, we can choose the paths $1 \rightarrow 2 \rightarrow 3 \rightarrow 6$ and $1 \rightarrow 4 \rightarrow 5 \rightarrow 3 \rightarrow 6$. Both paths increase the flow by 1, and the final graph is as follows:



It is not possible to increase the flow anymore, because there is no path from the source to the sink with positive edge weights. Hence, the algorithm terminates and the maximum flow is 7.

Finding paths

The Ford–Fulkerson algorithm does not specify how we should choose the paths that increase the flow. In any case, the algorithm will terminate sooner or later and correctly find the maximum flow. However, the efficiency of the algorithm depends on the way the paths are chosen.

A simple way to find paths is to use depth-first search. Usually, this works well, but in the worst case, each path only increases the flow by 1 and the algorithm is slow. Fortunately, we can avoid this situation by using one of the following techniques:

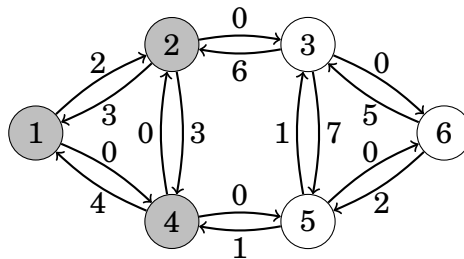
The **Edmonds–Karp algorithm** [18] chooses each path so that the number of edges on the path is as small as possible. This can be done by using breadth-first search instead of depth-first search for finding paths. It can be proven that this guarantees that the flow increases quickly, and the time complexity of the algorithm is $O(m^2n)$.

The **scaling algorithm** [2] uses depth-first search to find paths where each edge weight is at least a threshold value. Initially, the threshold value is some large number, for example the sum of all edge weights of the graph. Always when a path cannot be found, the threshold value is divided by 2. The time complexity of the algorithm is $O(m^2 \log c)$, where c is the initial threshold value.

In practice, the scaling algorithm is easier to implement, because depth-first search can be used for finding paths. Both algorithms are efficient enough for problems that typically appear in programming contests.

Minimum cuts

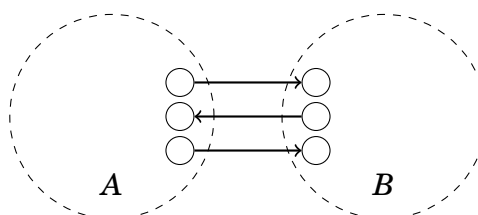
It turns out that once the Ford–Fulkerson algorithm has found a maximum flow, it has also determined a minimum cut. Let A be the set of nodes that can be reached from the source using positive-weight edges. In the example graph, A contains nodes 1, 2 and 4:



Now the minimum cut consists of the edges of the original graph that start at some node in A , end at some node outside A , and whose capacity is fully used in the maximum flow. In the above graph, such edges are $2 \rightarrow 3$ and $4 \rightarrow 5$, that correspond to the minimum cut $6 + 1 = 7$.

Why is the flow produced by the algorithm maximum and why is the cut minimum? The reason is that a graph cannot contain a flow whose size is larger than the weight of any cut of the graph. Hence, always when a flow and a cut are equally large, they are a maximum flow and a minimum cut.

Let us consider any cut of the graph such that the source belongs to A , the sink belongs to B and there are some edges between the sets:



The size of the cut is the sum of the edges that go from A to B . This is an upper bound for the flow in the graph, because the flow has to proceed from A to B . Thus, the size of a maximum flow is smaller than or equal to the size of any cut in the graph.

On the other hand, the Ford–Fulkerson algorithm produces a flow whose size is *exactly* as large as the size of a cut in the graph. Thus, the flow has to be a maximum flow and the cut has to be a minimum cut.

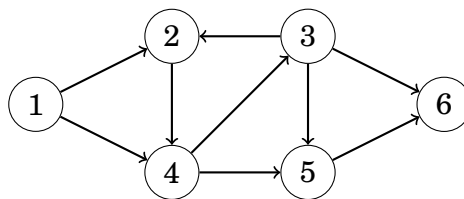
20.2. Disjoint paths

Many graph problems can be solved by reducing them to the maximum flow problem. Our first example of such a problem is as follows: we are given a directed graph with a source and a sink, and our task is to find the maximum number of disjoint paths from the source to the sink.

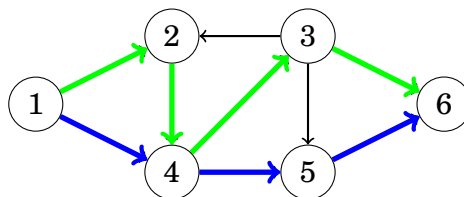
Edge-disjoint paths

We will first focus on the problem of finding the maximum number of **edge-disjoint paths** from the source to the sink. This means that we should construct a set of paths such that each edge appears in at most one path.

For example, consider the following graph:



In this graph, the maximum number of edge-disjoint paths is 2. We can choose the paths $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 6$ and $1 \rightarrow 4 \rightarrow 5 \rightarrow 6$ as follows:



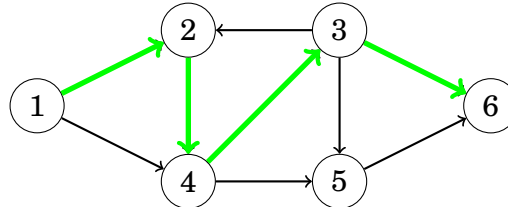
It turns out that the maximum number of edge-disjoint paths equals the maximum flow of the graph, assuming that the capacity of each edge is one. After the maximum flow has been constructed, the edge-disjoint paths can be found greedily by following paths from the source to the sink.

Node-disjoint paths

Let us now consider another problem: finding the maximum number of **node-disjoint paths** from the source to the sink. In this problem, every node, except for

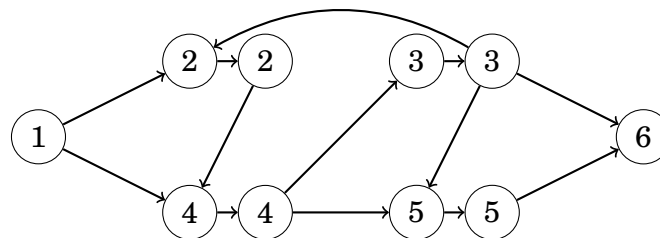
the source and sink, may appear in at most one path. The number of node-disjoint paths may be smaller than the number of edge-disjoint paths.

For example, in the previous graph, the maximum number of node-disjoint paths is 1:

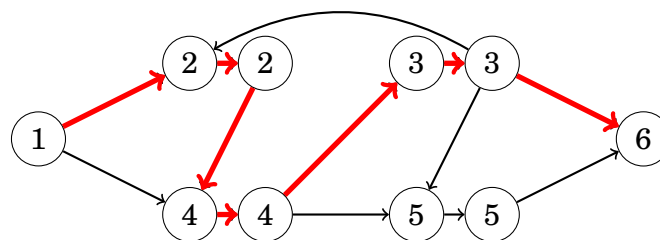


We can reduce also this problem to the maximum flow problem. Since each node can appear in at most one path, we have to limit the flow that goes through the nodes. A standard method for this is to divide each node into two nodes such that the first node has the incoming edges of the original node, the second node has the outgoing edges of the original node, and there is a new edge from the first node to the second node.

In our example, the graph becomes as follows:



The maximum flow for the graph is as follows:



Thus, the maximum number of node-disjoint paths from the source to the sink is 1.

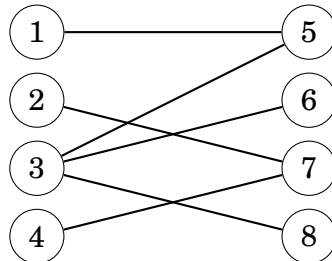
20.3. Maximum matchings

The **maximum matching** problem asks to find a maximum-size set of node pairs in an undirected graph such that each pair is connected with an edge and each node belongs to at most one pair.

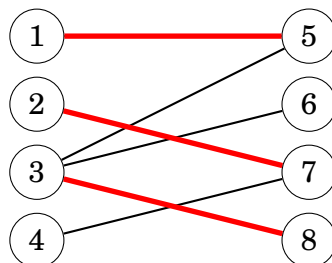
There are polynomial algorithms for finding maximum matchings in general graphs [17], but such algorithms are complex and rarely seen in programming contests. However, in bipartite graphs, the maximum matching problem is much easier to solve, because we can reduce it to the maximum flow problem.

Finding maximum matchings

The nodes of a bipartite graph can be always divided into two groups such that all edges of the graph go from the left group to the right group. For example, in the following bipartite graph, the groups are $\{1, 2, 3, 4\}$ and $\{5, 6, 7, 8\}$.

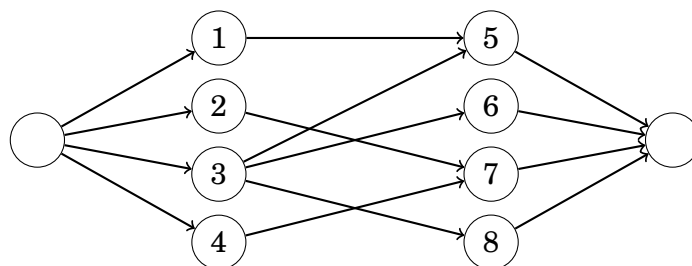


The size of a maximum matching of this graph is 3:

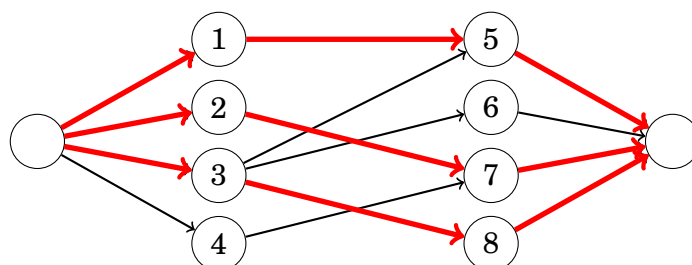


We can reduce the bipartite maximum matching problem to the maximum flow problem by adding two new nodes to the graph: a source and a sink. We also add edges from the source to each left node and from each right node to the sink. After this, the size of a maximum flow in the graph equals the size of a maximum matching in the original graph.

For example, the reduction for the above graph is as follows:



The maximum flow of this graph is as follows:

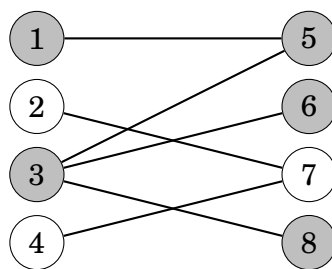


Hall's theorem

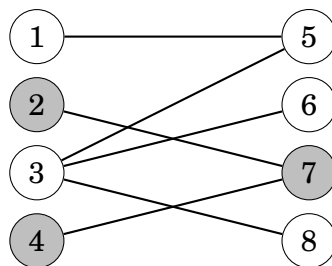
Hall's theorem can be used to find out whether a bipartite graph has a matching that contains all left or right nodes. If the number of left and right nodes is the same, Hall's theorem tells us if it is possible to construct a **perfect matching** that contains all nodes of the graph.

Assume that we want to find a matching that contains all left nodes. Let X be any set of left nodes and let $f(X)$ be the set of their neighbors. According to Hall's theorem, a matching that contains all left nodes exists exactly when for each X , the condition $|X| \leq |f(X)|$ holds.

Let us study Hall's theorem in the example graph. First, let $X = \{1, 3\}$ which yields $f(X) = \{5, 6, 8\}$:



The condition of Hall's theorem holds, because $|X| = 2$ and $|f(X)| = 3$. Next, let $X = \{2, 4\}$ which yields $f(X) = \{7\}$:



In this case, $|X| = 2$ and $|f(X)| = 1$, so the condition of Hall's theorem does not hold. This means that it is not possible to form a perfect matching for the graph. This result is not surprising, because we already know that the maximum matching of the graph is 3 and not 4.

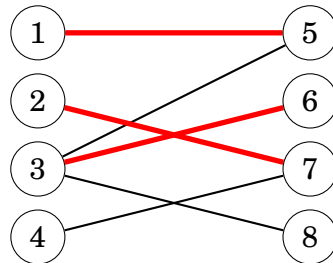
If the condition of Hall's theorem does not hold, the set X provides an explanation *why* we cannot form such a matching. Since X contains more nodes than $f(X)$, there are no pairs for all nodes in X . For example, in the above graph, both nodes 2 and 4 should be connected with node 7 which is not possible.

König's theorem

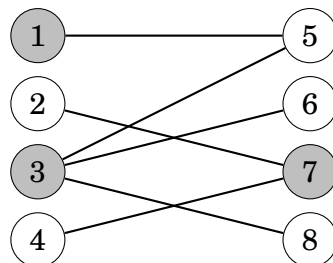
A **minimum node cover** of a graph is a minimum set of nodes such that each edge of the graph has at least one endpoint in the set. In a general graph, finding a minimum node cover is a NP-hard problem. However, if the graph is bipartite, **König's theorem** tells us that the size of a minimum node cover and

the size of a maximum matching are always equal. Thus, we can calculate the size of a minimum node cover using a maximum flow algorithm.

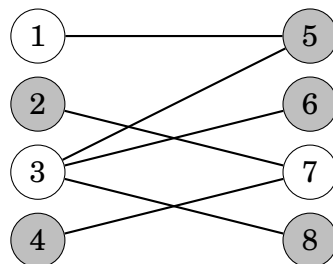
Let us consider the following graph with a maximum matching of size 3:



Now König's theorem tells us that the size of a minimum node cover is also 3. Such a cover can be constructed as follows:



The nodes that do *not* belong to a minimum node cover form a **maximum independent set**. This is the largest possible set of nodes such that no two nodes in the set are connected with an edge. Once again, finding a maximum independent set in a general graph is a NP-hard problem, but in a bipartite graph we can use König's theorem to solve the problem efficiently. In the example graph, the maximum independent set is as follows:

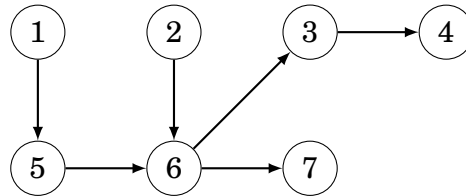


20.4. Path covers

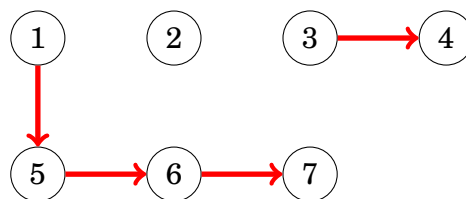
A **path cover** is a set of paths in a graph such that each node of the graph belongs to at least one path. It turns out that in directed, acyclic graphs, we can reduce the problem of finding a minimum path cover to the problem of finding a maximum flow in another graph.

Node-disjoint path cover

In a **node-disjoint path cover**, each node belongs to exactly one path. As an example, consider the following graph:



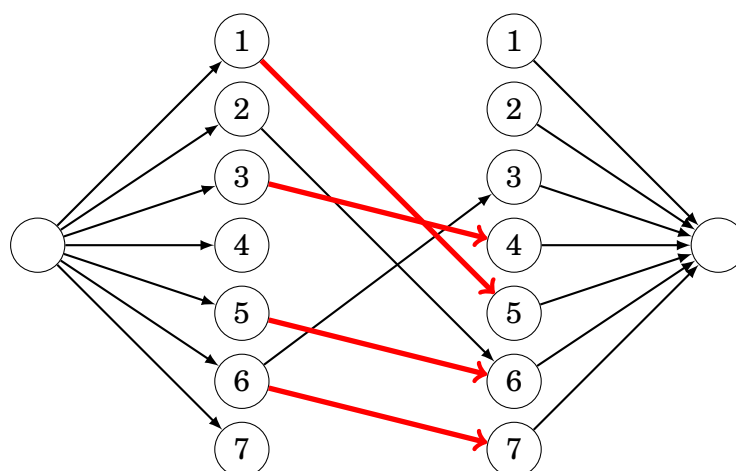
A minimum node-disjoint path cover of this graph consists of three paths. For example, we can choose the following paths:



Note that one of the paths only contains node 2, so it is possible that a path does not contain any edges.

We can find a minimum node-disjoint path cover by constructing a *matching graph* where each node of the original graph is represented by two nodes: a left node and a right node. There is an edge from a left node to a right node if there is such an edge in the original graph. In addition, the matching graph contains a source and a sink, and there are edges from the source to all left nodes and from all right nodes to the sink.

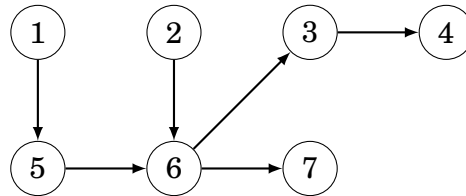
A maximum matching in the resulting graph corresponds to a minimum node-disjoint path cover in the original graph. For example, the following matching graph for the above graph contains a maximum matching of size 4:



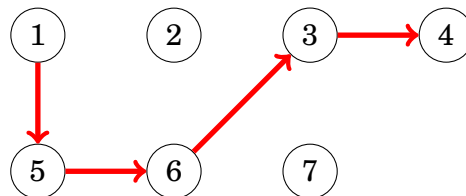
Each edge in the maximum matching of the matching graph corresponds to an edge in the minimum node-disjoint path cover of the original graph. Thus, the size of the minimum node-disjoint path cover is $n - c$, where n is the number of nodes in the original graph and c is the size of the maximum matching.

General path cover

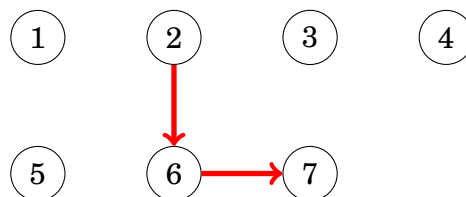
A **general path cover** is a path cover where a node can belong to more than one path. A minimum general path cover may be smaller than a minimum node-disjoint path cover, because a node can be used multiple times in paths. Consider again the following graph:



The minimum general path cover of this graph consists of two paths. For example, the first path may be as follows:

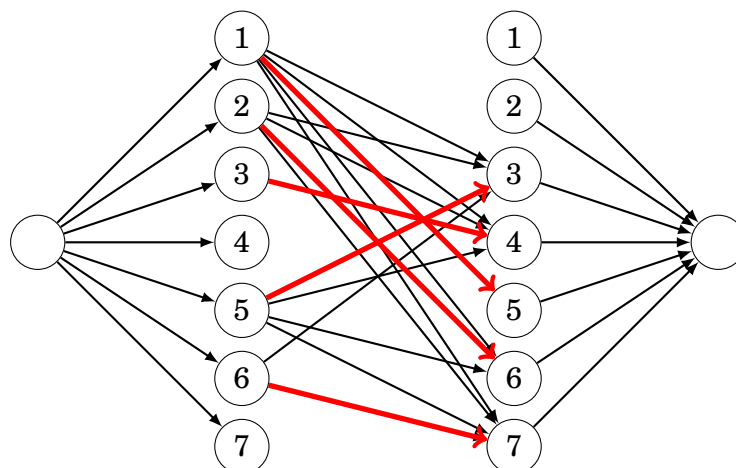


And the second path may be as follows:



A minimum general path cover can be found almost like a minimum node-disjoint path cover. It suffices to add some new edges to the matching graph so that there is an edge $a \rightarrow b$ always when there is a path from a to b in the original graph (possibly through several edges).

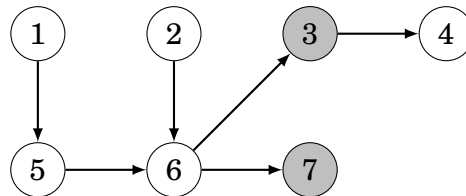
The matching graph for the above graph is as follows:



Dilworth's theorem

An **antichain** is a set of nodes of a graph such that there is no path from any node to another node using the edges of the graph. **Dilworth's theorem** states that in a directed acyclic graph, the size of a minimum general path cover equals the size of a maximum antichain.

For example, nodes 3 and 7 form an antichain in the following graph:



This is a maximum antichain, because it is not possible to construct any antichain that would contain three nodes. We have seen before that the size of a minimum general path cover of this graph consists of two paths.

Bibliografia

- [1] A. V. Aho, J. E. Hopcroft and J. Ullman. *Data Structures and Algorithms*, Addison-Wesley, 1983.
- [2] R. K. Ahuja and J. B. Orlin. Distance directed augmenting path algorithms for maximum flow and parametric maximum flow problems. *Naval Research Logistics*, 38(3):413–430, 1991.
- [3] A. M. Andrew. Another efficient algorithm for convex hulls in two dimensions. *Information Processing Letters*, 9(5):216–219, 1979.
- [4] B. Aspvall, M. F. Plass and R. E. Tarjan. A linear-time algorithm for testing the truth of certain quantified boolean formulas. *Information Processing Letters*, 8(3):121–123, 1979.
- [5] R. Bellman. On a routing problem. *Quarterly of Applied Mathematics*, 16(1):87–90, 1958.
- [6] M. Beck, E. Pine, W. Tarrat and K. Y. Jensen. New integer representations as the sum of three cubes. *Mathematics of Computation*, 76(259):1683–1690, 2007.
- [7] M. A. Bender and M. Farach-Colton. The LCA problem revisited. In *Latin American Symposium on Theoretical Informatics*, 88–94, 2000.
- [8] J. Bentley. *Programming Pearls*. Addison-Wesley, 1999 (2nd edition).
- [9] J. Bentley and D. Wood. An optimal worst case algorithm for reporting intersections of rectangles. *IEEE Transactions on Computers*, C-29(7):571–577, 1980.
- [10] C. L. Bouton. Nim, a game with a complete mathematical theory. *Annals of Mathematics*, 3(1/4):35–39, 1901.
- [11] Croatian Open Competition in Informatics, <http://hsin.hr/coci/>
- [12] Codeforces: On "Mo's algorithm", <http://codeforces.com/blog/entry/20032>
- [13] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein. *Introduction to Algorithms*, MIT Press, 2009 (3rd edition).
- [14] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1):269–271, 1959.

- [15] K. Diks et al. *Looking for a Challenge? The Ultimate Problem Set from the University of Warsaw Programming Competitions*, University of Warsaw, 2012.
- [16] M. Dima and R. Ceterchi. Efficient range minimum queries using binary indexed trees. *Olympiad in Informatics*, 9(1):39–44, 2015.
- [17] J. Edmonds. Paths, trees, and flowers. *Canadian Journal of Mathematics*, 17(3):449–467, 1965.
- [18] J. Edmonds and R. M. Karp. Theoretical improvements in algorithmic efficiency for network flow problems. *Journal of the ACM*, 19(2):248–264, 1972.
- [19] S. Even, A. Itai and A. Shamir. On the complexity of time table and multi-commodity flow problems. *16th Annual Symposium on Foundations of Computer Science*, 184–193, 1975.
- [20] D. Fanding. A faster algorithm for shortest-path – SPFA. *Journal of Southwest Jiaotong University*, 2, 1994.
- [21] P. M. Fenwick. A new data structure for cumulative frequency tables. *Software: Practice and Experience*, 24(3):327–336, 1994.
- [22] J. Fischer and V. Heun. Theoretical and practical improvements on the RMQ-problem, with applications to LCA and LCE. In *Annual Symposium on Combinatorial Pattern Matching*, 36–48, 2006.
- [23] R. W. Floyd Algorithm 97: shortest path. *Communications of the ACM*, 5(6):345, 1962.
- [24] L. R. Ford. Network flow theory. RAND Corporation, Santa Monica, California, 1956.
- [25] L. R. Ford and D. R. Fulkerson. Maximal flow through a network. *Canadian Journal of Mathematics*, 8(3):399–404, 1956.
- [26] R. Freivalds. Probabilistic machines can use less running time. In *IFIP congress*, 839–842, 1977.
- [27] F. Le Gall. Powers of tensors and fast matrix multiplication. In *Proceedings of the 39th International Symposium on Symbolic and Algebraic Computation*, 296–303, 2014.
- [28] M. R. Garey and D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*, W. H. Freeman and Company, 1979.
- [29] Google Code Jam Statistics (2017), <https://www.go-hero.net/jam/17>
- [30] A. Grønlund and S. Pettie. Threesomes, degenerates, and love triangles. In *Proceedings of the 55th Annual Symposium on Foundations of Computer Science*, 621–630, 2014.

- [31] P. M. Grundy. Mathematics and games. *Eureka*, 2(5):6–8, 1939.
- [32] D. Gusfield. *Algorithms on Strings, Trees and Sequences: Computer Science and Computational Biology*, Cambridge University Press, 1997.
- [33] S. Halim and F. Halim. *Competitive Programming 3: The New Lower Bound of Programming Contests*, 2013.
- [34] M. Held and R. M. Karp. A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics*, 10(1):196–210, 1962.
- [35] C. Hierholzer and C. Wiener. Über die Möglichkeit, einen Linienzug ohne Wiederholung und ohne Unterbrechung zu umfahren. *Mathematische Annalen*, 6(1), 30–32, 1873.
- [36] C. A. R. Hoare. Algorithm 64: Quicksort. *Communications of the ACM*, 4(7):321, 1961.
- [37] C. A. R. Hoare. Algorithm 65: Find. *Communications of the ACM*, 4(7):321–322, 1961.
- [38] J. E. Hopcroft and J. D. Ullman. A linear list merging algorithm. Technical report, Cornell University, 1971.
- [39] E. Horowitz and S. Sahni. Computing partitions with applications to the knapsack problem. *Journal of the ACM*, 21(2):277–292, 1974.
- [40] D. A. Huffman. A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, 40(9):1098–1101, 1952.
- [41] The International Olympiad in Informatics Syllabus, <https://people.ksp.sk/~misof/ioi-syllabus/>
- [42] R. M. Karp and M. O. Rabin. Efficient randomized pattern-matching algorithms. *IBM Journal of Research and Development*, 31(2):249–260, 1987.
- [43] P. W. Kasteleyn. The statistics of dimers on a lattice: I. The number of dimer arrangements on a quadratic lattice. *Physica*, 27(12):1209–1225, 1961.
- [44] C. Kent, G. M. Landau and M. Ziv-Ukelson. On the complexity of sparse exon assembly. *Journal of Computational Biology*, 13(5):1013–1027, 2006.
- [45] J. Kleinberg and É. Tardos. *Algorithm Design*, Pearson, 2005.
- [46] D. E. Knuth. *The Art of Computer Programming. Volume 2: Seminumerical Algorithms*, Addison–Wesley, 1998 (3rd edition).
- [47] D. E. Knuth. *The Art of Computer Programming. Volume 3: Sorting and Searching*, Addison–Wesley, 1998 (2nd edition).

- [48] J. B. Kruskal. On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical Society*, 7(1):48–50, 1956.
- [49] V. I. Levenshtein. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet physics doklady*, 10(8):707–710, 1966.
- [50] M. G. Main and R. J. Lorentz. An $O(n \log n)$ algorithm for finding all repetitions in a string. *Journal of Algorithms*, 5(3):422–432, 1984.
- [51] J. Pachocki and J. Radoszewski. Where to use and how not to use polynomial string hashing. *Olympiads in Informatics*, 7(1):90–100, 2013.
- [52] I. Parberry. An efficient algorithm for the Knight’s tour problem. *Discrete Applied Mathematics*, 73(3):251–260, 1997.
- [53] D. Pearson. A polynomial-time algorithm for the change-making problem. *Operations Research Letters*, 33(3):231–234, 2005.
- [54] R. C. Prim. Shortest connection networks and some generalizations. *Bell System Technical Journal*, 36(6):1389–1401, 1957.
- [55] 27-Queens Puzzle: Massively Parallel Enumeration and Solution Counting. <https://github.com/preusser/q27>
- [56] M. I. Shamos and D. Hoey. Closest-point problems. In *Proceedings of the 16th Annual Symposium on Foundations of Computer Science*, 151–162, 1975.
- [57] M. Sharir. A strong-connectivity algorithm and its applications in data flow analysis. *Computers & Mathematics with Applications*, 7(1):67–72, 1981.
- [58] S. S. Skiena. *The Algorithm Design Manual*, Springer, 2008 (2nd edition).
- [59] S. S. Skiena and M. A. Revilla. *Programming Challenges: The Programming Contest Training Manual*, Springer, 2003.
- [60] SZKOpuł, <https://szkopul.edu.pl/>
- [61] R. Sprague. Über mathematische Kampfspiele. *Tohoku Mathematical Journal*, 41:438–444, 1935.
- [62] P. Stańczyk. *Algorytmika praktyczna w konkursach Informatycznych*, MSc thesis, University of Warsaw, 2006.
- [63] V. Strassen. Gaussian elimination is not optimal. *Numerische Mathematik*, 13(4):354–356, 1969.
- [64] R. E. Tarjan. Efficiency of a good but not linear set union algorithm. *Journal of the ACM*, 22(2):215–225, 1975.
- [65] R. E. Tarjan. Applications of path compression on balanced trees. *Journal of the ACM*, 26(4):690–715, 1979.

- [66] R. E. Tarjan and U. Vishkin. Finding biconnected components and computing tree functions in logarithmic parallel time. In *Proceedings of the 25th Annual Symposium on Foundations of Computer Science*, 12–20, 1984.
- [67] H. N. V. Temperley and M. E. Fisher. Dimer problem in statistical mechanics – an exact result. *Philosophical Magazine*, 6(68):1061–1063, 1961.
- [68] USA Computing Olympiad, <http://www.usaco.org/>
- [69] H. C. von Warnsdorf. *Des Rösselsprunges einfachste und allgemeinste Lösung*. Schmalkalden, 1823.
- [70] S. Warshall. A theorem on boolean matrices. *Journal of the ACM*, 9(1):11–12, 1962.

